

Pengembangan Arsitektur Platform DataOps dan MLOps Terintegrasi pada Kubernetes dengan Pemanfaatan *Open Source Tools*

Laporan Tugas Akhir

Oleh

Bryan P. Hutagalung
18222130



**PROGRAM STUDI SISTEM DAN TEKNOLOGI INFORMASI
SEKOLAH TEKNIK ELEKTRO DAN INFORMATIKA
INSTITUT TEKNOLOGI BANDUNG**

Juni 2026

LEMBAR PENGESAHAN

Pengembangan Arsitektur Platform DataOps dan MLOps Terintegrasi pada Kubernetes dengan Pemanfaatan *Open Source Tools*

Laporan Tugas Akhir

Oleh

**Bryan P. Hutagalung
18222130**

Program Studi Sistem dan Teknologi Informasi
Sekolah Teknik Elektro dan Informatika
Institut Teknologi Bandung

Laporan Tugas Akhir ini telah disetujui dan disahkan
di Bandung, pada tanggal 12 Juni 2026

Pembimbing

Achmad Imam Kistijantoro, S.T, M.Sc., Ph.D.

NIP. 197308092006041001

PERNYATAAN ORISINALITAS

Saya yang bertanda tangan di bawah ini menyatakan dengan sesungguhnya bahwa:

1. Tugas Akhir ini adalah hasil karya saya sendiri yang dibuat dengan sebenarnya sesuai dengan kaidah ilmiah dan etika akademik.
2. Semua sumber rujukan dan data yang saya gunakan dalam penyusunan laporan tugas akhir ini, baik yang berupa kutipan langsung maupun tidak langsung, telah saya cantumkan sumbernya dengan baik dan benar sesuai dengan ketentuan penulisan laporan tugas akhir.
3. Isi laporan ini belum pernah diajukan pada program pendidikan di institusi mana pun.

Apabila di kemudian hari terbukti bahwa laporan ini melanggar hal-hal di atas, saya bersedia menerima sanksi sesuai dengan peraturan yang berlaku di Institut Teknologi Bandung.

Bandung, 12 Juni 2026

Bryan P. Hutagalung

NIM: 18222130

PERNYATAAN PENGGUNAAN AI

Saya yang bertanda tangan di bawah ini menyatakan bahwa dalam penulisan dokumen Tugas Akhir ini, saya menggunakan alat bantu kecerdasan artifisial (AI) untuk beberapa aspek tertentu di dalam proses penulisan, seperti yang tercantum dalam tabel berikut.

No.	Jenis	Alat Bantu	Tingkat	Tempat
1	Memeriksa ejaan dan tata bahasa	Grammarly	Tinggi	Semua bab
2	Pembuatan teks	Tidak ada	Tidak ada	Tidak ada
3	Pembuatan grafik, gambar, atau ilustrasi	Tidak ada	Tidak ada	Tidak ada
4	Pencarian informasi atau referensi	Tidak ada	Tidak ada	Tidak ada
5	Penyusunan bahan diskusi	Tidak ada	Tidak ada	Tidak ada
6	Penyusunan kode program	GitHub Copilot	Sedang	Lampiran A (manifest platform)
7	Penyusunan formula atau perhitungan matematis	Tidak ada	Tidak ada	Tidak ada
8	Penyusunan konsep desain atau algoritma	Tidak ada	Tidak ada	Tidak ada
9	Penyusunan analisis data	Tidak ada	Tidak ada	Tidak ada
10	Penyusunan kesimpulan atau rekomendasi	Tidak ada	Tidak ada	Tidak ada

Bandung, 12 Juni 2026

Bryan P. Hutagalung

NIM: 18222130

ABSTRAK

Pengembangan Arsitektur Platform DataOps dan MLOps Terintegrasi pada Kubernetes dengan Pemanfaatan Open Source Tools

oleh

Bryan P. Hutagalung

18222130

Pengembangan model *machine learning* ke tahap produksi masih bersinggungan dengan fragmentasi *tool* dan pemisahan kerja antara tim data dan tim model. Praktik DataOps memperkuat kualitas dan kecepatan alur data, sementara MLOps memperkuat kehandalan siklus hidup model, namun kedua praktik tersebut sering tumbuh terpisah sehingga *lineage*, deteksi *drift*, dan penyajian fitur antara mode *batch* dan *streaming* tidak konsisten lintas siklus. Penelitian ini mengembangkan arsitektur platform yang mengintegrasikan DataOps dan MLOps pada Kubernetes dengan memanfaatkan perangkat *open source*, sehingga satu bidang kendali menyatukan ingestasi data, pemrosesan, penyimpanan fitur, pelatihan, penyajian model, observabilitas, dan tata kelola data. Metode yang digunakan adalah *Design Science Research Methodology* (DSRM) yang melingkupi identifikasi masalah, penyusunan tujuan artefak, perancangan tujuh lapis arsitektur, implementasi berbasis GitOps, serta uji jalan menggunakan *use case* pasar aset kripto sebagai instans verifikasi. Arsitektur ini menempatkan Kubernetes sebagai bidang orkestrasi, Apache Kafka dan Apache Flink untuk *streaming*, Apache Spark untuk pemrosesan *batch*, Feast sebagai *feature store* ganda dengan ClickHouse *offline*, Valkey sebagai *online store* fitur tabular berlatensi rendah, dan Qdrant sebagai indeks vektor terpisah dengan pencarian HNSW, MLflow dan Kubeflow untuk siklus hidup model, KServe untuk penyajian, serta DataHub untuk katalog dan *lineage*. Mekanisme deteksi *drift* berbasis indikator *Population Stability Index* dan Kolmogorov-Smirnov memicu *retraining* secara otomatis melalui Argo CronWorkflow, sementara *point-in-time correctness* dijaga oleh Feast pada fase pelatihan dan penyajian *online*. Argo CD menyatukan praktik GitOps untuk reproduksibilitas *deployment* dari satu sumber kebenaran berbasis Git, dan Prometheus, Grafana, OpenTelemetry, Loki, serta Grafana Tempo menutup kebutuhan observabilitas pada tiga lapis metrik, log, dan *trace*. Hasil rancangan dan implementasi inti menunjukkan bahwa platform berhasil menyatukan komponen DataOps dan MLOps dalam satu siklus yang berkesinambungan. Angka evaluasi kuantitatif akan dilengkapi pada laporan akhir setelah seluruh tahap pengujian beban dan kualitas selesai.

Kata kunci: DataOps, MLOps, Kubernetes, Feature Store, Drift Detection, GitOps, Open Source.

KATA PENGANTAR

Puji syukur penulis panjatkan ke hadirat Tuhan Yang Maha Esa karena atas rahmat dan karunia-Nya, penulis dapat menyelesaikan penulisan Laporan Tugas Akhir yang berjudul “Pengembangan Arsitektur Platform DataOps dan MLOps Terintegrasi pada Kubernetes dengan Pemanfaatan *Open Source Tools*”. Laporan ini disusun sebagai salah satu syarat untuk menyelesaikan program Sarjana di Program Studi Sistem dan Teknologi Informasi, Sekolah Teknik Elektro dan Informatika, Institut Teknologi Bandung.

Penulis menyampaikan terima kasih kepada Bapak Achmad Imam Kistijantoro, S.T, M.Sc., Ph.D. selaku dosen pembimbing yang telah memberikan arahan, masukan, dan diskusi mendalam sepanjang proses pengerjaan Tugas Akhir. Penulis juga mengucapkan terima kasih kepada seluruh dosen Program Studi Sistem dan Teknologi Informasi ITB atas ilmu yang diberikan selama masa studi, serta kepada keluarga dan rekan-rekan yang senantiasa memberikan dukungan moral selama proses penulisan.

Penulis menyadari bahwa laporan ini masih memiliki kekurangan. Oleh karena itu, kritik dan saran yang membangun sangat diharapkan untuk perbaikan di masa mendatang. Semoga laporan ini bermanfaat bagi pembaca dan pengembangan platform DataOps dan MLOps di lingkungan akademik maupun industri.

Bandung, 12 Juni 2026

Penulis

Bryan P. Hutagalung

DAFTAR ISI

LEMBAR PENGESAHAN	i
PERNYATAAN ORISINALITAS	ii
PERNYATAAN PENGGUNAAN AI	iii
ABSTRAK	iv
KATA PENGANTAR	v
DAFTAR ISI	vii
DAFTAR GAMBAR	xi
DAFTAR TABEL	xiii
DAFTAR PERSAMAAN	xv
DAFTAR ALGORITMA	xvii
DAFTAR <i>LISTING</i>	xix
DAFTAR SIMBOL	xxi
DAFTAR SINGKATAN	xxiii
I PENDAHULUAN	1
I.1 Latar Belakang	1
I.2 Rumusan Masalah	3
I.3 Tujuan	5
I.4 Batasan Masalah	6
I.5 Metodologi	7
I.6 Sistematika Penulisan	8
II STUDI LITERATUR	11
II.1 Cakupan Studi Literatur	11
II.2 DataOps dan MLOps	11
II.2.1 DataOps	11
II.2.2 MLOps	12
II.2.3 Tingkat Kematangan MLOps	13
II.2.4 Integrasi DataOps dan MLOps	15
II.3 Kubernetes dan Orkestrasi <i>Cloud-Native</i>	16
II.4 Manajemen Data: Ingestasi, <i>Streaming</i> , dan <i>Batch</i>	18
II.4.1 Pola Lambda dan Kappa	18
II.4.2 Apache Kafka	19
II.4.3 Apache Flink	20
II.4.4 Apache Spark dan dbt	20
II.4.5 Format Tabel <i>Lakehouse</i>	21
II.4.6 Penyimpanan Objek dan Versi Data	22
II.4.7 Penyimpanan Relasional dan Pencarian	22
II.5 Layanan Fitur (<i>Feature Store</i>)	23
II.5.1 Konsep dan Peran	23
II.5.2 Penyimpanan <i>Offline</i> dan <i>Online</i>	23
II.5.3 Pencarian Vektor dan HNSW	24
II.5.4 <i>Point-in-Time Correctness</i>	25
II.6 Siklus Hidup Model	26

II.6.1	Eksperimen dan <i>Tracking</i>	26
II.6.2	Orkestrasi Pelatihan	27
II.6.3	Eksperimen Interaktif dan <i>Tuning</i>	27
II.6.4	Penyajian Model	28
II.7	Tata Kelola Data: Katalog, <i>Lineage</i> , dan Kualitas	29
II.8	Deteksi <i>Drift</i>	29
II.9	Observabilitas <i>Cloud-Native</i>	30
II.10	GitOps dan <i>Continuous Delivery</i>	31
II.11	Keamanan Platform: Identitas, Rahasia, <i>Mesh</i> , dan Kebijakan	32
II.12	Penelitian Terkait dan Posisi Penelitian	35
III	ANALISIS MASALAH	43
III.1	Analisis Kondisi Saat Ini	43
III.1.1	Beban Konfigurasi Platform dari Nol	43
III.1.2	Biaya Berlangganan Layanan Terkelola	44
III.1.3	Efek Domino Fragmentasi Lintas Peran	45
III.2	Analisis Kebutuhan	48
III.2.1	Identifikasi Masalah Pengguna	48
III.2.2	Kebutuhan Fungsional	49
III.2.3	Kebutuhan Nonfungsional	52
III.3	Analisis Pemilihan Solusi	55
III.3.1	Alternatif Solusi	56
III.3.2	Analisis Penentuan Solusi	57
IV	PERANCANGAN ARSITEKTUR PLATFORM DATAOPS DAN MLO-PS	61
IV.1	Gambaran Umum Platform	61
IV.2	Perancangan Arsitektur Terintegrasi	63
IV.2.1	Pandangan per Peran Pengguna	63
IV.2.2	Lapisan Infrastruktur dan Bidang Kendali	68
IV.2.3	Lapisan Ingestasi dan Pemrosesan Data	69
IV.2.4	Lapisan Siklus Hidup Model dan Penyajian	69
IV.2.5	Lapisan Tata Kelola dan Observabilitas	70
IV.3	Perancangan Sub-sistem Tata Kelola Data	70
IV.3.1	Katalog Metadata dan <i>Lineage</i>	71
IV.3.2	Kontrak Data dan Kualitas	71
IV.3.3	Identitas dan Kontrol Akses	72
IV.4	Perancangan Sub-sistem Deteksi <i>Drift</i> dan <i>Continuous Training</i>	73
IV.4.1	Pengukuran <i>Drift</i>	73
IV.4.2	Alur Pelatihan Ulang Otomatis	73
IV.4.3	Mekanisme Penanganan Kasus Khusus	74
IV.5	Perancangan Sub-sistem Layanan Fitur <i>Dual-Store</i>	74
IV.5.1	Penyimpanan <i>Offline</i> dan <i>Online</i>	75
IV.5.2	Dukungan <i>Vector Embeddings</i> dan HNSW	75
IV.5.3	Jaminan <i>Point-in-Time Correctness</i>	76
IV.5.4	Aliran Materialisasi Fitur	77
IV.6	Alur Kerja <i>End-to-End</i>	77

V	IMPLEMENTASI ARSITEKTUR PLATFORM DATAOPS DAN MLO-PS	79
V.1	Lingkungan Implementasi	79
V.1.1	Konfigurasi Dasar <i>Cluster</i>	80
V.1.2	Manajemen Rahasia dan Identitas	80
V.1.3	Pola GitOps	81
V.2	Implementasi Arsitektur Terintegrasi	81
V.2.1	Lapisan Ingestasi	82
V.2.2	Lapisan Pemrosesan	82
V.2.3	Lapisan Penyimpanan	83
V.2.4	Lapisan Siklus Hidup Model dan Penyajian	83
V.2.5	Lapisan Observabilitas	84
V.3	Implementasi Sub-sistem Tata Kelola Data	84
V.3.1	Katalog Metadata dan <i>Lineage</i>	85
V.3.2	Kontrak Data dan Kualitas	85
V.3.3	Kontrol Akses	86
V.4	Implementasi Sub-sistem Deteksi <i>Drift</i> dan <i>Continuous Training</i>	86
V.4.1	Detektor <i>Drift</i>	86
V.4.2	<i>Control Loop</i> Pelatihan Ulang	87
V.4.3	Penanganan Kasus Khusus	88
V.5	Implementasi Sub-sistem Layanan Fitur <i>Dual-Store</i>	88
V.5.1	Konfigurasi Feast	88
V.5.2	Dukungan <i>Vector Embeddings</i>	89
V.5.3	Materialisasi dan <i>Point-in-Time Correctness</i>	89
V.6	Verifikasi Implementasi	90
V.6.1	Lingkup Verifikasi	90
V.6.2	Pengamatan Hasil Awal	91
VI	EVALUASI	93
VI.1	Metode Evaluasi	93
VI.1.1	Instans Verifikasi dan Lingkungan Uji	94
VI.1.2	Evaluasi Tujuan T-1: Arsitektur Terintegrasi dan GitOps	95
VI.1.3	Evaluasi Tujuan T-2: Tata Kelola Data	95
VI.1.4	Evaluasi Tujuan T-3: Deteksi <i>Drift</i> dan Pelatihan Ulang	96
VI.1.5	Evaluasi Tujuan T-4: Layanan Fitur <i>Dual-Store</i>	97
VI.1.6	Matriks Cakupan dan Uji Penerimaan	97
VI.2	Hasil Evaluasi	100
VI.3	Pembahasan Hasil Evaluasi	102
VII	PENUTUP	105
VII.1	Kesimpulan	105
VII.2	Saran	105
Lampiran A	DAFTAR ARTEFAK PLATFORM	121
A.1	Tata Letak Repositori Platform	121
A.2	Tata Letak Repositori Use Case	121
A.3	Contoh Berkas Konfigurasi Feast	121

Lampiran B	HASIL VERIFIKASI USE CASE	123
Lampiran C	FRAGMENTASI ARSITEKTUR PER PERAN	125
C.1	Fragmentasi pada Peran <i>Business User</i>	125
C.2	Fragmentasi pada Peran <i>Data Engineer</i>	126
C.3	Fragmentasi pada Peran <i>Data Scientist</i>	127
C.4	Fragmentasi pada Peran <i>Machine Learning Engineer</i>	127

DAFTAR GAMBAR

II.1	MLOps <i>level</i> 0: proses manual dengan penyerahan model satu arah.	13
II.2	MLOps <i>level</i> 1: otomasi <i>pipeline</i> pelatihan dengan komponen yang dapat dipakai ulang.	16
II.3	MLOps <i>level</i> 2: orkestrasi CI/CD/CT dengan deteksi <i>drift</i> sebagai pemicu pelatihan ulang.	17
III.1	Pandangan Umum Fragmentasi DataOps dan MLOps	47
IV.1	Pandangan Umum Platform DataOps dan MLOps Terintegrasi	62
IV.2	Pandangan Platform Terintegrasi untuk <i>Business User</i>	64
IV.3	Pandangan Platform Terintegrasi untuk <i>Data Engineer</i>	65
IV.4	Pandangan Platform Terintegrasi untuk <i>Data Scientist</i>	66
IV.5	Pandangan Platform Terintegrasi untuk <i>Machine Learning Engineer</i>	67
C.1	Fragmentasi pada Peran <i>Business User</i>	126
C.2	Fragmentasi pada Peran <i>Data Engineer</i>	126
C.3	Fragmentasi pada Peran <i>Data Scientist</i>	127
C.4	Fragmentasi pada Peran <i>Machine Learning Engineer</i>	127

DAFTAR TABEL

II.1	Perbandingan Framework Tingkat Kematangan MLOps	13
II.2	Evaluasi Alternatif <i>Schema Registry</i> untuk Apache Kafka	19
II.3	Evaluasi Alternatif <i>Framework</i> Pemrosesan Data	20
II.4	Evaluasi Alternatif Format Tabel <i>Lakehouse Open-Source</i>	21
II.5	Evaluasi Alternatif Penyimpanan <i>Offline</i> (OLAP)	24
II.6	Evaluasi Alternatif Penyimpanan <i>Online</i> (Latensi Rendah)	24
II.7	Evaluasi Alternatif <i>Vector Index Open-Source</i> untuk Pencarian Pendekatan Tetangga Terdekat	25
II.8	Evaluasi Alternatif <i>Feature Store Open-Source</i>	26
II.9	Evaluasi Alternatif <i>Experiment Tracking</i> dan <i>Model Registry</i>	26
II.10	Evaluasi Alternatif Orkestrasi ML <i>Open-Source</i>	27
II.11	Evaluasi Alternatif <i>Model Serving Open-Source</i>	28
II.12	Evaluasi Alternatif Manajemen Metadata dan <i>Governance</i>	29
II.13	Evaluasi Alternatif <i>Business Intelligence</i> dan <i>Dashboarding Open-Source</i>	31
II.14	Evaluasi Alternatif <i>Monitoring</i> dan <i>Observability</i>	31
II.15	Evaluasi Alternatif <i>GitOps Controller Open-Source</i>	32
II.16	Evaluasi Alternatif <i>Identity Provider Open-Source</i>	32
II.17	Evaluasi Alternatif Pengelola Rahasia <i>Open-Source</i>	33
II.18	Evaluasi Alternatif <i>Service Mesh Open-Source</i>	33
II.19	Evaluasi Alternatif <i>API Gateway Open-Source</i>	34
II.20	Evaluasi Alternatif <i>Policy Engine Open-Source</i>	34
II.21	Evaluasi Alternatif <i>Runtime Security Open-Source</i> pada Kubernetes	34
II.22	Perbandingan Detail Arsitektur Saat Ini dengan yang Diusulkan	36
III.1	Kebutuhan Fungsional Sistem	50
III.2	Kebutuhan Nonfungsional Sistem	52
III.3	Evaluasi Pemenuhan Kebutuhan Fungsional	58
III.4	Evaluasi Pemenuhan Kebutuhan Nonfungsional	58
VI.1	Skenario Uji Penerimaan per Tujuan Platform	98

VI.2 Target Metrik, Instrumen Ukur, dan Status Evaluasi per Kebutuhan . 100

DAFTAR PERSAMAAN

DAFTAR ALGORITMA

DAFTAR *LISTING*

VI.1 Ilustrasi <i>overlay</i> Kustomize <i>use case</i> yang menambahkan prefiks crypto- pada sumber daya basis platform	94
---	----

DAFTAR SIMBOL

Notasi	Deskripsi	Pemakaian pertama kali
p99	Persentil ke-99 dari distribusi latensi permintaan, dipakai sebagai <i>indicator tail latency</i> pada SLO layanan inferensi.	Bab IV

DAFTAR SINGKATAN

Singkatan	Deskripsi	Pemakaian pertama kali
ACID	<i>Atomicity, Consistency, Isolation, Durability</i> , sifat transaksi basis data.	Bab IV
ANN	<i>Approximate Nearest Neighbor</i> , kelas algoritma pencarian tetangga terdekat secara aproksimasi pada ruang vektor.	Bab IV
API	<i>Application Programming Interface</i> .	Bab I
APISIX	Apache APISIX, <i>API gateway</i> berbasis Nginx dan etcd.	Bab IV
BERT	<i>Bidirectional Encoder Representations from Transformers</i> , model bahasa berbasis <i>Transformer</i> .	Bab IV
CD	<i>Continuous Delivery</i> atau <i>Continuous Deployment</i> .	Bab I
CI	<i>Continuous Integration</i> .	Bab I
CNPG	<i>CloudNativePG</i> , operator Kubernetes untuk PostgreSQL.	Bab IV
CPU	<i>Central Processing Unit</i> .	Bab IV
CT	<i>Continuous Training</i> , pelatihan model secara berkelanjutan dengan pemicu <i>drift</i> atau jadwal.	Bab I
DORA	<i>DevOps Research and Assessment</i> , metrik kinerja <i>delivery</i> perangkat lunak.	Bab IV
DSRM	<i>Design Science Research Methodology</i> , kerangka metodologi penelitian rekayasa artefak.	Bab I
GPU	<i>Graphics Processing Unit</i> .	Bab IV
HNSW	<i>Hierarchical Navigable Small World</i> , struktur graf untuk ANN.	Bab IV
HPA	<i>Horizontal Pod Autoscaler</i> , kontrol penskalaan jumlah <i>pod</i> pada Kubernetes.	Bab IV
HTML	<i>HyperText Markup Language</i> .	Bab V
JSON	<i>JavaScript Object Notation</i> .	Bab IV
KEDA	<i>Kubernetes Event-driven Autoscaling</i> , <i>autoscaler</i> berbasis sinyal eksternal.	Bab IV
KS	<i>Kolmogorov-Smirnov test</i> , uji statistik kesetaraan dua distribusi.	Bab IV

MLOps	<i>Machine Learning Operations</i> , disiplin praktik operasional pengembangan dan penyajian model <i>machine learning</i> .	Bab I
ONNX	<i>Open Neural Network Exchange</i> , format pertukaran model lintas <i>framework</i> .	Bab IV
OIDC	<i>OpenID Connect</i> , protokol identitas berbasis OAuth 2.0.	Bab IV
PSI	<i>Population Stability Index</i> , metrik pergeseran distribusi fitur antar dua periode.	Bab IV
REST	<i>Representational State Transfer</i> .	Bab IV
SDK	<i>Software Development Kit</i> .	Bab IV
SLA	<i>Service Level Agreement</i> .	Bab IV
SLO	<i>Service Level Objective</i> .	Bab IV
TLS	<i>Transport Layer Security</i> .	Bab IV
VPA	<i>Vertical Pod Autoscaler</i> , kontrol penyetelan <i>request/limit</i> sumber daya <i>pod</i> .	Bab IV

BAB I

PENDAHULUAN

I.1 Latar Belakang

Praktik *machine learning* pada lingkungan produksi tidak lagi diukur dari ketepatan model semata, melainkan dari kemampuan organisasi memindahkan model dari laptop *data scientist* ke layanan yang berjalan stabil, terpantau, dan dapat diperbarui ketika data berubah. Sculley dkk. (2015) menunjukkan bahwa kode model hanya berperan sebagai bagian kecil dari sistem *machine learning* di lapangan. Sisanya berupa pengelolaan data, konfigurasi, pemantauan, layanan, dan perekaman *lineage*. Paleyes, Urma, dan Lawrence (2022) memperkuat temuan tersebut dengan memetakan tantangan utama yang muncul ketika model dipasang di produksi, dari persoalan kualitas data hingga pemeliharaan model setelah penyebaran.

Praktik DevOps yang diadaptasi oleh komunitas *machine learning* kemudian dikenal sebagai MLOps. Kreuzberger, Kühl, dan Hirschl (2023) mendefinisikan MLOps sebagai disiplin yang menyatukan rekayasa perangkat lunak, operasional, dan ilmu data untuk mengotomasi siklus hidup model dari eksperimen ke produksi. Pada saat yang sama, DataOps tumbuh sebagai pendekatan serupa untuk *pipeline* data dengan penekanan pada kolaborasi antar peran, kualitas data, dan iterasi cepat. Rella (2022) mencatat bahwa kedua praktik tersebut berbagi prinsip otomasi yang sama, tetapi di lapangan keduanya kerap dijalankan sebagai dua tumpukan teknologi yang terpisah. Jain dan Das (2025) menelaah kerangka integrasi rekayasa data dan MLOps berikut tantangannya, lalu menempatkan integrasi tersebut sebagai syarat *pipeline machine learning* yang skalabel dan tangguh. Dalam praktik, upaya mewujudkan integrasi tersebut dihadapkan pada tiga tekanan yang saling memperkuat, dan ketiganya menjadi pemicu pengembangan Tugas Akhir ini.

1. **Beban membangun platform dari nol di atas tumpukan komponen yang terfragmentasi**

Kajian sistematis arsitektur MLOps oleh Amou Najafabadi dkk. (2024) terhadap 43 studi primer mengidentifikasi 76 *tool* berbeda yang terpetakan pada

35 komponen arsitektur, gambaran betapa luasnya ruang pilihan yang harus dipertimbangkan sebelum satu platform utuh terbentuk. Organisasi yang merangkai sendiri *tool* sebanyak itu menanggung kurva belajar yang panjang, kontrak data lintas komponen yang berubah-ubah, dan kebutuhan menyelaraskan versi pustaka secara berkala. Burgueno-Romero, Cánovas Izquierdo, dan García-García (2025) mengamati bahwa komponen *open source* pada arsitektur MLOps cenderung berfungsi sebagai entitas yang berdiri sendiri alih-alih satu kerangka kerja yang kohesif, sehingga upaya integrasinya terasa berat dan mahal di awal. Beban tersebut bertambah ketika tim harus pula menyatukan jalur data dengan jalur model dalam satu lingkaran operasi, sementara arsitektur referensi yang menyatukan keduanya pada satu bidang kendali Kubernetes masih terbatas.

2. **Biaya berlangganan layanan terkelola dan *trade-off* antara waktu pengembangan dengan anggaran berjalan**

Layanan terkelola memang mempercepat adopsi, sebagaimana diperlihatkan Li dkk. (2023) pada kasus *feature store* terkelola yang membebaskan tim dari pembangunan infrastruktur penyimpanan fitur sehingga tim dapat berfokus pada rekayasa fitur inti. Akan tetapi, studi empiris Hamza, Akbar, dan Capilla (2024) terhadap lima belas praktisi dari delapan perusahaan menemukan bahwa model penagihan bayar per pakai pada komputasi *serverless* dan pelaporan biaya penyedia *cloud* yang kurang transparan menyulitkan estimasi anggaran, bahkan dapat menjadi tidak hemat biaya pada sebagian aplikasi berskala besar. Adusumilli (2025) mencatat hal senada, yaitu skema harga berbasis konsumsi menjadi kurang menarik secara ekonomi ketika utilisasi tinggi berlangsung secara konsisten. Risiko berikutnya adalah keterikatan terhadap satu penyedia layanan, sebab Mo dkk. (2023) menunjukkan bahwa migrasi aplikasi antar penyedia menjadi sulit ketika fitur kunci seperti persistensi data bergantung pada layanan pendukung yang spesifik penyedia. Organisasi dengan demikian dihadapkan pada dilema: berinvestasi pada tim platform dan menanggung waktu pengembangan yang panjang sebelum jalur *end-to-end* stabil, atau berlangganan layanan terkelola dengan biaya berjalan yang sulit diestimasi dan ruang kustomisasi yang sempit.

3. **Efek domino fragmentasi yang merambat ke seluruh rantai DataOps dan MLOps**

Ketika dua tekanan sebelumnya tidak terselesaikan pada satu bidang kendali, sambungan yang terputus antara lapis ingestasi, penyimpanan fitur, pelatihan, dan penyajian memaksa integrasi dirakit ulang pada tiap proyek, sementara

katalog metadata, identitas, dan format *lineage* dikelola sendiri-sendiri oleh tiap *tool*. Kondisi tersebut menyulitkan penelusuran asal data, membuka celah perbedaan nilai fitur antara fase pelatihan dan fase penyajian, dan menunda deteksi degradasi model karena tidak ada titik pantau bersama. Shankar dkk. (2022), melalui wawancara mendalam terhadap delapan belas insinyur *machine learning* lintas organisasi, mendokumentasikan bahwa pemeliharaan *glue code* dan *pipeline* antar komponen merupakan keluhan yang berulang dalam praktik MLOps. Rincian dampak per lapisan disusun pada Bab III sebagai dasar perumusan kebutuhan, sedangkan efek pada empat peran utama pengguna platform, yaitu *business user*, *data engineer*, *data scientist*, dan *machine learning engineer*, dipetakan pada Lampiran C.

Ketiga tekanan tersebut menggerakkan pengembangan sebuah platform yang menyatukan praktik DataOps dan MLOps di atas Kubernetes dengan komponen *open source*. Kubernetes dipilih karena posisinya sebagai standar *de facto* untuk orkestrasi beban kerja *cloud-native* dan karena adopsinya yang luas pada lingkungan *enterprise* (Lakka 2025). Pemanfaatan komponen *open source* ditujukan agar arsitektur dapat ditiru tanpa keterikatan pada satu penyedia layanan, sekaligus mempermudah audit teknis dan menghapus biaya lisensi berlangganan. Rancangan platform dijaga *domain-agnostic* agar dapat diadaptasi pada beragam domain tanpa perubahan pada lapis kendali, dan luaran berupa empat *sub-sistem* ditujukan untuk memutus rantai konsekuensi fragmentasi yang dipetakan di atas. Pengembangan platform ditata pada repositori Gitea sebagai sumber kebenaran tunggal dengan Argo CD sebagai mesin rekonsiliasi deklaratif, sehingga komponen pada tujuh lapis arsitektur, dari lapis ingestasi hingga lapis tata kelola, terhubung pada satu jalur GitOps. Komponen *open source* pengisi tiap lapis dirinci pada Bab IV.

I.2 Rumusan Masalah

Latar belakang di atas mengarah pada empat rumusan masalah yang ingin diselesaikan oleh platform yang dikembangkan. Keempat butir dirumuskan agar setiap permasalahan memiliki batas teknis yang jelas, pemilik dalam arsitektur platform, dan rute jawaban berupa *sub-sistem* atau kontrak antar komponen pada bab-bab berikutnya. Setiap rumusan masalah ditautkan pada satu tujuan yang dipaparkan pada Subbab I.3, satu *sub-sistem* pada Bab IV, satu paket implementasi pada Bab V, dan satu butir kesimpulan pada Bab VII. Keempat butir di bawah ini selanjutnya dirujuk dengan singkatan RM-1 hingga RM-4 pada bab-bab berikutnya.

1. Fragmentasi tumpukan teknologi DataOps dan MLOps

Tool pada lapis ingestasi, pemrosesan, penyimpanan, pelatihan, dan penyajian model dirakit terpisah sehingga integrasi mengulang dan kontrak antar lapis tidak seragam. Burgueno-Romero, Cánovas Izquierdo, dan García-García (2025) mencatat komponen *open source* cenderung beroperasi sebagai entitas yang terpisah-pisah, bukan sebagai satu kerangka MLOps yang kohesif, sehingga beban integrasi jatuh ke tim pemakainya.

2. Tata kelola data tidak konsisten lintas siklus data, fitur, dan model

Katalog metadata, *lineage*, dan kontrol kualitas kerap dijalankan pasca-fakta, terpisah dari *pipeline* produksi dan pencatatan eksperimen. Polyzotis dkk. (2018) menempatkan validasi data sebagai bagian *pipeline* produksi, bukan pekerjaan terpisah setelah masalah muncul.

3. Deteksi *drift* tidak terotomasi dan risiko kebocoran temporal antara mode *batch* dan *streaming*

Mekanisme deteksi *drift* aktif dan jaminan *point-in-time correctness* pada penyajian fitur belum menjadi bagian standar *pipeline* produksi. Kapoor dan Narayanan (2023) mendokumentasikan kebocoran data sebagai sumber krisis reproduksibilitas yang teridentifikasi pada 294 publikasi lintas 17 bidang ilmu, termasuk kebocoran temporal akibat pemisahan data yang mengabaikan urutan waktu.

4. Penyajian fitur multimoda dari satu sumber kebenaran

Fitur tabular, fitur agregat, dan fitur *embedding* vektor berdimensi tinggi harus konsisten antara nilai pelatihan dan nilai penyajian melalui satu kontrak API. Tanpa layanan fitur yang menyatukan jalur *offline* dan *online*, tiap aplikasi mengambil fitur dengan caranya sendiri sehingga nilai pelatihan dan nilai penyajian mudah menyimpang, kondisi yang dikenal sebagai *train-serving skew*.

Keempat rumusan masalah di atas dijawab secara berurutan oleh empat tujuan pada Subbab I.3, dengan rincian kebutuhan fungsional dan nonfungsional diturunkan pada Bab III. Penomoran RM-1 hingga RM-4 dipertahankan pada seluruh bab agar jawaban tiap permasalahan dapat ditelusuri dari rumusan hingga implementasinya. Pemetaan tersebut juga menjadi dasar penyusunan tabel uji penerimaan dan tabel evaluasi pada Bab VI. Keempat rumusan masalah pada akhirnya membentuk struktur kesimpulan pada Bab VII yang menyajikan jawaban beserta keterbatasannya.

I.3 Tujuan

Penelitian ini merumuskan empat tujuan yang masing-masing menjawab satu rumusan masalah, dengan luaran berupa artefak teknis yang dapat ditunjukkan dan dipakai ulang. Setiap tujuan menetapkan luaran berupa rancangan arsitektur, manifest *deployment* pada repositori Gitea, dan kontrak antar komponen yang dapat diuji pada *use case* verifikasi. Keempat tujuan di bawah ini selanjutnya dirujuk dengan singkatan T-1 hingga T-4 pada bab-bab berikutnya, dengan pemetaan satu lawan satu terhadap RM-1 hingga RM-4. Pemetaan tersebut menjadi acuan bagi Bab IV yang menjabarkan empat *sub-sistem* dan Bab V yang memaparkan implementasi dari masing-masing *sub-sistem*.

1. Arsitektur platform DataOps dan MLOps terintegrasi

Merancang dan mewujudkan arsitektur platform DataOps dan MLOps terintegrasi di atas Kubernetes yang menyatukan ingestasi, pemrosesan, penyimpanan fitur, pelatihan, penyajian model, observabilitas, dan tata kelola pada satu bidang kendali deklaratif dengan praktik GitOps. Tujuan ini menjawab RM-1.

2. *Sub-sistem* tata kelola data

Membangun *sub-sistem* tata kelola data yang menyediakan katalog metadata, perekaman *lineage*, dan kontrol kualitas data sebagai layanan bersama yang otomatis dijalankan di seluruh *pipeline* data, fitur, dan model. Tujuan ini menjawab RM-2.

3. *Sub-sistem* deteksi *drift* terotomasi

Mengimplementasikan *sub-sistem* deteksi *drift* terotomasi dengan kebijakan *retraining* berbasis ambang batas serta menjamin *point-in-time correctness* pada penyajian fitur lintas mode *batch* dan *streaming*. Tujuan ini menjawab RM-3.

4. Layanan fitur *dual-store offline* dan *online*

Mengembangkan layanan fitur *dual-store* yang melayani fitur tabular, fitur agregat, dan fitur *embedding* vektor berdimensi tinggi melalui satu kontrak API dengan jaminan konsistensi nilai antara fase pelatihan dan fase penyajian. Tujuan ini menjawab RM-4.

Kriteria keberhasilan keseluruhan platform diukur dari empat sudut yang masing-masing menautkan satu tujuan. Pertama, seluruh artefak dapat dipasang dari satu repositori Git melalui Argo CD tanpa intervensi manual setelah *bootstrap* awal kluster (T-1). Kedua, seluruh *pipeline* data, fitur, dan model terdaftar pada katalog metadata

dengan *lineage* yang dapat ditelusuri (T-2). Ketiga, mekanisme deteksi *drift* memicu *retraining* secara otomatis ketika ambang batas dilewati pada *use case* verifikasi (T-3). Keempat, fitur tabular, agregat, dan *embedding* tersaji melalui satu kontrak API dengan nilai yang konsisten antara penyimpanan *offline* dan *online* (T-4).

I.4 Batasan Masalah

Pengembangan platform pada Tugas Akhir ini dibatasi sebagai berikut. Batasan disusun agar lingkup kerja, lingkup pengujian, dan lingkup evaluasi terdefinisi secara eksplisit sehingga hasil dapat diukur dan penilaian dapat diarahkan pada bagian yang relevan. Setiap batasan mengacu pada karakter platform yang *domain-agnostic*, lingkungan pengujian satu *node*, dan patokan versi April 2026. Kelima batasan di bawah ini selanjutnya dirujuk dengan singkatan B-1 hingga B-5 pada bab-bab berikutnya.

- 1. Fokus pada lapis kendali platform**

Fokus berada pada lapis kendali platform, bukan pengembangan model spesifik. *Use case* domain hanya dipakai sebagai instans verifikasi rancangan sehingga pembahasan platform dijaga *domain-agnostic*.

- 2. Lingkungan Kubernetes *k3s* satu *node***

Platform berjalan di atas *k3s* pada satu *node* sebagai lingkungan pengembangan dan pengujian, dengan skala diatur oleh HPA, VPA, dan KEDA ketika beban nyata muncul. *Overlay* Kustomize untuk *environment* multi-*node* tetap dapat ditambahkan tanpa modifikasi basis kode.

- 3. Komponen *open source* dengan patokan versi April 2026**

Versi pustaka dan basis *image container* dipatok pada rilis stabil tertinggi per April 2026 agar pengujian dapat direproduksi pada lingkungan yang sama. Patokan tersebut diberlakukan pada `targetRevision` Helm *chart* di repositori Gitea agar Argo CD merekonsiliasi versi yang konsisten.

- 4. Cakupan keamanan terbatas pada akses antar layanan internal**

Aspek keamanan berfokus pada akses antar layanan internal melalui *mutual TLS* Istio, manajemen rahasia OpenBao, dan kebijakan jaringan Kubernetes. Otentikasi pengguna akhir pada lapisan aplikasi berada di luar cakupan pelaporan.

- 5. Evaluasi kuantitatif menyusul setelah implementasi inti**

Evaluasi *end-to-end* seperti pengukuran latensi penuh, *throughput*, dan stabilitas penyebaran lanjutan dilakukan setelah seluruh tahapan implementasi

inti berjalan. Evaluasi tidak mencakup *deployment* produksi pada lingkungan *multi-tenant* dengan trafik publik.

Kelima batasan di atas menjadi konteks pembacaan seluruh laporan, terutama pada Bab III dan Bab VI. Setiap usulan penambahan lingkup selama pengerjaan dievaluasi terhadap kelima batas tersebut agar pengembangan tetap terfokus. Asumsi terkait *open source* dan patokan versi memberi peluang pengembang lain untuk memverifikasi arsitektur tanpa kontak dengan penyedia layanan berbayar. Batasan ini tidak menutup kemungkinan pengembangan lanjutan pada lingkungan *multi-node* dan *multi-domain* di masa depan.

I.5 Metodologi

Penelitian ini mengikuti kerangka *Design Science Research Methodology* (DSRM) yang dirumuskan oleh Peffers dkk. (2007) untuk penelitian sistem informasi yang menghasilkan artefak. Luaran utama Tugas Akhir ini berupa platform yang berjalan, yaitu artefak jenis instansiasi (*instantiation*) dalam taksonomi *design science*, sehingga kerangka penelitian berorientasi artefak menjadi pijakan yang sesuai (Hevner dkk. 2004). DSRM dipilih dibandingkan metodologi rekayasa perangkat lunak murni karena DSRM menempatkan iterasi antara perancangan, demonstrasi, dan evaluasi sebagai bagian inti proses penelitian, bukan sekadar tahapan pengembangan. Tahapan yang dijalankan meliputi enam fase berikut dengan luaran teknis yang dapat diukur.

1. Identifikasi masalah dan motivasi

Studi literatur DataOps, MLOps, dan tata kelola data dilakukan untuk menyusun peta tantangan. Tiga pemicu dan empat rumusan masalah (RM-1 sampai RM-4) menjadi luaran fase ini.

2. Penetapan tujuan artefak

Tujuan dirumuskan sebagai daftar artefak teknis yang dapat dipasang, dioperasikan, dan diaudit, dipetakan satu lawan satu terhadap rumusan masalah. Kebutuhan fungsional KF-01 sampai KF-14 dan kebutuhan nonfungsional KNF-01 sampai KNF-12 disusun pada Bab III.

3. Perancangan dan pengembangan artefak

Arsitektur tujuh lapis dirancang sebagai bidang kendali deklaratif berbasis Kubernetes, bertolak dari alternatif arsitektur terpilih hasil matriks evaluasi multi-kriteria pada Bab III. Pengembangan manifes dilakukan modular pada repositori Gitea dengan *folder* per komponen dan *overlay* Kustomize.

4. **Demonstrasi**

Platform dipasang pada lingkungan *k3s* dan diberi muatan dari satu *use case* domain nyata sebagai instans verifikasi, yang dirinci pada Bab VI. Demonstrasi diatur sebagai empat tahap iteratif, dengan setiap tahap memvalidasi satu *sub-sistem* sesuai T-1 hingga T-4.

5. **Evaluasi**

Evaluasi dilaksanakan pada akhir siklus implementasi dengan tiga dimensi: fungsional, nonfungsional (skalabilitas, ketersediaan, observabilitas, keamanan, biaya), serta tata kelola (*lineage* tertelusur, kontrak data ditegakkan, kebijakan akses dapat diaudit). Angka kuantitatif evaluasi dilaporkan pada Bab VI.

6. **Komunikasi**

Hasil rancangan dan implementasi didokumentasikan pada laporan ini, beserta repositori Gitea yang menjadi acuan reproduksi. Luaran fase ini berupa laporan, repositori sumber, dan presentasi ujian akhir.

Keenam fase di atas dijalankan secara iteratif, dengan kemungkinan kembali ke fase sebelumnya ketika temuan baru muncul pada fase berikutnya. Iterasi tersebut sejalan dengan karakter DSRM sebagai metodologi berorientasi artefak yang menempatkan umpan balik dari demonstrasi dan evaluasi sebagai pemicu perbaikan rancangan. Pola pengembangan serupa, yaitu membangun platform atau *pipeline* MLOps berbasis Kubernetes lalu menilainya melalui demonstrasi dan eksperimen, dipakai oleh Hsu dkk. (2024) dan Rodrigues dkk. (2025). Pelaporan tiap fase disusun pada bab yang sesuai sehingga keterhubungan antara fase metodologi dan struktur laporan tetap eksplisit.

I.6 Sistematika Penulisan

Laporan Tugas Akhir disusun atas tujuh bab dengan urutan yang mengikuti DSRM dan praktik pelaporan rekayasa perangkat lunak. Tiap bab menjawab satu atau lebih fase pada metodologi sehingga keterhubungan antara metodologi dan struktur laporan tetap eksplisit. Pemetaan antara rumusan masalah, tujuan, *sub-sistem*, dan butir kesimpulan dijaga konsisten lewat penomoran RM-N, T-N, dan butir pada Bab IV hingga Bab VII. Bab IV dan Bab V membentuk pasangan perancangan dan implementasi yang dibaca berurutan, sedangkan Bab VI dan Bab VII menutup siklus dengan evaluasi serta kesimpulan dan saran. Urutan bab beserta isi utamanya dirinci pada poin-poin berikut.

1. **Bab I Pendahuluan**

Memuat latar belakang, rumusan masalah, tujuan, batasan masalah, metodo-

logi penelitian, serta sistematika penulisan.

2. **Bab II Studi Literatur**

Meninjau konsep DataOps, MLOps, Kubernetes, *feature store*, siklus hidup model, penyajian model, tata kelola data, deteksi *drift*, observabilitas, dan praktik GitOps sebagai landasan teoretis.

3. **Bab III Analisis Masalah**

Memetakan kondisi sistem dan menyusun kebutuhan fungsional KF-01 hingga KF-14 serta kebutuhan nonfungsional KNF-01 hingga KNF-12 sebagai turunan empat tujuan. Tiga alternatif arsitektur dibandingkan melalui matriks evaluasi multi-kriteria.

4. **Bab IV Perancangan Arsitektur Platform DataOps dan MLOps**

Menjabarkan arsitektur tujuh lapis dan perancangan empat *sub-sistem* yang menjawab T-1 hingga T-4. Diagram arsitektur disusun mengikuti notasi C4.

5. **Bab V Implementasi Arsitektur Platform DataOps dan MLOps**

Memaparkan implementasi tiap *sub-sistem* yang sejalan dengan Bab IV, dengan praktik GitOps melalui Argo CD pada repositori Gitea sebagai sumber kebenaran tunggal.

6. **Bab VI Evaluasi**

Berisi metode evaluasi, hasil pengukuran, dan pembahasan terhadap kebutuhan fungsional dan nonfungsional pada Bab III.

7. **Bab VII Penutup**

Menyimpulkan jawaban atas T-1 hingga T-4 dan memberikan saran pengembangan lanjutan untuk lingkungan multi-*node* dan multi-domain.

Setelah Bab VII, dokumen ditutup oleh Daftar Pustaka yang memuat seluruh referensi ter kutip dan Lampiran yang memuat berkas pendukung. Daftar Pustaka memakai gaya *Chicago author-date* (`biblatex-chicago`) dengan label Indonesia agar konsisten dengan tubuh laporan. Lampiran memuat daftar artefak platform, hasil verifikasi *use case*, dan pemetaan fragmentasi arsitektur per peran pengguna. Penomoran lampiran mengikuti abjad (Lampiran A, B, C, dan seterusnya) sebagaimana konvensi laporan Tugas Akhir di STI ITB.

BAB II

STUDI LITERATUR

II.1 Cakupan Studi Literatur

Bab ini merangkum landasan teori dan ekosistem perangkat lunak yang relevan untuk perancangan platform pada Bab IV. Tinjauan disusun mulai dari konsep DataOps dan MLOps, dilanjutkan dengan komponen teknis yang menjadi bahan baku perancangan, kemudian ditutup dengan posisi penelitian terdahulu dan kebaruan yang ditawarkan. Susunan ini mengikuti tujuh lapis arsitektur (Infrastruktur, Ingestasi, Pemrosesan, Penyimpanan Fitur, Siklus Hidup Model, Penyajian, serta Tata Kelola dan Observabilitas) sehingga setiap topik literatur memiliki pemilik lapis yang jelas. Sumber rujukan menggabungkan publikasi akademik dengan dokumentasi resmi komponen *open source* agar setiap pilihan teknis dapat ditelusuri ke spesifikasi sumber.

Peta tematik bab ini juga menjadi rujukan saat menyusun matriks evaluasi alternatif pada Bab III. Setiap subbab teknis ditutup dengan tabel perbandingan ketika lapis tersebut memiliki lebih dari satu alternatif yang layak. Pemilihan komponen pada platform mengikuti hasil perbandingan tersebut, dengan rincian skor dan kriteria pada Bab III. Pendekatan ini memastikan bahwa keputusan teknis pada Bab IV dan Bab V dapat ditelusuri ke landasan teori dan ke pertimbangan komparatif.

II.2 DataOps dan MLOps

II.2.1 DataOps

DataOps merupakan adaptasi praktik DevOps untuk siklus hidup data. Rella (2022) mendefinisikan DataOps sebagai pendekatan kolaboratif yang menempatkan kualitas data, kecepatan iterasi, dan pengukuran tertaut sebagai inti. Kleppmann (2017) memberi fondasi konseptual lewat penjelasan tentang sistem berbasis data yang andal, dapat diskalakan, dan dapat dipelihara, sementara Stopford (2018) memperkenalkan pola arsitektur *event-driven* dengan Apache Kafka sebagai tulang punggung.

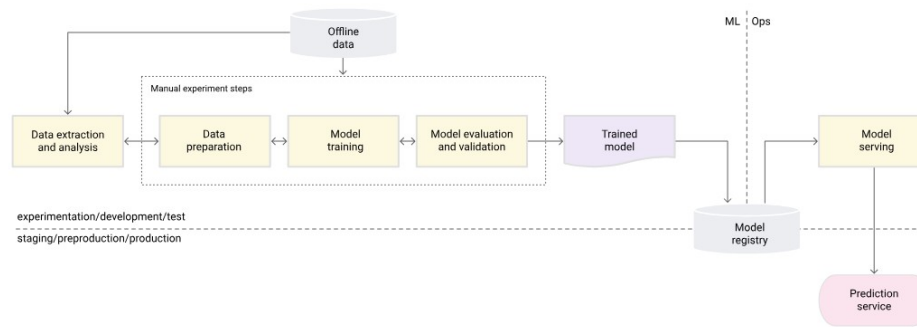
Pada praktiknya, DataOps menuntut otomatisasi *pipeline* ingestasi dan transformasi, pengujian kontrak data, perekaman *lineage*, dan pemantauan kualitas data sebagai bagian permanen dari operasi.

Empat pilar operasi DataOps yang dipakai pada platform ini adalah otomatisasi *pipeline*, kontrak skema, pengujian kualitas data, dan pemantauan SLO. Otomatisasi *pipeline* disusun dengan Argo Workflows dan Tekton, kontrak skema dijaga oleh Karapace sebagai *schema registry*, pengujian kualitas dijalankan oleh Great Expectations sebelum data ditulis ke lapis hilir, dan SLO didefinisikan melalui Sloth dalam bentuk aturan Prometheus. Polyzotis dkk. (2018) menempatkan validasi data sebagai bagian dari *pipeline* produksi, bukan sebagai pekerjaan terpisah setelah masalah muncul. Kombinasi keempat pilar tersebut menjadi prasyarat bagi integrasi DataOps dan MLOps yang dibahas pada Subbab II.2.4.

II.2.2 MLOps

MLOps memperluas DevOps ke seluruh siklus hidup model *machine learning*. Kreuzberger, Köhl, dan Hirschl (2023) memberikan definisi konsolidatif: MLOps sebagai gabungan budaya kerja dan perangkat lunak untuk mengotomasi eksperimen, pelatihan, evaluasi, penyebaran, dan pemantauan model. Sculley dkk. (2015) menjelaskan akar persoalannya: *hidden technical debt* pada sistem *machine learning* muncul terutama dari komponen non-model seperti perekayasa fitur, konfigurasi, perekaman, pengujian data, dan layanan pemantauan. Paleyes, Urma, dan Lawrence (2022) melengkapi pemetaan dengan survei tantangan operasionalisasi, dan Shankar dkk. (2022) membawa perspektif praktisi melalui wawancara dengan tim MLOps lintas industri.

Tiga karakteristik penting yang membedakan MLOps dari DevOps tradisional adalah *continuous training*, *continuous monitoring*, dan *model registry* sebagai sumber kebenaran versi model. Amershi dkk. (2019) menggambarkan sembilan tahap rekayasa perangkat lunak *machine learning* mulai dari pengumpulan data hingga pemantauan, dengan setiap tahap memerlukan dukungan otomatisasi tersendiri. Breck dkk. (2017) memperkenalkan *rubric* kesiapan produksi yang menjadi acuan banyak tim untuk menilai kematangan *pipeline* mereka. Platform ini mengoperasionalkan ketiga karakteristik tersebut dengan Kubeflow Pipelines untuk *continuous training*, Evidently dan Prometheus untuk *continuous monitoring*, serta MLflow Model Registry sebagai sumber kebenaran versi model yang ditelaah ulang pada Subbab II.6.1.



Gambar II.1 MLOps *level 0*: proses manual dengan penyerahan model satu arah.

II.2.3 Tingkat Kematangan MLOps

Beberapa kerangka kematangan diusulkan untuk menggambarkan jenjang kapabilitas tim. Google Cloud (2024) membagi MLOps menjadi tiga level: *level 0* dengan proses manual, *level 1* dengan otomatisasi *pipeline* pelatihan, dan *level 2* dengan otomatisasi penyebaran berbasis CI/CD/CT. Tiga level ini diilustrasikan pada Gambar II.1, Gambar II.2, dan Gambar II.3. Microsoft Azure (2023) memberi varian dengan lima level dari 0 hingga 4, dan McKnight (January 2020) menambah perspektif organisasi. Perbandingan ringkas dapat dilihat pada Tabel II.1.

Platform yang dikembangkan menargetkan kematangan setara *level 2* pada kerangka Google Cloud, dengan rincian sebagai berikut. Pertama, *pipeline* pelatihan dijalankan secara otomatis melalui Kubeflow Pipelines yang dikomposisi pada Pipeline CRD dan dipicu oleh peristiwa data baru atau ambang *drift*. Kedua, penyebaran model dijalankan otomatis melalui KServe InferenceService dengan strategi penyebaran progresif berbasis Argo Rollouts. Ketiga, pemantauan model dilakukan secara berkelanjutan oleh *job* pemantauan Argo Workflows yang melaporkan metrik *drift* ke Prometheus serta menulis laporan Evidently ke MinIO untuk audit historis. Empat pilar tersebut menjadi acuan implementasi pada Bab V dan dasar evaluasi pada Bab VI.

Tabel II.1 Perbandingan Framework Tingkat Kematangan MLOps

FW	L	DevOps Practices	ML Pipeline Automation	CI/CD Integration	CT (Continuous Training)	Monitoring & Feedback Loop	Governance & Lineage
GCP	L0	Tidak ada (silo tim)	Manual (script, notebook)	Tidak ada	Tidak ada	Tidak ada (<i>model untracked</i>)	Tidak ada (<i>no lineage</i>)

FW	L	DevOps Practices	ML Pipeline Automation	CI/CD Integration	CT (Continuous Training)	Monitoring & Feedback Loop	Governance & Lineage
	L1	Parsial (<i>operational symmetry</i>)	Ya (<i>end-to-end</i>)	Parsial (<i>pipeline</i> saja)	Ya (<i>auto-mated</i>)	Ya (<i>drift detection</i>)	Ya (<i>metadata store, feature store</i>)
	L2	Ya (<i>fully integrated</i>)	Ya (<i>fully automated</i>)	Ya (CI-/CD/CT penuh)	Ya (<i>fully automated</i>)	Ya (<i>integrated</i>)	Ya (<i>automated melalui sistem CI/CD</i>)
Azure	L0	Tidak ada (<i>disparate teams</i>)	Manual (<i>ad hoc</i>)	Tidak ada	Tidak ada	Tidak ada (<i>black box</i>)	Tidak ada (<i>no tracking</i>)
	L1	Ya (<i>aplikasi</i> saja)	Manual (<i>model handoff</i>)	Aplikasi saja (<i>bukan untuk ML</i>)	Tidak ada	Minimal (<i>limited feedback</i>)	Minimal (<i>sulit ditelusuri</i>)
	L2	Ya (<i>DevOps aplikasi</i>)	Ya (<i>pelatihan</i> saja)	Tidak ada (<i>release manual</i>)	Ya (<i>pelatihan otomatis</i>)	Parsial (<i>pelacakan terpusat</i>)	Ya (<i>version control, manajemen model</i>)
	L3	Ya (<i>terintegrasi</i>)	Ya (<i>latih dan deploy</i>)	Ya (CD model)	Ya (<i>pelatihan otomatis</i>)	Parsial (<i>A/B testing</i>)	Ya (<i>pengujian terintegrasi</i>)
	L4	Ya (<i>fully integrated</i>)	Ya (<i>fully automated</i>)	Ya (CI-/CD/CT penuh)	Ya (<i>auto retrain</i>)	Ya (<i>verbose metrics, closed loop</i>)	Ya (<i>komprehensif</i>)
AWS	Initial	Tidak ada	Manual (<i>notebooks, lokal</i>)	Tidak ada	Tidak ada	Tidak ada	Tidak ada (<i>no standardization</i>)
	Repeatable	Parsial (<i>infrastruktur</i>)	Ya (<i>training pipeline</i>)	Tidak ada (<i>deploy manual</i>)	Ya (<i>auto-mated</i>)	Minimal (<i>infrastruktur eksperimen</i>)	Ya (<i>model registry, version control</i>)
	Reliable	Ya (<i>terintegrasi</i>)	Ya (<i>auto-mated</i>)	Ya (CI/CD terintegrasi)	Ya (<i>auto-mated</i>)	Ya (<i>prod monitoring, rollback</i>)	Ya (<i>IaC, pengujian otomatis</i>)

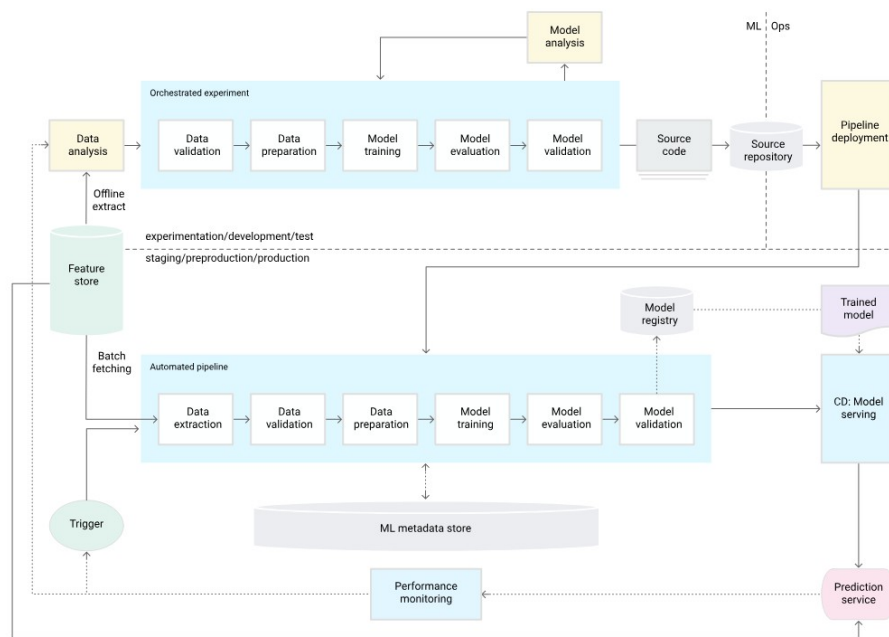
FW	L	DevOps Practices	ML Pipeline Automation	CI/CD Integration	CT (Continuous Training)	Monitoring & Feedback Loop	Governance & Lineage
	Scalable	Ya (multi-account/re)	Ya (fully automated, multi-region)	Ya (otomasi penuh)	Ya (auto retrain melalui drift detection)	Ya (drift detection, optimasi otomatis)	Ya (centralized feature store, full governance)
Giga Om	L0	Tidak ada (<i>no standards</i>)	Manual (inkonsisten)	Tidak ada	Tidak ada	Minimal	Minimal (keamanan minimal)
	L1	Parsial (<i>basic practices</i>)	Parsial (sebagian otomatis)	Tidak ada (sebagian besar manual)	Tidak ada	Minimal (<i>basic tracking</i>)	Parsial (<i>limited governance</i>)
	L2	Ya (diimplementasikan)	Ya (<i>pipeline CI/CD</i>)	Ya (diimplementasikan)	Parsial (sebagian otomatis)	Parsial (<i>basic monitoring</i>)	Ya (<i>formal governance dimulai</i>)
	L3	Ya (<i>fully integrated</i>)	Ya (<i>fully automated</i>)	Ya (komprehensif)	Ya (<i>automated</i>)	Ya (<i>comprehensive monitoring/alerting</i>)	Ya (<i>automated governance, self-service</i>)
	L4	Ya (<i>fully adaptive</i>)	Ya (<i>AI-optimized</i>)	Ya (<i>self-optimizing</i>)	Ya (<i>continuous learning</i>)	Ya (<i>continuous learning dari produksi</i>)	Ya (platform terintegrasi dan adaptif penuh)

Keterangan:

1. Merah : Tidak ada implementasi atau sepenuhnya manual
2. Kuning : Implementasi parsial atau terbatas
3. Hijau : Implementasi penuh atau otomatis

II.2.4 Integrasi DataOps dan MLOps

Jain dan Das (2025) menyatakan bahwa integrasi DataOps dan MLOps menjadi prasyarat skalabilitas dan ketahanan *pipeline machine learning* modern. Burgueno-Romero, Cánovas Izquierdo, dan García-García (2025) mengusulkan arsitektur *open source* yang menyatukan kedua praktik tersebut, namun belum membahas penyajian fitur *dual-store* dan kebijakan deteksi *drift* sebagai satu kesatuan. Rodrigues dkk.



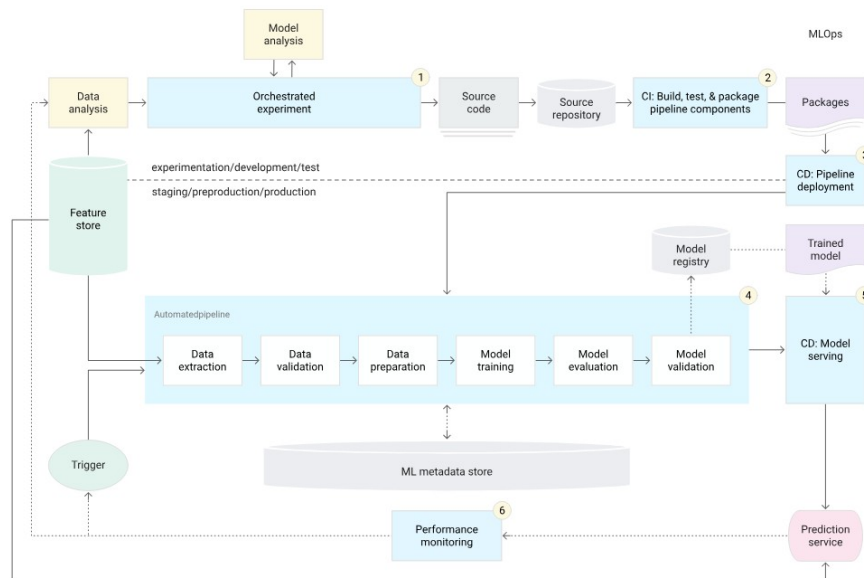
Gambar II.2 MLOps *level 1*: otomasi *pipeline* pelatihan dengan komponen yang dapat dipakai ulang.

(2025) membahas arsitektur untuk *stream learning* mendekati waktu nyata, dengan penekanan pada deteksi *drift* dan pembaruan model. Penelitian ini berdiri pada irisan ketiganya: arsitektur *open source* yang menyatukan DataOps dan MLOps, dengan layanan fitur *dual-store* dan deteksi *drift* terotomasi sebagai kelas pertama.

Integrasi tersebut diwujudkan melalui tiga jembatan teknis yang dijelaskan pada bab berikutnya. Jembatan pertama adalah *feature contract* pada Feast yang menyatukan *offline store* dan *online store* dengan satu definisi `FeatureView`, sehingga jalur DataOps yang menulis fitur dan jalur MLOps yang membaca fitur memakai kontrak yang sama. Jembatan kedua adalah *lineage* terbuka melalui OpenLineage yang menyatukan jejak transformasi DataOps (Flink, Spark, dbt) dengan jejak pelatihan MLOps (Kubeflow Pipelines) ke satu jalur penerimaan DataHub. Jembatan ketiga adalah bidang kendali deklaratif Argo CD yang merekonsiliasi manifes komponen DataOps dan MLOps dari satu repositori Gitea, sehingga kebijakan dan versi komponen tetap konsisten lintas dua disiplin.

II.3 Kubernetes dan Orkestrasi *Cloud-Native*

Kubernetes berperan sebagai bidang orkestrasi yang menjalankan beban kerja ter-containerisasi pada *cluster* mesin. Lakka (2025) mendokumentasikan adopsi Kubernetes pada skala perusahaan dan menyoroti perannya sebagai standar *de facto*



Gambar II.3 MLOps level 2: orkestrasi CI/CD/CT dengan deteksi *drift* sebagai pemicu pelatihan ulang.

cloud-native. Adusumilli (2025) memperluas perspektif tersebut dengan pola *serverless* di atas Kubernetes, yang relevan untuk beban kerja *inference* dengan pola lalu lintas berfluktuasi. Cloud Native Computing Foundation (2024a) memberikan referensi resmi mengenai objek-objek inti seperti Pod, Deployment, StatefulSet, Service, dan Ingress yang menjadi landasan setiap komponen platform.

Pemilihan Kubernetes sebagai bidang dasar memberi tiga keuntungan teknis: orkestrasi beban kerja melalui objek deklaratif, mekanisme penskalaan otomatis, dan ekosistem operator yang memperluas API *cluster* dengan *Custom Resource Definition* (CRD). Distribusi *lightweight* seperti *k3s* mengakomodasi pengembangan dan pengujian pada satu *node* tanpa mengorbankan kompatibilitas API. Pilihan distribusi tersebut sejalan dengan asumsi B-2 pada Bab I bahwa lingkungan pengujian memakai satu *node* dengan *footprint* sumber daya yang ringkas. Mekanisme *cluster autoscaler* tidak diaktifkan pada lingkungan satu *node*, sedangkan HPA, VPA, dan KEDA menjadi mekanisme penskalaan beban kerja yang dipakai.

Penskalaan otomatis dijalankan oleh tiga komponen yang saling melengkapi. *Horizontal Pod Autoscaler* (HPA) menambah atau mengurangi jumlah *replica* pada Deployment dan StatefulSet berdasarkan metrik CPU dan memori yang dipasok oleh Metrics Server. *Vertical Pod Autoscaler* (VPA) (Kubernetes Autoscaling SIG 2025) merekomendasikan dan menyetel *request* sumber daya pada level *pod* berdasarkan pengukuran historis. KEDA (KEDA Authors 2025) memperluas pens-

kalaan ke sumber peristiwa eksternal melalui `ScaledObject`, termasuk *lag* antrean Kafka dan kueri metrik Prometheus, sehingga komponen pemroses peristiwa dapat berskala turun ke nol pada saat *idle*.

Komponen operator menyediakan abstraksi tingkat tinggi pada perangkat data dan model. Strimzi (Strimzi Authors 2025) mengoperasikan kluster Apache Kafka, *Schema Registry*, dan *Kafka Connect* melalui CRD seperti `Kafka`, `KafkaUser`, dan `KafkaTopic`. CloudNativePG (CloudNativePG Contributors 2025) mengelola siklus hidup kluster PostgreSQL, termasuk *backup* dan *failover*. *Operator* Apache Flink (Apache Flink Authors 2025) dan *operator* Apache Spark (Apache Spark Authors 2025) menyiapkan `FlinkDeployment` dan `SparkApplication` sebagai unit kerja deklaratif, sedangkan *operator* Altinity untuk ClickHouse (Altinity 2025) menyediakan CRD `ClickHouseInstallation` untuk replikasi dan *sharding*.

Lapisan *runtime* dilengkapi oleh Knative Serving (Knative Authors 2025) yang menambah kemampuan penskalaan ke nol pada beban kerja *request-response* dan menjadi fondasi bagi KServe. Kueue (Kueue Authors 2024) menyediakan antrean kerja untuk beban kerja *batch* sehingga prioritas dan kuota dapat ditegakkan lintas *namespace*. cert-manager (cert-manager Contributors 2025) mengotomasi penerbitan dan rotasi sertifikat TLS bagi *Ingress*, *webhook*, dan komunikasi internal antar layanan. Ketiga komponen tersebut bekerja saling melengkapi, sebab Knative menyediakan penskalaan ke nol, Kueue menyediakan kontrol kuota lintas *tenant*, dan cert-manager menjaga rantai sertifikat agar mTLS Istio terus berlaku.

II.4 Manajemen Data: Ingestasi, *Streaming*, dan *Batch*

II.4.1 Pola Lambda dan Kappa

Marz dan Warren (2015) memperkenalkan arsitektur Lambda yang memisahkan *batch layer* dan *speed layer*. Kreps (2014) mengkritisi pola tersebut karena ganda operasi dan mengusulkan arsitektur Kappa yang menyatukan keduanya menjadi satu jalur *stream-first*. Amazon Web Services (2024) memberikan rangkuman pola *event-driven* yang banyak diadopsi di lingkungan *cloud-native*. Fowler dan Lewis (2014) memberikan latar arsitektural mengenai dekomposisi layanan yang menjadi pra-syarat bagi adopsi pola *event-driven* berskala besar.

Platform ini mengadopsi pola Kappa dengan modifikasi pada lapis pemrosesan: aliran utama berjalan pada Flink untuk *streaming* sedangkan *backfill* historis dijalankan oleh Spark dan dbt pada referensi waktu yang sama. Pola tersebut menjaga konsis-

tensi temporal antara fitur *streaming* dan fitur *batch* sebagaimana dijabarkan pada Subbab II.5.4. Pendekatan ini juga sejalan dengan rekomendasi Stopford (2018) mengenai *event log* sebagai sumber kebenaran tunggal. Hasilnya, kebutuhan *retraining* model yang melibatkan data historis dapat dijawab tanpa menyalin data ke jalur terpisah, sebab *offline store* ClickHouse dipopulasi dari *topic* Kafka yang sama.

II.4.2 Apache Kafka

Apache Kafka berperan sebagai *distributed log* yang menyimpan kejadian dalam urutan tertentu dan memungkinkan pelanggan untuk memutar ulang aliran data dari posisi tertentu (Apache Software Foundation 2024c). Kafka mendukung jaminan *at-least-once* secara baku dan *exactly-once* dengan transaksi, sehingga cocok untuk dipakai sebagai tulang punggung penghubung antara produsen data dan konsumen pemroses. Registrasi skema dipisahkan ke Karapace (Aiven Karapace Authors 2025) sebagai *schema registry* kompatibel-Confluent yang berlisensi Apache 2.0, dengan dukungan Avro, JSON Schema, dan Protobuf serta verifikasi kompatibilitas saat publikasi. Tabel II.2 merangkum perbandingan beberapa pilihan *schema registry open-source*.

Tabel II.2 Evaluasi Alternatif *Schema Registry* untuk Apache Kafka

<i>Schema Registry</i>	Lisensi <i>Open</i>	Kompatibilitas Confluent	Dukungan Format	Maturitas	Total
Karapace (Aiven)	5	5	5	4	19
Apicurio Registry	5	4	5	4	18
Confluent <i>Schema Registry</i>	2	5	5	5	17
Strimzi <i>Schema Registry</i>	5	3	3	3	14

Pada platform ini, Kafka dioperasikan melalui *operator* Strimzi yang menyediakan CRD Kafka, KafkaUser, KafkaTopic, dan KafkaConnect, sehingga seluruh konfigurasi kluster, otentikasi pengguna, daftar *topic*, dan *connector* berada dalam bidang kendali yang sama dengan komponen lain. Mode otentikasi yang dipakai adalah SCRAM-SHA-512 melalui Strimzi KafkaUser dengan kredensial yang disebar oleh External Secrets Operator dari OpenBao. Topologi *topic* bawaan menggunakan tiga *partition* dengan faktor replikasi satu pada lingkungan satu *node*, dan dapat dinaikkan menjadi tiga pada lingkungan multi-*node*. Pemantauan *lag* antrean dilakukakan oleh KEDA ScaledObject yang membaca metrik Kafka Exporter melalui Prometheus.

II.4.3 Apache Flink

Apache Flink memproses aliran data dengan dukungan *event-time* dan *stateful operations*. Carbone dkk. (2015) menjelaskan dasar *pipeline streaming* pada Flink, dan dokumentasi resmi (Apache Software Foundation 2024b) menjabarkan fitur *exactly-once processing* dengan *checkpointing*. Flink dipakai untuk transformasi fitur waktu nyata dan agregasi *rolling window* yang dibutuhkan oleh model *streaming*. Penempatan komputasi pada *event-time* alih-alih *processing-time* membuat hasil transformasi tetap konsisten ketika data terlambat tiba.

Pada platform ini, Flink dijalankan melalui FlinkDeployment CRD yang disediakan oleh operator Apache Flink. *Checkpoint* dan *savepoint* ditulis ke MinIO sebagai penyimpanan objek S3-kompatibel, sehingga pemulihan *job* dapat dilakukan tanpa kehilangan keadaan. Aliran utama *job* membaca *topic* dari Kafka melalui konektor Flink Kafka, melakukan transformasi pada *event-time* dengan *watermark* per partisi, lalu menerbitkan hasil ke *topic sink* dan ke ClickHouse. Latensi pemrosesan dipantau melalui metrik Flink yang diekspos pada Prometheus, dengan SLO p95 sub-detik yang dipantau oleh Sloth.

II.4.4 Apache Spark dan dbt

Apache Spark menyediakan kemampuan pemrosesan *batch* dan *micro-batch* pada skala besar (Apache Software Foundation 2024d). Zaharia dkk. (2016) memberikan ringkasan arsitektur Spark sebagai mesin terpadu. Spark dipakai untuk transformasi fitur dengan referensi waktu historis dan untuk pelatihan model yang membutuhkan pemrosesan data besar. Transformasi SQL deklaratif pada lapisan *warehouse* dikerjakan oleh dbt (dbt Labs 2025), yang memetakan model ke berkas SQL dengan dukungan pengujian skema dan dokumentasi otomatis.

Tabel II.3 Evaluasi Alternatif *Framework* Pemrosesan Data

<i>Framework</i> Pemrosesan	Maturitas	Kapabilitas <i>Batch</i>	Kapabilitas <i>Streaming</i>	Komunitas	Total
Apache Spark	5	5	4	5	19
Apache Flink	4	4	5	4	17
Apache Kafka + Streams	5	2	5	4	16
Apache Beam	3	4	4	3	14

Pada platform ini, Spark dijalankan melalui SparkApplication CRD dari operator Apache Spark, sehingga *job* dapat dijadwalkan oleh Argo Workflows tanpa konfigu-

rasi tambahan. *Job* Spark biasanya berjalan untuk dua tujuan: *backfill* fitur historis ke FeatureView Feast dan pelatihan model dengan *data frame* berskala besar. dbt menjalankan transformasi SQL pada lapisan *warehouse* ClickHouse dengan model *ref()* yang menjaga *lineage* antar tabel. Hasil dbt yang berhasil dijalankan akan mendaftarkan *lineage* ke DataHub melalui adaptor OpenLineage sehingga jejak transformasi terdokumentasi.

II.4.5 Format Tabel *Lakehouse*

Penyimpanan jangka panjang pada *data lake* memerlukan format tabel yang menyediakan transaksi ACID, evolusi skema yang aman, dan ketertelusuran lintas *engine* pemrosesan. Apache Iceberg, Delta Lake, dan Apache Hudi merupakan tiga format tabel *open-source* yang lazim dipakai untuk arsitektur *lakehouse*, sementara penyimpanan langsung dalam berkas Parquet tanpa lapisan format tabel tidak menjamin transaksi maupun evolusi skema yang aman. Pada platform ini, Apache Iceberg dipilih sebagai format tabel *lakehouse* karena netralitas *engine*, dukungan katalog REST melalui Lakekeeper (Lakekeeper Authors 2025), dan kemampuan kueri federasi melalui Trino (Trino Software Foundation 2024). Tabel II.4 merangkum perbandingan beberapa format tabel *lakehouse* terhadap kriteria yang relevan dengan platform.

Tabel II.4 Evaluasi Alternatif Format Tabel *Lakehouse Open-Source*

Format Tabel	Maturitas	Transaksi ACID	Evolusi Skema	Integrasi Engine	Total
Apache Iceberg	5	5	5	5	20
Delta Lake	5	5	4	4	18
Apache Hudi	4	5	4	3	16
Parquet (tanpa format tabel)	5	1	2	3	11

Lakekeeper berperan sebagai *catalog server* REST yang menyimpan metadata Iceberg pada PostgreSQL, sehingga *engine* pemrosesan (Trino, Spark, Flink) dapat membaca dan menulis tabel Iceberg yang sama. Trino berperan sebagai *query engine* federatif yang memungkinkan kueri lintas ClickHouse, PostgreSQL, MySQL, dan Iceberg melalui satu jalur SQL. Kombinasi tersebut menghadirkan dua manfaat: *schema evolution* yang aman tanpa migrasi data fisik dan *time-travel* untuk *snapshot* historis. Pemilihan ini juga sejalan dengan asumsi *open source* pada batasan B-3 yang menghindari ketergantungan pada katalog terkelola berbayar.

II.4.6 Penyimpanan Objek dan Versi Data

Lapis penyimpanan objek menjadi landasan bagi *data lake*, *model registry*, dan *backup*. MinIO (MinIO, Inc. 2024) dipilih sebagai penyedia objek kompatibel S3 karena *footprint* ringan, dukungan enkripsi sisi server berbasis KMS, dan kemampuan dijalankan pada Kubernetes dengan modus distribusi maupun *single-node*. Versi data pada lapis objek dikelola oleh lakeFS (Treeverse Ltd. 2025) yang menambah model *branching* dan *commit* bergaya Git pada *bucket* objek, sehingga eksperimen yang membutuhkan jejak versi data dapat dirunut tanpa menggandakan berkas. *Encrypted storage* pada MinIO memakai *Server-Side Encryption with KMS* (SSE-KMS) dengan KES sebagai *gateway* ke OpenBao sebagaimana dirinci pada Subbab II.11.

Pada platform ini, MinIO disusun dengan *bucket* terpisah per kepentingan: *mlflow-artifacts* untuk artefak MLflow, *feast-registry* untuk *registry* Feast berbasis berkas, *velero-backups* untuk pencadangan, dan *evidently-reports* untuk laporan deteksi *drift*. lakeFS dipasang sebagai lapis tambahan di atas MinIO ketika eksperimen memerlukan *branching* data tanpa duplikasi fisik. Kombinasi MinIO dan lakeFS memberi keluwesan untuk dua pola pemakaian: penyimpanan datar untuk artefak yang tidak memerlukan versi serta penyimpanan bercabang untuk eksperimen yang memerlukan *point-in-time* pada data mentah. Pemantauan kapasitas dilakukan melalui metrik MinIO yang diekspos ke Prometheus dengan ambang peringatan pada 80%.

II.4.7 Penyimpanan Relasional dan Pencarian

Sebagian besar layanan platform memerlukan basis data relasional untuk menyimpan metadata. PostgreSQL (PostgreSQL Global Development Group 2025) dipakai sebagai basis data utama dan dioperasikan melalui *operator* CloudNativePG sebagaimana dibahas pada Subbab II.3. MySQL (Oracle Corporation 2025) dipakai khusus untuk *metadata store* pada layanan yang menuntut kompatibilitas wire-protocol MySQL, sedangkan OpenSearch (OpenSearch Foundation 2025) dipakai sebagai indeks pencarian dan penyimpanan grafik metadata yang menjadi tulang punggung DataHub. *Backup* pada ketiga lapis tersebut dijalankan oleh Velero dengan jadwal harian dan retensi tujuh hari.

Pemisahan tiga lapis penyimpanan tersebut mengikuti karakter beban kerja: PostgreSQL melayani transaksi metadata MLflow, Kubeflow Pipelines, Argo CD, OpenBao, dan Feast *registry*, MySQL melayani metadata DataHub pada *schema* GMS, dan OpenSearch melayani indeks pencarian DataHub serta *lineage graph* dengan dukungan kueri grafik. PostgreSQL dioperasikan dengan Cluster CRD Cloud-

NativePG dengan *barman-cloud* sebagai *plugin* pencadangan ke MinIO. MySQL berjalan sebagai *StatefulSet* dengan *persistent volume* pada *storage class* lokal. OpenSearch dioperasikan melalui *operator* OpenSearch dengan tiga peran (master, data, ingest) pada lingkungan *multi-node* atau satu peran gabungan pada lingkungan satu *node*.

II.5 Layanan Fitur (*Feature Store*)

II.5.1 Konsep dan Peran

Feature store berperan sebagai lapis penyatu antara produksi fitur dan konsumsi fitur. Lamer dkk. (2024) menjelaskan posisi *feature store* di antara *data lake*, *data warehouse*, dan *datamart*, sebagai komponen yang menyiapkan fitur untuk konsumsi model. Li dkk. (2023) menjabarkan arsitektur *feature store geo-distributed* dan menekankan pentingnya kontrak fitur yang seragam antara *batch* dan *online*. Kartakis dkk. (June 2022) memperlihatkan pengalaman menjalankan *feature store* pada lingkungan AWS yang menjadi rujukan banyak desain modern.

Tiga peran utama *feature store* adalah: (1) menyimpan fitur dengan kontrak skema yang konsisten lintas konsumen, (2) penyedia *point-in-time join* antara entitas dan fitur historis untuk pelatihan, dan (3) penyedia *lookup* fitur berlatensi rendah untuk *inference*. Tanpa *feature store*, ketiga peran tersebut harus dirakit ulang oleh setiap tim yang menyajikan model, sehingga muncul duplikasi kontrak dan risiko *train-serving skew* yang dibahas oleh Breck dkk. (2017). Platform ini menempatkan Feast sebagai *control plane feature store* dengan *offline store* ClickHouse, *online store* Valkey untuk fitur tabular dan agregat, serta Qdrant untuk fitur *embedding* vektor. Tiga lapis penyimpanan tersebut terhubung pada satu *FeatureView* sehingga konsumen pelatihan dan konsumen penyajian membaca kontrak yang sama.

II.5.2 Penyimpanan *Offline* dan *Online*

Penyimpanan *offline* dipakai untuk pelatihan dan *batch scoring*, sedangkan penyimpanan *online* dipakai untuk *inference* dengan latensi rendah. Tabel II.5 dan Tabel II.6 membandingkan beberapa pilihan teknologi untuk kedua peran tersebut. Feast (Feast Project 2024) dipilih sebagai kerangka *feature store* karena modularitas penyimpanan dan kontrak API yang konsisten. ClickHouse (ClickHouse Inc 2024) dipilih sebagai *offline store* karena kompresi *columnar* dan latensi kueri agregasi yang rendah, sedangkan Valkey (Valkey Contributors 2025) dipilih sebagai *online store* karena kompatibilitas *wire-protocol RESP* dengan ekosistem klien Redis dan

lisensi BSD yang tetap terbuka pasca-perubahan lisensi Redis.

Tabel II.5 Evaluasi Alternatif Penyimpanan *Offline* (OLAP)

Penyimpanan <i>Offline</i>	Maturitas	Performa Query	Dukungan Ingestion	Komunitas	Total
ClickHouse	5	5	4	4	18
Apache Pinot	4	4	5	3	16
Apache Druid	4	4	5	4	17
Apache Kudu	3	3	4	3	13

Tabel II.6 Evaluasi Alternatif Penyimpanan *Online* (Latensi Rendah)

Penyimpanan <i>Online</i>	Maturitas	Latensi	Throughput	Lisensi <i>Open</i>	Total
Valkey (RESP)	5	5	5	5	20
Redis <i>Community</i>	5	5	5	3	18
DragonflyDB	4	5	5	3	17
Apache Cassandra	5	4	5	5	19

Karakter beban kerja antara *offline* dan *online* secara mendasar berbeda. *Offline store* ClickHouse dioptimasi untuk kueri agregasi pada jutaan baris dengan kompresi kolumnar LZ4 atau ZSTD, sehingga cocok untuk pembuatan *training dataset* dengan *point-in-time join*. *Online store* Valkey memakai struktur data kunci-nilai pada memori dengan latensi p99 sub-milidetik per *lookup*, sehingga cocok untuk *inference* yang membutuhkan ribuan *lookup* per detik. *Materialization* dari *offline* ke *online* dijadwalkan oleh Argo Workflows berdasarkan FeatureView pada Feast, dengan dukungan inkremental untuk fitur yang sering berubah dan dukungan penuh untuk fitur historis.

II.5.3 Pencarian Vektor dan HNSW

Fitur *embedding* berdimensi tinggi memerlukan struktur indeks pendekatan tetangga terdekat (*approximate nearest neighbor*, ANN). Malkov dan Yashunin (2020) memperkenalkan *Hierarchical Navigable Small World* (HNSW) sebagai struktur indeks dengan keseimbangan baik antara kecepatan kueri dan kualitas hasil. Alternatif lain seperti *Product Quantization* (Jegou, Douze, dan Schmid 2011) dan pencarian skala miliar dengan GPU (Johnson, Douze, dan Jégou 2021) relevan untuk skala yang berbeda. Untuk platform ini, Qdrant (Qdrant Team 2025) dipilih sebagai *vector index* terpisah dari *online store* tabular karena dukungan HNSW *native*, mode *gRPC* berlatensi rendah, dan kemampuan filter pada *payload* yang menyatu dengan

kueri kemiripan. Tabel II.7 merangkum perbandingan beberapa pilihan *vector index open-source* pada empat kriteria yang relevan dengan kebutuhan platform.

Tabel II.7 Evaluasi Alternatif *Vector Index Open-Source* untuk Pencarian Pendekatan Tetangga Terdekat

<i>Vector Index</i>	Maturitas	Latensi Rendah	Komunitas	Operabilitas K8s	Total
Qdrant (HNSW)	5	5	4	5	19
Milvus	5	5	4	4	18
Weaviate	4	4	4	4	16
pgvector (PostgreSQL)	5	3	4	4	16

Pemilihan Qdrant juga didasarkan pada dukungan terhadap *hybrid search* yang menggabungkan kueri vektor dengan filter atribut, sebagaimana dibahas oleh Bruch, Gai, dan Ingber (2023) sebagai pola pencarian yang banyak diadopsi pada sistem *retrieval* modern. Devlin dkk. (2019) memberi konteks akademik mengenai sumber *embedding* dari model *language* yang banyak dipakai sebagai masukan ke *vector index*. Qdrant dioperasikan melalui Cluster CRD dengan dua mode *collection*: Cosine untuk kemiripan kosinus pada *embedding* dinormalkan dan Dot untuk produk titik pada *embedding* mentah. Indeks HNSW dikonfigurasi dengan parameter *m* dan *ef_construct* yang dapat ditingkatkan ketika rasio panggilan ulang ingin diperbesar.

II.5.4 *Point-in-Time Correctness*

Jamianan *point-in-time correctness* berarti nilai fitur yang dipakai saat pelatihan model harus identik dengan nilai yang tersedia pada saat itu, bukan nilai versi terkini. Wang dan Ruf (2021) memperlihatkan dampak buruk kebocoran temporal pada *backtesting* keuangan, dan analoginya berlaku pada beragam domain. Feast menyediakan operasi *get_historical_features* yang melakukan *point-in-time join* antara entitas dan riwayat fitur, sehingga konsistensi nilai antara fase pelatihan dan fase *inference* dapat ditegakkan. Vela dkk. (2022) menambah perspektif evaluasi temporal pada pemodelan urut waktu yang sejalan dengan kebutuhan *point-in-time*.

Pada platform ini, jaminan *point-in-time* dijamin oleh dua mekanisme. Pertama, FeatureView pada Feast mendaftarkan kolom *event_timestamp* dan *created_timestamp* yang dipakai untuk *join* historis. Kedua, *online store* memakai versi terbaru dari nilai fitur sehingga *lookup* saat *inference* membaca nilai yang konsisten dengan *event_timestamp* terkini. Kombinasi keduanya menyebabkan kebocoran tempo-

ral tidak terjadi pada fase pelatihan dan tidak terjadi pada fase *inference*. Campese, Lauriola, dan Moschitti (2024) memperkuat argumen tersebut dengan menempatkan koherensi temporal sebagai sifat kelas pertama bagi sistem fitur.

Tabel II.8 Evaluasi Alternatif *Feature Store Open-Source*

<i>Feature Store</i>	Maturitas	Ekstensibilitas	Komunitas	Integrasi K8s	Total
Feast	5	5	5	5	20
Hopsworks <i>Feature Store</i>	4	4	3	4	15
Feathr (LinkedIn)	3	3	2	3	11
Tecton (basis <i>open-source</i>)	4	3	2	3	12

II.6 Siklus Hidup Model

II.6.1 Eksperimen dan *Tracking*

MLflow (MLflow Contributors 2024) menyediakan *tracking* eksperimen, *registry* model, dan API penyebaran. Pimentel dkk. (2019) menyoroti masalah reproduktibilitas *notebook*, yang menjadi alasan mengapa *tracking* terpusat menjadi prasyarat. Tabel II.9 membandingkan beberapa pilihan platform *tracking*. Anaconda, Inc. (July 2020) memberikan konteks lapangan mengenai praktik kerja *data scientist* yang dapat dijadikan acuan saat memilih alat *tracking*.

Tabel II.9 Evaluasi Alternatif *Experiment Tracking* dan *Model Registry*

<i>Tracking & Registry</i>	Maturitas	Ekstensibilitas	Komunitas	Integrasi	Total
MLflow	5	5	5	5	20
DVC (<i>Data Version Control</i>)	4	4	4	3	15
TensorBoard	4	2	4	2	12
Neptune	3	3	3	3	12

Pada platform ini, MLflow dijalankan sebagai Deployment dengan PostgreSQL sebagai *backend store* dan MinIO sebagai *artifact store*. *Tracking server* diintegrasikan dengan Dex sebagai *identity provider* OIDC dan oauth2-proxy sebagai *gateway* autentikasi, sehingga konsol MLflow berbagi sesi dengan konsol pengembang lain. *Model registry* dipakai sebagai sumber kebenaran versi model yang akan dipromosi oleh *pipeline continuous training* ke KServe InferenceService. Setiap *run* ML-

flow juga melaporkan *lineage* ke DataHub melalui adaptor OpenLineage agar jejak eksperimen tertaut ke jejak data masukan.

II.6.2 Orkestrasi Pelatihan

Kubeflow Pipelines (Kubeflow Contributors 2024) dan Argo Workflows (Argo Project 2025) menjadi dua pilihan utama untuk orkestrasi pelatihan pada Kubernetes. Apache Airflow (Apache Software Foundation 2024a) masih relevan untuk *pipeline* data berorientasi *batch*. Kubeflow Pipelines dipakai untuk *pipeline* pelatihan dan penerapan model, sedangkan Argo Workflows dipakai pada *control loop* berbentuk CronWorkflow untuk memicu deteksi *drift* dan pelatihan ulang. Tabel II.10 membandingkan ketiganya berdasarkan kebutuhan domain.

Tabel II.10 Evaluasi Alternatif Orkestrasi ML *Open-Source*

Orkestrasi ML	Maturitas	K8s-Native	Pipeline ML	Komunitas	Total
Kubeflow Pipelines	4	5	5	4	18
Apache Airflow	5	3	3	5	16
Argo Workflows	4	5	3	4	16
Prefect	3	2	3	3	11

Kubeflow Pipelines dipilih untuk jalur pelatihan karena dukungan *component* berbasis kontainer dengan kontrak input dan output yang ditegakkan oleh Pipeline IR (Intermediate Representation). Argo Workflows dipilih untuk *control loop* karena dukungan CronWorkflow yang ringan dan kompatibel dengan kebijakan *retries* pada level langkah. Pemisahan kedua *orchestrator* dilakukan agar *pipeline* pelatihan dapat dipromosikan ke produksi tanpa mengganggu *control loop* pemantauan. Kedua *orchestrator* berjalan pada *namespace* terpisah dengan akses ke *registry* ML-flow dan Feast melalui Service Kubernetes berlabel *istio-injection=enabled* agar mTLS Istio berlaku.

II.6.3 Eksperimen Interaktif dan *Tuning*

Kubeflow Notebooks (Kubeflow Authors 2025b) menyediakan lingkungan *notebook* multi-pengguna di atas Kubernetes dengan tata kelola sumber daya per pengguna. Untuk *hyperparameter tuning*, Katib (Kubeflow Authors 2025a) menjalankan pencarian *grid*, *random*, *Bayesian*, dan *neural architecture search* melalui Experiment CRD dengan integrasi terhadap Kubeflow Pipelines sebagai eksekutor. Kubeflow Trainer (Kubeflow Authors 2025c) memperluas dukungan ke pelatihan terdistribusi dengan *operator* TrainJob untuk PyTorch dan TensorFlow. Vangala, Mehta, dan

Singh (2022) memberi gambaran praktik MLOps di lapangan yang menempatkan ketiga komponen tersebut sebagai bagian dari satu lapis pengembangan model.

Tata kelola sumber daya per pengguna pada Kubeflow Notebooks dijalankan melalui *Profile* CRD yang menentukan *namespace* pengguna, kuota sumber daya, dan akses ke *Profile* lain. Setiap *Profile* memiliki *ServiceAccount* sendiri sehingga pengguna dapat mengakses Feast, MLflow, dan Kubeflow Pipelines tanpa berbagi kredensial. Katib menjalankan *trial* sebagai Job terpisah dengan parameter yang diberi label sehingga hasilnya dapat dirunut pada konsol Katib. Kubeflow Trainer dijalankan secara opsional pada lingkungan multi-*node* untuk pelatihan model dalam dengan dukungan komunikasi NCCL antar GPU.

II.6.4 Penyajian Model

KServe (KServe Contributors 2024) menyediakan kerangka penyajian model di atas Kubernetes dengan dukungan banyak *runtime*, otomatisasi penskalaan, dan integrasi dengan praktik GitOps. Hsu dkk. (2024) memperlihatkan ujicoba penyebaran terotomasi dengan KServe yang sejalan dengan pendekatan platform ini. Liang dkk. (2024) menyoroti integrasi sistem dengan *machine learning* sebagai pendorong otomatisasi pelatihan dan penyebaran. Ahmad dkk. (June 2024) memberikan tinjauan strategi keamanan MLOps yang relevan untuk lapis penyajian.

Tabel II.11 Evaluasi Alternatif *Model Serving Open-Source*

<i>Model Serving</i>	Maturitas	Skalabilitas	<i>Framework</i>	Komunitas	Total
KServe	4	5	5	4	18
Seldon Core	4	4	4	4	16
BentoML	3	3	4	3	13
TorchServe	3	3	2	3	11

Pada platform ini, KServe dijalankan melalui *InferenceService* CRD dengan dukungan *predictor*, *transformer*, dan *explainer* yang dapat dirangkai pada satu jalur. *Runtime* yang dipakai meliputi MLflow *Server* untuk model MLflow, Triton *Inference Server* untuk model dalam, dan *ServingRuntime* kustom untuk model dengan kebutuhan khusus. Penskalaan ke nol disediakan oleh Knative *Serving* sebagai fondasi KServe, sedangkan *traffic split* antar versi model dijalankan oleh Argo Rollouts dengan strategi *canary*. Pemantauan kualitas prediksi dilakukan melalui *transformer* pra-prediksi yang melaporkan distribusi masukan ke Prometheus untuk dideteksi oleh *job drift*.

II.7 Tata Kelola Data: Katalog, *Lineage*, dan Kualitas

Bernardo dkk. (2024) memetakan tata kelola data sebagai disiplin lintas fungsi yang mencakup metadata, kualitas, akses, dan kepemilikan. Stoyanovich, Howe, dan Jagdish (2020) memberi konteks etis dengan menempatkan tanggung jawab atas data pada level platform, bukan hanya level aplikasi. Chien (February 2022) memberi panduan praktis untuk peningkatan kualitas data berbasis pengukuran. Empat aspek tersebut diwujudkan oleh tiga komponen yang saling melengkapi pada platform ini: DataHub sebagai katalog, Great Expectations sebagai kontrak kualitas, dan OpenLineage sebagai protokol *lineage* terbuka.

DataHub (DataHub Project 2024) dipilih sebagai katalog metadata karena dukungan integrasi luas dengan komponen DataOps dan MLOps. Great Expectations (Great Expectations 2024) dipakai untuk pengujian kontrak data yang dijalankan pada *pipeline* produksi. OpenLineage (OpenLineage Authors 2025) dipakai sebagai standar terbuka pengumpulan *lineage* antar komponen *pipeline*, sehingga jejak dari ingestasi hingga model dapat disusun dengan kosa kata yang seragam. Tabel II.12 merangkum perbandingan beberapa pilihan katalog. Kontrak kualitas data dikemas sebagai *expectation suite* yang disimpan pada repositori Gitea dan diuji oleh Great Expectations sebelum data ditulis ke ClickHouse, dengan jejak hasil diserahkan ke DataHub melalui adaptor.

Tabel II.12 Evaluasi Alternatif Manajemen Metadata dan *Governance*

Metadata & Governance	Maturitas	Ekstensibilitas	Komunitas	Integrasi	Total
DataHub (LinkedIn)	5	5	5	5	20
Apache Atlas	4	4	3	4	15
OpenMetadata	3	4	4	4	15
Amundsen (Lyft)	3	3	3	3	12

II.8 Deteksi *Drift*

Lu dkk. (2019) memberikan tinjauan luas tentang pembelajaran di bawah *concept drift*. Bayram, Ahmed, dan Kassler (2022) mengelompokkan keluarga detektor berbasis performa, dan Hinder dkk. (2023) memperluas pembahasan ke arah penjelasan model. Beberapa metrik yang relevan untuk *covariate drift* dan *prior probability shift* antara lain *Population Stability Index* (PSI), uji Kolmogorov-Smirnov (Reis dkk. 2016), dan jarak Wasserstein. Céspedes Sisniega dkk. (2024) memperlihatkan implementasi deteksi *drift* pada lingkungan *serverless*, sementara Jourand dkk.

(2023) membahas penanganan *drift* pada *deep learning* untuk pemantauan proses.

Pada platform ini, PSI dan uji Kolmogorov-Smirnov dipakai sebagai detektor utama, dengan ambang batas konfigurasi yang diatur per fitur. Detektor dijalankan oleh CronWorkflow Argo sebagai *control loop* yang memicu pelatihan ulang melalui Kubeflow Pipelines apabila ambang dilewati. Evidently (Evidently AI 2025) dipakai sebagai *library* pelaporan *drift* yang menulis laporan HTML ke MinIO dan mengekspos metrik agregat ke Prometheus. *Reference distribution* disimpan pada MinIO dengan versi yang diberi label prod-vN, sehingga perbandingan distribusi dapat dirunut ke versi referensi tertentu.

II.9 Observabilitas *Cloud-Native*

Cloud Native Computing Foundation (2024b) dan IBM (2024) mendefinisikan observabilitas sebagai gabungan tiga pilar: metrik, log, dan *trace*. Untuk platform ini, Prometheus (Prometheus Authors 2024) dipakai untuk metrik, Loki (Grafana Labs 2024b) untuk log, dan Grafana Tempo (Grafana Labs 2024c) untuk *trace*, dengan OpenTelemetry sebagai *collector* bersama dan Grafana (Grafana Labs 2024a) sebagai antarmuka visualisasi. Sloth (Sloth Authors 2025) membantu menurunkan definisi *service-level objective* ke dalam aturan Prometheus, sedangkan Evidently (Evidently AI 2025) memberi laporan kualitas data dan model yang melengkapi pilar observabilitas. Pyroscope (Grafana Labs 2025) menyediakan *continuous profiling* untuk profil CPU dan memori, Pushgateway (Prometheus Authors 2025) menjadi titik kumpul metrik untuk *job* berdurasi pendek di luar siklus *scrape*, dan OpenCost (OpenCost Authors 2025) memantau biaya per beban kerja pada kluster.

Apache Superset (Apache Superset Authors 2025) berperan sebagai antarmuka BI yang menyatukan kueri SQL lintas ClickHouse, Trino, dan MySQL pada satu lapis dasbor. Tabel II.13 merangkum perbandingan beberapa pilihan *BI open-source*. Pemantauan tambahan untuk kualitas data dan model dirangkai sebagai *dashboard* pada Grafana sehingga pengamatan terhadap kondisi sistem dapat dilakukan dari satu antarmuka. Tabel II.14 merangkum perbandingan beberapa pilihan tumpukan observabilitas. Korelasi antara metrik, log, dan *trace* dimungkinkan melalui label *trace_id* yang dilampirkan pada baris log Loki dan rentang Tempo, sehingga pelacakan kegagalan dapat berpindah antar pilar tanpa kehilangan konteks.

Tabel II.13 Evaluasi Alternatif *Business Intelligence* dan *Dashboarding Open-Source*

BI Tool	Maturitas	Konektor SQL	Multi-Tenant K8s	Komunitas	Total
Apache Superset	5	5	5	5	20
Metabase	4	4	4	5	17
Redash	4	4	3	3	14
Lightdash	3	4	3	3	13

Tabel II.14 Evaluasi Alternatif *Monitoring* dan *Observability*

Monitoring & Observability	Maturitas	Ekstensibilitas	Komunitas	Integrasi K8s	Total
Prometheus + Grafana	5	5	5	5	20
InfluxDB + Chronograf	4	4	3	4	15
Elastic Stack (ELK)	5	4	4	4	17
Jaeger + OpenTelemetry	4	4	4	5	17

II.10 GitOps dan *Continuous Delivery*

GitOps menempatkan repositori Git sebagai sumber kebenaran tunggal untuk konfigurasi sistem. Limoncelli (2022) menjelaskan pola GitOps sebagai pendekatan IT *self-service* yang memetakan operasi ke kontrol versi. Argo CD (Argo Project 2024) dipakai sebagai *operator* GitOps yang merekonsiliasi keadaan *cluster* dengan deklarasi pada Git. Kim dkk. (2016) memberi dasar teoretis tentang aliran kerja CI/CD yang menjadi induk GitOps.

Repositori sumber kebenaran dijalankan secara mandiri pada Gitea (Gitea Contributors 2025) sebagai layanan Git *self-hosted* yang ringan dan kompatibel dengan API yang umum, sehingga seluruh manifest, *chart* Helm, dan *overlay* Kustomize berada pada satu bidang kendali yang dapat ditarik oleh Argo CD tanpa ketergantungan terhadap penyedia eksternal. Tekton (Tekton Contributors 2024) dipakai sebagai *pipeline* CI yang membangun *image* dan menjalankan pengujian sebelum komit menjadi sumber penyebaran. Argo Rollouts (Argo Rollouts Authors 2025) melengkapi rantai dengan strategi penyebaran progresif berbasis *canary* dan analisis metrik. Tabel II.15 merangkum perbandingan beberapa *GitOps controller open-source* berdasarkan kriteria yang relevan dengan platform.

Tabel II.15 Evaluasi Alternatif *GitOps Controller Open-Source*

<i>GitOps Controller</i>	Maturitas	K8s-Native	Antarmuka	Komunitas	Total
Argo CD	5	5	5	5	20
Flux CD	5	5	3	4	17
Rancher Fleet	4	5	3	3	15
Jenkins X	3	4	3	2	12

II.11 Keamanan Platform: Identitas, Rahasia, *Mesh*, dan Kebijakan

Lapisan keamanan menjadi prasyarat bagi platform yang melayani DataOps dan MLOps secara berdampingan. Empat aspek dipertimbangkan: identitas pengguna terpadu, pengelolaan rahasia, *service mesh*, dan *policy engine*. Empat aspek tersebut bekerja sebagai lapisan pertahanan yang saling melengkapi, sesuai prinsip *defense in depth* pada Bondi (2000). Pelanggaran pada satu lapis tidak otomatis membuka akses ke lapis berikutnya sebab tiap lapis menegakkan kontrol yang berbeda jenis: identitas mengontrol *siapa*, *secret* mengontrol *apa yang dipegang*, *mesh* mengontrol *siapa berbicara ke siapa*, dan kebijakan mengontrol *apa yang boleh dibuat di cluster*.

Untuk identitas, Dex (Dex Authors 2025) dipakai sebagai *identity provider* berbasis OpenID Connect (OIDC) karena *footprint* ringan, dukungan federasi terhadap penyedia identitas eksternal, dan integrasi sederhana dengan *cluster* Kubernetes. Untuk antarmuka konsol yang tidak menyediakan OIDC secara *native*, oauth2-proxy (OAuth2 Proxy Authors 2025) dipakai sebagai *reverse proxy* yang menanyakan sesi Dex sebelum meneruskan permintaan ke layanan, sehingga seluruh konsol pengembang berbagi satu jalur autentikasi. Untuk otorisasi berbutir halus pada layanan yang membutuhkan kontrol relasi, SpiceDB (AuthZed 2025) menyediakan basis data otorisasi gaya Google Zanzibar dengan skema relasi deklaratif dan API permintaan izin yang konsisten. Tabel II.16 merangkum perbandingan beberapa *identity provider open-source* berdasarkan kriteria yang relevan dengan platform.

Tabel II.16 Evaluasi Alternatif *Identity Provider Open-Source*

<i>Identity Provider</i>	Bobot <i>Footprint</i>	Federasi OIDC	Operabilitas K8s	Maturitas	Total
Dex <i>IdP</i>	5	5	5	5	20
Keycloak	3	5	4	5	17
Authelia	4	4	3	4	15
Authentik	3	4	3	4	14

Untuk pengelolaan rahasia, OpenBao (OpenBao Authors 2025) dipakai sebagai *successor open-source* dari HashiCorp Vault yang menjamin penyimpanan rahasia terenkripsi serta penerbitan kredensial dinamis. Integrasi pada beban kerja dilakukan melalui External Secrets Operator (External Secrets Authors 2025) yang menyalin rahasia ke Secret Kubernetes dengan rotasi otomatis melalui CRD ExternalSecret dan ClusterSecretStore. Enkripsi data *at rest* pada MinIO dijumpai oleh MinIO KES (MinIO, Inc. 2025) sebagai *stateless KMS gateway* yang merutekan permintaan kunci ke OpenBao tanpa menyimpan kunci pada *cluster*. Tabel II.17 merangkum perbandingan beberapa pengelola rahasia *open-source*.

Tabel II.17 Evaluasi Alternatif Pengelola Rahasia *Open-Source*

Pengelola Rahasia	Lisensi Open	Rotasi Dinamis	Integrasi ESO	Maturitas	Total
OpenBao	5	5	5	4	19
HashiCorp Vault Community	3	5	5	5	18
Bitnami Sealed Secrets	5	2	3	5	15
Mozilla SOPS	5	2	3	4	14

Untuk komunikasi antar-layanan, Istio (Istio Authors 2024) dipakai sebagai *service mesh* yang menyediakan *mutual TLS* (mTLS) seragam, kontrol L7 berbasis VirtualService dan AuthorizationPolicy, serta integrasi telemetri dengan OpenTelemetry. Tabel II.18 merangkum perbandingan beberapa *service mesh open-source* berdasarkan kriteria fitur, ekosistem, dan kematangan. Pada perbatasan *mesh*, Apache APISIX (Apache APISIX Authors 2025) berperan sebagai *API gateway* yang menyaring lalu lintas masuk, menerapkan kebijakan kuota, dan menjembatani mTLS terhadap konsumen di luar *mesh*. Tabel II.19 merangkum perbandingan beberapa *API gateway open-source*.

Tabel II.18 Evaluasi Alternatif *Service Mesh Open-Source*

Service Mesh	Maturitas	Fitur L7	Ekosistem CRD	Komunitas	Total
Istio ambient/sidecar	5	5	5	5	20
Linkerd	4	3	3	4	14
Cilium Service Mesh	4	4	4	4	16
Consul Connect	4	3	3	3	13

Tabel II.19 Evaluasi Alternatif *API Gateway Open-Source*

<i>API Gateway</i>	Maturitas	Lisensi Open	Operabilitas K8s	Plugin/Ekstensi	Total
Apache APISIX	5	5	5	5	20
Kong <i>Gateway OSS</i>	5	4	4	5	18
Emissary Ingress (Ambassador)	4	5	4	3	16
NGINX Ingress	5	5	4	3	17

Untuk penegakan kebijakan, Kyverno (Kyverno Authors 2024) dipakai sebagai *policy engine native* Kubernetes dengan sintaks YAML yang ringkas serta dukungan *mutate*, *validate*, dan *generate*. Open Policy Agent (OPA) Gatekeeper (Open Policy Agent Authors 2024) juga umum dipakai sebagai alternatif berbasis Rego. Tabel II.20 merangkum perbandingan beberapa *policy engine open-source*.

Tabel II.20 Evaluasi Alternatif *Policy Engine Open-Source*

<i>Policy Engine</i>	Sintaks Native	Mode Mutate	Pelaporan Pelanggaran	Komunitas	Total
Kyverno	5	5	5	5	20
OPA Gatekeeper	3	4	4	5	16
jsPolicy	4	3	3	3	13
Kubewarden	4	4	3	3	14

Untuk pemantauan ancaman pada lapis *runtime*, Falco (Falco Authors 2025) dipakai sebagai detektor berbasis eBPF yang memantau *syscall* dan menerbitkan peristiwa keamanan, sedangkan Trivy Operator (Aqua Security 2025) memindai *image* dan beban kerja secara terjadwal. Pencadangan *cluster* dan *persistent volume* dijaga oleh Velero (Velero Authors 2025), dan uji ketahanan dijalankan dengan Chaos Mesh (Chaos Mesh Authors 2025) pada *namespace* eksperimen. Tabel II.21 merangkum perbandingan beberapa *runtime security open-source*.

Tabel II.21 Evaluasi Alternatif *Runtime Security Open-Source* pada Kubernetes

<i>Runtime Security</i>	Maturitas	Cakupan eBPF/Kernel	Aturan Bawaan	Komunitas	Total
Falco (CNCF)	5	5	5	5	20
Cilium Tetragon	4	5	4	4	17
Aqua Tracee	3	5	3	3	14
Sysdig Secure <i>OSS</i>	4	4	4	3	15

II.12 Penelitian Terkait dan Posisi Penelitian

Beberapa penelitian terdahulu menjadi tetangga dekat platform ini. Burgueno-Romero, Cánovas Izquierdo, dan García-García (2025) mengusulkan arsitektur MLOps *open source*, namun belum memasukkan layanan fitur dengan dukungan vektor sebagai komponen kelas pertama. Amou Najafabadi dkk. (2024) memetakan beragam arsitektur MLOps tetapi tidak menggabungkan DataOps secara menyeluruh. Rodrigues dkk. (2025) menyentuh *near real-time stream learning*, namun tidak meletakkan tata kelola data sebagai bidang kendali yang ditegakkan. Ahmad dkk. (June 2024) membahas strategi keamanan MLOps, dan dengan demikian melengkapi sudut pandang yang dibahas pada platform ini di bagian kebijakan jaringan dan manajemen rahasia.

Platform yang dikembangkan pada Tugas Akhir ini menggabungkan tiga karakter yang belum dilingkupi sekaligus pada penelitian sebelumnya: integrasi DataOps dan MLOps pada satu bidang kendali Kubernetes, layanan fitur *dual-store* dengan dukungan vektor HNSW, dan deteksi *drift* terotomasi yang memicu pelatihan ulang melalui kontrol GitOps. Hsu dkk. (2024) dan Liang dkk. (2024) memberikan basis komparatif yang dipakai pada Bab III ketika menilai alternatif. Penelitian ini menambahkan kontribusi berupa kontrak *lineage* terbuka melalui OpenLineage dan kontrak kualitas data melalui Great Expectations yang menempel pada *pipeline* produksi. Tabel II.22 menyajikan posisi platform ini terhadap penelitian terkait.

Tabel II.22 Perbandingan Detail Arsitektur Saat Ini dengan yang Diusulkan

Aspek	Arsitektur Saat Ini	Arsitektur Diusulkan	Keuntungan Terukur	Referensi
Orkestrasi	<i>Compute</i> heterogen dengan <i>provisioning</i> manual, waktu tunggu <i>scaling</i> panjang, dan tingkat standarisasi rendah.	Platform Kubernetes terpadu dengan manajemen deklaratif dan alokasi sumber daya otomatis, sejalan dengan komponen arsitektur MLOps yang diidentifikasi Kreuzberger, Köhl, dan Hirschl (2023). GitOps dengan Argo CD memastikan perubahan infrastruktur dan aplikasi mengikuti <i>single source of truth</i> di Git.	Lakka (2025) menunjukkan bahwa adopsi Kubernetes pada perusahaan disertai praktik DevOps yang matang meningkatkan <i>deployment frequency</i> dan menekan <i>Mean Time To Recovery</i> (MTTR) secara signifikan, dan arsitektur yang diusulkan menargetkan karakteristik performa pada arah yang sama.	(Kreuzberger, Köhl, dan Hirschl 2023; Lakka 2025)
Manajemen Fitur	Logika fitur tersebar di <i>notebook</i> dan skrip dengan duplikasi tinggi dan banyak fungsi yang saling tumpang tindih.	Feast dengan <i>feature registry</i> terpusat, <i>dual serving offline/online</i> , serta dukungan <i>point-in-time correctness</i> .	Duplikasi fitur dapat ditekan, <i>training-serving skew</i> berkurang melalui pemisahan <i>offline</i> dan <i>online</i> serta <i>point-in-time retrieval</i> , dan <i>feature reuse</i> berpotensi meningkat signifikan.	(Munappy dkk. 2022; Polyzotis dkk. 2018)

Aspek	Arsitektur Saat Ini	Arsitektur Diusulkan	Keuntungan Terukur	Referensi
<i>Vector Embeddings</i>	Fitur <i>embedding</i> tidak didukung secara <i>native</i> atau diimplementasikan secara kustom dengan latensi tinggi dan <i>throughput</i> terbatas.	Qdrant sebagai indeks vektor terpisah dari <i>online store</i> tabular Valkey, dengan HNSW <i>native</i> , <i>API gRPC</i> , dan filter <i>payload</i> .	Johnson, Douze, dan Jégou (2021) menunjukkan bahwa indeks vektor berbasis HNSW maupun FAISS dapat melayani <i>billion-scale similarity search</i> dengan latensi rendah. Desain ini memakai Qdrant agar fitur <i>embedding</i> dilayani secara <i>online</i> tanpa membebani <i>online store</i> kunci-nilai dan tanpa membangun layanan kustom.	(Johnson, Douze, dan Jégou 2021; Qdrant Team 2025)
Validitas Temporal	<i>Point-in-time join</i> dilakukan manual dan rentan kesalahan, sehingga banyak tim mengalami <i>data leakage</i> .	<i>Point-in-time join</i> otomatis dari Feast dengan pola verifikasi yang lebih sistematis.	Pendekatan ini mencegah kebocoran data temporal yang dapat menghasilkan estimasi performa <i>backtest</i> yang terlalu optimistis dan tidak realistis di produksi.	(Polyzotis dkk. 2018)

Aspek	Arsitektur Saat Ini	Arsitektur Diusulkan	Keuntungan Terukur	Referensi
Pemrosesan Data	<i>Batch</i> dan <i>stream</i> diproses oleh <i>pipeline</i> terpisah tanpa arsitektur yang koheren, sehingga hasil pemrosesan tidak konsisten.	Spark sebagai mesin <i>batch processing</i> dan Flink sebagai mesin <i>stream processing</i> yang hasilnya dimaterialisasikan langsung ke <i>Feature Store</i> . Airflow mengorkestrasi <i>pipeline</i> transformasi data sebagai DAG sehingga pengembangan dan penjadwalan transformasi menjadi lebih sistematis.	Waktu pengembangan <i>pipeline</i> dapat menurun dan <i>freshness</i> data meningkat berkat pemrosesan <i>streaming</i> yang lebih efisien dan terintegrasi.	(Rodrigues dkk. 2025)
Pelatihan Model	<i>Workflow</i> pelatihan berbasis <i>notebook</i> dengan <i>tracking</i> eksperimen informal dan <i>artifact</i> tidak terkelola.	Kubeflow <i>Pipelines</i> untuk orkestrasi pelatihan dan MLflow untuk <i>tracking</i> eksperimen.	Reprodusibilitas meningkat melalui kontainerisasi dan pengelolaan eksperimen terpusat, sementara siklus pengembangan <i>pipeline</i> menjadi lebih terstruktur.	(Amershi dkk. 2019; Pimentel dkk. 2019)

Aspek	Arsitektur Saat Ini	Arsitektur Diusulkan	Keuntungan Terukur	Referensi
Deployment Model	Proses <i>deployment</i> manual dengan <i>refactoring</i> besar dan sering terjadi <i>bug</i> pasca- <i>deploy</i> .	KServe dengan <i>InferenceService</i> deklaratif dan <i>deployment</i> otomatis melalui GitOps di atas Knative Serving. OpenTelemetry mengumpulkan log dan <i>trace</i> inferensi ke Loki dan Grafana Tempo sebagai dasar <i>monitoring</i> dan analitik lanjutan.	Waktu <i>deployment</i> berpotensi turun secara signifikan, dan insiden terkait kesalahan konfigurasi <i>deployment</i> dapat dikurangi lewat otomasi.	(Paleyes, Urma, dan Lawrence 2022)
Pola Deployment Lanjut	Pola seperti <i>canary release</i> jarang digunakan, organisasi umumnya mengandalkan <i>binary switchover</i> .	Dukungan <i>canary release</i> , A/B <i>testing</i> , dan <i>automatic rollback</i> prediktif.	<i>Rollout</i> dapat dilakukan secara bertahap dan aman, sehingga MTTR menurun karena kegagalan dapat dibatasi pada <i>canary traffic</i> .	(Shankar dkk. 2022)
Deteksi Drift	Deteksi <i>drift</i> bersifat reaktif dengan jeda deteksi yang panjang terhadap isu penting dan perubahan gradual.	<i>Monitoring</i> pro-aktif dengan uji statistik, <i>alert real-time</i> , dan penetapan ambang adaptif.	Jeda deteksi <i>drift</i> dapat dipersingkat sehingga <i>retraining</i> dapat dijadwalkan lebih tepat waktu untuk menjaga kinerja model.	(Céspedes Sisniega dkk. 2024; Bayram, Ahmed, dan Kassler 2022)
Governance & Lineage	Dokumentasi dilakukan secara manual dan <i>audit trail</i> sering tidak lengkap atau tersebar.	DataHub dengan <i>metadata ingestion</i> otomatis dan <i>lineage</i> yang dapat ditelusuri dari <i>source</i> hingga model, dibantu OpenLineage sebagai protokol pengirim peristiwa lintas <i>engine</i> pemrosesan.	Waktu persiapan audit dapat berkurang dan cakupan metadata meningkat melalui proses <i>ingestion</i> dan <i>lineage</i> otomatis.	(Stoyanovich, Howe, dan Jagadish 2020; Lamer dkk. 2024)

Aspek	Arsitektur Saat Ini	Arsitektur Diusulkan	Keuntungan Terukur	Referensi
<i>Observability</i>	Fokus <i>monitoring</i> pada metrik infrastruktur, tanpa cakupan metrik ML spesifik di sebagian besar organisasi.	<i>Observability</i> mencakup metrik ML, <i>dashboard</i> kinerja model, dan <i>alert</i> berbasis indikator <i>drift</i> . Prometheus dan Grafana menjadi inti, OpenTelemetry mengalirkan log dan <i>trace</i> ke Loki dan Grafana Tempo, sementara Alertmanager mengonsolidasikan <i>alert</i> ke berbagai kanal notifikasi.	Proses <i>debugging</i> dan <i>root cause analysis</i> menjadi lebih efektif sehingga MTTR dapat diturunkan.	(Shankar dkk. 2022)
Integrasi CI/CD	CI/CD untuk <i>pipeline</i> dan model dilakukan secara manual atau semi-otomatis, dengan <i>workflow</i> yang tidak seragam.	GitOps dengan Argo CD sebagai mekanisme <i>continuous deployment</i> deklaratif, dilengkapi Tekton untuk <i>continuous integration</i> dan Argo Rollouts untuk <i>progressive delivery</i> , dengan Git sebagai sumber kebenaran tunggal yang mencakup seluruh konfigurasi DataOps dan MLOps.	Frekuensi <i>deployment</i> dapat meningkat dan variasi kesalahan <i>deployment</i> dapat ditekan melalui otomasi dan <i>peer review</i> pada <i>pull request</i> .	(Kreuzberger, Kühl, dan Hirschl 2023)

Aspek	Arsitektur Saat Ini	Arsitektur Diusulkan	Keuntungan Terukur	Referensi
Pengalaman <i>Developer</i>	<i>Developer</i> menghabiskan banyak waktu untuk tugas repetitif, <i>onboarding</i> lama, dan penggunaan alat yang terpisah.	Platform terintegrasi dengan <i>self-service</i> untuk data, fitur, <i>pipeline</i> , dan akses. <i>dex</i> sebagai <i>identity provider</i> OIDC dan <i>oauth2-proxy</i> menyatukan otentikasi tunggal di seluruh antarmuka pengembang.	<i>Onboarding</i> dapat dipercepat dan produktivitas <i>data scientist</i> meningkat karena hambatan akses infrastruktur berkurang.	(Paleyes, Urma, dan Lawrence 2022; Rella 2022)
Efisiensi Biaya	Utilisasi sumber daya tidak optimal dan tidak ada atribusi biaya yang jelas ke <i>workload</i> .	<i>Auto-scaling</i> berbasis HPA, VPA, dan KEDA, pemantauan biaya melalui OpenCost, serta alokasi sumber daya yang lebih terukur pada kluster Kubernetes. Seluruh komponen inti berbasis <i>open source</i> sehingga biaya lisensi dapat ditekan.	Penghematan biaya signifikan dapat dicapai melalui optimasi pemakaian sumber daya dan <i>auto-scaling</i> , sebagaimana ditunjukkan dalam berbagai studi kasus adopsi Kubernetes (Lakka 2025). Desain ini mengadopsi prinsip yang sama untuk mengoptimalkan pemakaian sumber daya.	(Lakka 2025)

Aspek	Arsitektur Saat Ini	Arsitektur Diusulkan	Keuntungan Terukur	Referensi
Keamanan & Compliance	Kontrol akses beragam di tiap sistem, dan <i>audit</i> tersebar tanpa pandangan terpadu.	RBAC terintegrasi, <i>audit trail</i> komprehensif, dan pengecekan <i>compliance</i> yang terotomatisasi. dex sebagai <i>identity provider</i> OIDC memusatkan identitas, oauth2-proxy menegakkan otentikasi pada antarmuka aplikasi, Kyverno menegakkan kebijakan admisi pada klaster, OpenBao mengelola <i>secrets</i> , sementara Falco dan Trivy Operator memantau ancaman <i>runtime</i> dan kerentanan citra.	Risiko insiden keamanan dapat berkurang melalui kontrol terpusat dan proses audit menjadi lebih ringkas.	(Ahmad dkk. June 2024)

BAB III

ANALISIS MASALAH

Bab ini membahas analisis masalah pada pengembangan platform DataOps dan MLOps, mencakup pemetaan kondisi saat ini dengan tiga sumber tekanan utama yang sejajar dengan tiga pemicu pada Subbab I.1, perumusan kebutuhan fungsional dan nonfungsional, serta proses pemilihan solusi yang paling sesuai dengan tujuan penelitian. Pembahasan dijaga sejajar dengan rantai keterhubungan dari rumusan masalah pada Bab I agar tiap kebutuhan dapat dirunut ke akar masalah yang sama. Pemetaan kondisi pada Subbab III.1 mengikuti urutan pemicu pada latar belakang sehingga keterhubungan antara masalah, kebutuhan, alternatif, dan solusi yang dipilih dapat ditelusuri dari atas ke bawah. Subbab III.2 kemudian menurunkan empat belas kebutuhan fungsional dan dua belas kebutuhan nonfungsional yang menjadi tolok ukur kelayakan, dan Subbab III.3 memilih solusi terbaik melalui skor pemenuhan kebutuhan terhadap empat alternatif yang dipertimbangkan.

III.1 Analisis Kondisi Saat Ini

Kondisi pengembangan platform DataOps dan MLOps saat ini dipengaruhi oleh tiga sumber tekanan yang saling memperkuat. Pertama, beban konfigurasi puluhan komponen *open source* dari nol membatasi kecepatan pembangunan platform internal. Kedua, biaya berlangganan layanan terkelola membatasi kelayakan operasional pada skala produksi sekaligus menghadirkan pertukaran antara waktu pengembangan dan anggaran berjalan. Ketiga, fragmentasi yang merambat lintas peran (*business user*, *data engineer*, *data scientist*, dan *machine learning engineer*) memicu efek domino pada tata kelola data, deteksi *drift*, dan penyajian fitur multimoda. Ketiga subbab berikut menelaah setiap sumber tekanan tersebut secara berurutan sejalan dengan urutan pemicu pada Subbab I.1.

III.1.1 Beban Konfigurasi Platform dari Nol

Organisasi yang membangun platform dari nol harus merangkai puluhan komponen lintas lapis ingestasi, pemrosesan, penyimpanan, pelatihan, dan penyajian mo-

del. Amou Najafabadi dkk. (2024) mengidentifikasi lebih dari empat puluh *tool* pada arsitektur MLOps yang khas, dan Burgueno-Romero, Cánovas Izquierdo, dan García-García (2025) mencatat bahwa biaya integrasi pada arsitektur *open source* dapat menyamai biaya pengembangan model itu sendiri. Setiap komponen memiliki kontrak konfigurasi, protokol akses, dan model identitas masing-masing, sehingga *glue code* antar lapis terus tumbuh seiring penambahan komponen. Perubahan versi pustaka pada satu komponen kerap memicu rework pada komponen sekitarnya, terutama ketika skema data dan kontrak API ikut berubah. Tim platform juga menanggung kurva belajar yang panjang karena harus menguasai paradigma operasional yang berbeda untuk basis data, sistem antrian, mesin pemrosesan, registri model, dan layanan penyajian.

Beban tersebut bersifat akumulatif karena waktu yang habis untuk integrasi tidak dapat dipulihkan menjadi pengembangan kapabilitas. Shankar dkk. (2022) mendokumentasikan praktik MLOps yang sering tersendat justru karena lapisan integrasi yang tidak pernah selesai disempurnakan oleh tim platform. Kondisi tersebut menjelaskan mengapa banyak inisiatif platform internal terhenti pada tahap arsitektur referensi dan tidak pernah mencapai jalur produksi yang lengkap. Tantangan akan semakin sulit ketika tim berukuran kecil harus menjaga pakta layanan untuk ingestasi, pemrosesan, pelatihan, penyajian, dan tata kelola sekaligus, karena setiap pergantian versi melahirkan rantai uji regresi yang panjang. Beban ini menjadi pemicu pertama yang melatarbelakangi rancangan referensi pada Tugas Akhir ini, yang menyatukan komponen *open source* pada satu bidang kendali Kubernetes agar tim platform tidak perlu menanggung beban integrasi dari nol untuk setiap penerapan baru.

III.1.2 Biaya Berlangganan Layanan Terkelola

Sebagai alternatif terhadap konfigurasi dari nol, organisasi dapat berlangganan layanan terkelola dari penyedia *cloud*. Li dkk. (2023) menunjukkan bahwa layanan *feature store*, pelatihan, dan penyajian terkelola mempercepat adopsi tetapi membebankan biaya operasional yang meningkat seiring volume *ingest*, jumlah *training run*, dan trafik inferensi. Biaya tersebut bersifat berulang setiap bulan dan jarang dapat ditekan dengan optimasi internal karena pengguna tidak memegang kendali penuh atas alokasi sumber daya pada lapis bawah. Ketergantungan pada satu penyedia menimbulkan keterikatan vendor yang membuat migrasi menjadi mahal, terutama ketika format *feature view*, *registry model*, dan *lineage* memakai skema kepemilikan tertutup. Beban ini bertambah ketika organisasi memakai layanan ter-

kelola dari lapis berbeda yang dipasok beberapa penyedia, sebab perbedaan model identitas, protokol akses, dan format *audit log* memaksa tim platform tetap merakit otomatisasi tata kelola lintas penyedia secara manual.

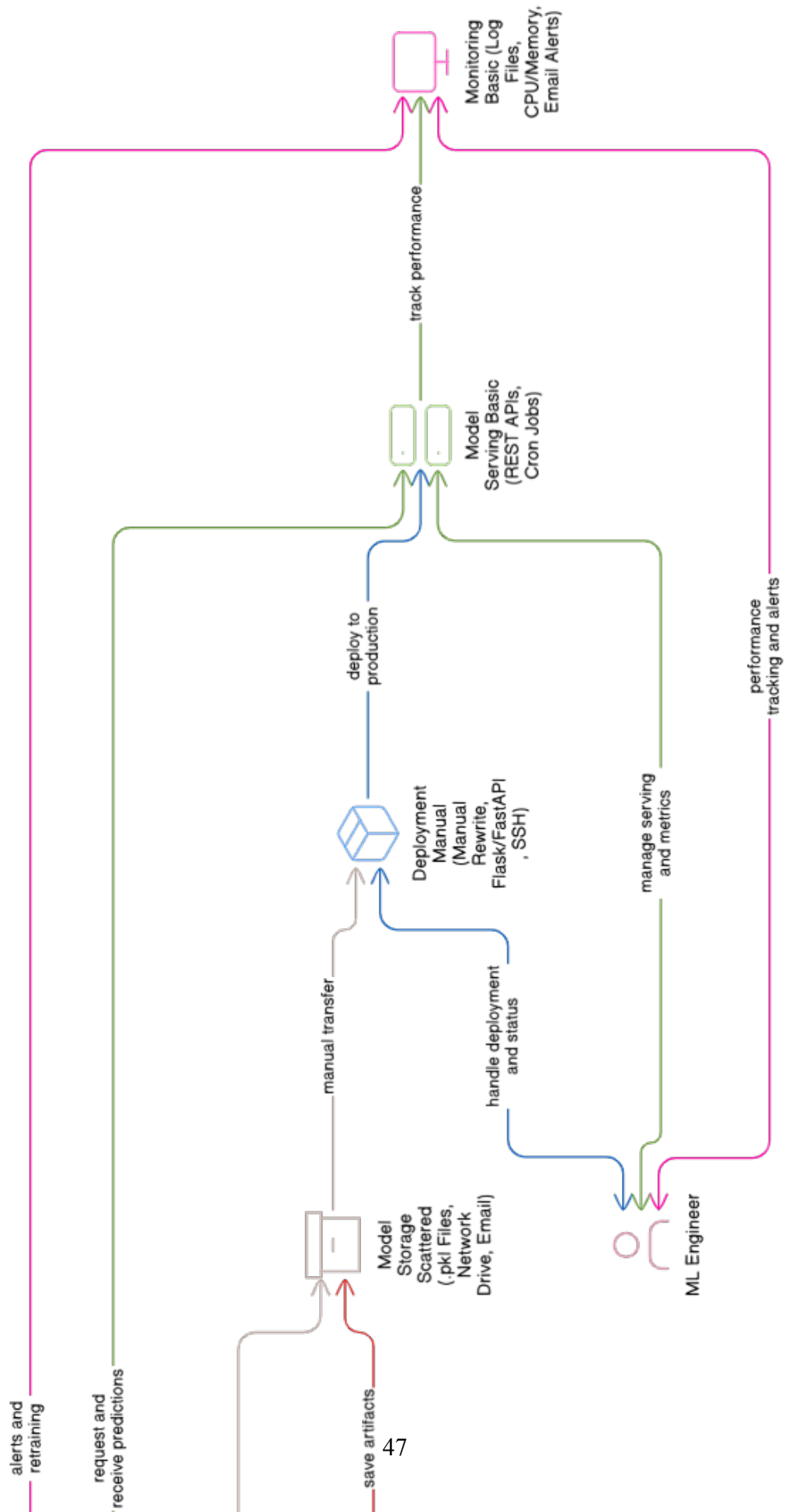
Kedua jalur tersebut menghadirkan pertukaran antara waktu pengembangan dan anggaran berjalan. Membangun platform internal dari nol menekan biaya berlangganan tetapi memerlukan waktu pengembangan yang panjang, ketergantungan pada tim yang sangat kompeten, dan risiko kegagalan integrasi yang menunda penyajian model ke produksi. Berlangganan layanan terkelola memangkas waktu pengembangan tetapi memerlukan anggaran operasional yang membesar seiring beban dan membatasi ruang kustomisasi. Pertukaran ini menjadi keputusan arsitektur yang strategis karena keduanya melibatkan pengorbanan: tim yang mengejar kecepatan menanggung biaya berjalan tinggi, sementara tim yang menekan biaya menanggung penundaan rilis dan beban pemeliharaan jangka panjang. Pengembangan platform pada Tugas Akhir ini ditujukan untuk memotong pertukaran tersebut dengan menyediakan rancangan referensi yang menyatukan komponen *open source* pada satu bidang kendali Kubernetes, sehingga organisasi dapat memilih lokasi penyebaran tanpa membayar biaya integrasi berulang dan tanpa terikat penyedia layanan tunggal.

III.1.3 Efek Domino Fragmentasi Lintas Peran

Pengembangan model *machine learning* untuk tahap produksi melibatkan empat peran utama, yaitu *business user*, *data engineer*, *data scientist*, dan *machine learning engineer*. Kreuzberger, Kühl, dan Hirschl (2023) memperlihatkan bahwa peran tersebut sering bekerja pada *tool* yang berbeda, sehingga muncul fragmentasi yang menghambat kolaborasi lintas tim. Paleyes, Urma, dan Lawrence (2022) menambahkan bahwa fragmentasi ini melahirkan integrasi yang lemah pada sambungan data dan model, dan pada akhirnya menurunkan kecepatan rilis serta kualitas keluaran. Pemicu pertama dan kedua memperdalam fragmentasi tersebut, karena tim yang sudah terbebani integrasi atau biaya berlangganan tidak punya kapasitas untuk memperbaiki sambungan lintas peran.

Gambar III.1 merangkum pandangan umum fragmentasi yang ditemui pada keempat peran tersebut. Rincian fragmentasi per peran disajikan pada Lampiran C, sementara pembahasan pada subbab ini difokuskan pada pandangan umum karena beban inti yang menjadi titik tolak penelitian bukan pada satu peran tertentu, melainkan pada sambungan antar peran yang terputus pada lapisan ingestasi, fitur, pelatihan, dan penyajian. Efek domino muncul ketika satu kontrak data antar lapis pecah dan

menyebar ke lapis di atasnya: kerusakan skema pada lapis ingestasi merambat ke lapis pemrosesan, kemudian ke fitur yang dipakai pelatihan, sehingga model di lapis penyajian menerima distribusi yang berbeda dengan pelatihan. Rantai keterhubungan tersebut menjelaskan mengapa fragmentasi pada satu titik dapat memengaruhi keseluruhan jalur produksi model.



Amou Najafabadi dkk. (2024) dan Burgueno-Romero, Cánovas Izquierdo, dan García-García (2025) memperlihatkan bahwa fragmentasi tidak hanya muncul pada antar-muka tim, tetapi juga pada bidang kendali infrastruktur, kebijakan akses, dan observabilitas. Kondisi inilah yang menjadi titik tolak penelitian, karena empat masalah inti yang dirumuskan pada Bab I pada hakikatnya merupakan konsekuensi dari rangkaian sambungan yang terputus, yaitu integrasi tumpukan, ketidakkonsistenan tata kelola, kombinasi *drift* dan *temporal leakage*, serta keterbatasan dalam menyajikan fitur multimoda dari satu sumber kebenaran. Konsekuensi tersebut tidak dapat ditangani satu per satu tanpa memutus efek domino pada akar penyebabnya, sehingga rancangan yang dikembangkan pada Bab IV memilih pendekatan terpadu pada satu bidang kendali Kubernetes. Pemilihan tersebut juga selaras dengan temuan Lakka (2025) bahwa konsolidasi DataOps dan MLOps di atas Kubernetes memperpendek rantai integrasi sehingga efek domino dapat dipatahkan pada titik sambungan yang paling banyak dikunjungi.

III.2 Analisis Kebutuhan

Setelah pemetaan kondisi saat ini, kebutuhan platform dirumuskan dalam dua kategori: kebutuhan fungsional yang menggambarkan fitur yang harus dimiliki platform, dan kebutuhan nonfungsional yang menggambarkan kualitas atributif yang harus dipenuhi. Identifikasi kebutuhan menggunakan empat rumusan masalah dari Bab I sebagai titik berangkat, dengan mempertimbangkan beban pada Subbab III.1.1 dan Subbab III.1.2 agar rancangan tetap layak operasional dan finansial. Setiap kebutuhan fungsional dipetakan langsung pada satu sub-bidang arsitektur agar tidak terjadi tumpang tindih cakupan antar kebutuhan, sementara setiap kebutuhan nonfungsional disertai target metrik yang dapat diuji pada Bab VI. Identifikasi awal masalah pengguna pada Subbab III.2.1 memberikan rangkuman kebutuhan kontekstual sebelum diturunkan menjadi daftar formal pada Subbab III.2.2 dan Subbab III.2.3.

III.2.1 Identifikasi Masalah Pengguna

Identifikasi masalah pengguna dijabarkan dengan menelusuri empat rumusan masalah pada Bab I. Pertama, pengguna pada lintas peran membutuhkan satu bidang kendali untuk mengatur ingestasi, pelatihan, penyajian, dan pemantauan, sehingga sambungan antar lapis tidak lagi dirakit secara manual dan integrasi *tool* dapat dipulihkan dari sumber kebenaran tunggal. Kedua, pengguna pada peran *data engineer* dan *data steward* membutuhkan tata kelola yang seragam di antara lapisan data, fitur, dan model, mencakup *lineage*, katalog, dan validasi kualitas. Ketiga, penggu-

na pada peran *data scientist* dan *machine learning engineer* membutuhkan deteksi *drift* yang terotomasi sekaligus jaminan *point-in-time correctness* agar evaluasi model tidak mengandung kebocoran temporal sebagaimana ditegaskan oleh Vela dkk. (2022) dan Wang dan Ruf (2021). Keempat, pengguna pada peran *data scientist* membutuhkan dukungan fitur multimoda yang terdiri dari fitur tabular, urutan waktu, dan *embedding* berdimensi tinggi dengan latensi rendah pada saat *inference*, sebagaimana diperlihatkan oleh Devlin dkk. (2019), OpenAI (December 2022), dan Malkov dan Yashunin (2020).

Empat kebutuhan pengguna tersebut menyiratkan satu pola yang sama, yaitu kebutuhan untuk memutus sambungan ad hoc antar lapis dengan kontrak yang seragam dan dapat dipanggil dari satu antarmuka. Pengguna pada peran *business user* menjadi penerima manfaat terakhir karena hanya melihat keluaran model pada lapis penyajian, sehingga kualitas kontrak pada lapis bawah menjadi penentu apakah keputusan bisnis dapat diambil dengan tepat waktu. Identifikasi tersebut menjadi titik berangkat untuk menyusun daftar formal kebutuhan fungsional pada Subbab III.2.2 dan kebutuhan nonfungsional pada Subbab III.2.3. Daftar formal kemudian dipetakan pada empat rumusan masalah dan empat tujuan dari Bab I agar setiap kebutuhan dapat dirunut ke sumber masalah yang sama dan dapat diuji pemenuhannya pada Bab VI.

III.2.2 Kebutuhan Fungsional

Tabel III.1 menyajikan empat belas kebutuhan fungsional. Manajemen fitur terpusat, penyajian fitur ganda *online-offline*, jaminan validitas temporal, dan dukungan *vector embedding* merangkum kebutuhan pada lapisan fitur. Otomatisasi *pipeline* pelatihan, otomatisasi penerapan model, pelacakan eksperimen, dan registri model merangkum kebutuhan pada siklus hidup model. Pemantauan *drift* otomatis, *governance* dan penemuan, pemrosesan *batch* dan *stream*, konfigurasi deklaratif, API dan SDK terpadu, serta manajemen sumber daya merangkum kebutuhan pada lapisan operasional. Daftar ini dipetakan langsung pada empat rumusan masalah dan empat tujuan dari Bab I.

Tabel III.1 Kebutuhan Fungsional Sistem

ID	Kebutuhan Fungsional	Deskripsi	Prioritas
KF-01	Manajemen Fitur Terpusat	Sistem menyediakan <i>feature registry</i> terpusat berbasis Feast dengan <i>file-based registry</i> pada MinIO untuk mendefinisikan, mengelola, dan melakukan <i>versioning</i> fitur. <i>Registry</i> menyimpan metadata seperti deskripsi, tipe data, <i>owner</i> , dan dependensi, serta dikelola di dalam repositori Gitea agar mendukung <i>workflow</i> GitOps dan penelusuran perubahan.	Tinggi
KF-02	Penyajian Fitur Ganda	Sistem mampu melakukan <i>serving</i> fitur dalam dua mode, yaitu <i>online serving</i> berbasis Valkey melalui protokol RESP untuk <i>real-time inference</i> dengan target latensi p99 sub-detik, dan <i>offline serving</i> berbasis ClickHouse untuk <i>historical retrieval</i> berkapasitas besar yang mendukung <i>point-in-time correct join</i> pada data historis.	Tinggi
KF-03	Jaminan Validitas Temporal	Sistem secara bawaan menjamin <i>point-in-time correctness</i> ketika menyusun <i>training dataset</i> untuk data <i>time-series</i> , sehingga mencegah kebocoran data temporal yang dapat membuat evaluasi dan model menjadi tidak valid.	Tinggi
KF-04	Dukungan <i>Vector Embeddings</i>	Sistem mendukung penyimpanan, pengindeksan, dan <i>serving vector embeddings</i> pada indeks vektor terpisah berbasis Qdrant dengan <i>similarity search</i> HNSW melalui antarmuka gRPC dan filter <i>payload</i> , dengan target latensi p99 sub-detik untuk volume permintaan menengah.	Tinggi
KF-05	Otomatisasi <i>Pipeline</i> Pelatihan	Sistem menyediakan <i>pipeline</i> otomatis untuk mengorkestrasi proses pelatihan model dari ujung ke ujung melalui Kubeflow Pipelines, terintegrasi dengan Feast sebagai <i>feature store</i> , MLflow untuk <i>experiment tracking</i> dan <i>model registry</i> , serta Katib untuk <i>hyperparameter tuning</i> , sehingga mendukung otomasi setara MLOps Level 2.	Tinggi
KF-06	Otomatisasi Penerapan Model	Sistem mengotomatisasi <i>deployment</i> model yang telah lulus validasi ke <i>inference endpoint</i> produksi melalui KServe <i>InferenceService</i> di atas Knative Serving, serta mendukung strategi <i>rolling update</i> , <i>canary deployment</i> , dan <i>progressive delivery</i> berbasis Argo Rollouts dengan mekanisme <i>rollback</i> otomatis ketika terjadi degradasi.	Tinggi
KF-07	Pemantauan <i>Drift</i> Otomatis	Sistem secara otomatis dan berkala memantau <i>data drift</i> dan <i>concept drift</i> dengan membandingkan distribusi fitur di produksi terhadap distribusi pada saat pelatihan menggunakan uji statistik (misalnya PSI dan Kolmogorov-Smirnov) dengan ambang adaptif, serta memicu <i>retraining</i> otomatis melalui Argo CronWorkflow ketika ambang terlampaui.	Tinggi

ID	Kebutuhan Fungsional	Deskripsi	Prioritas
KF-08	<i>Governance</i> & Penemuan Otomatis	Sistem menangkap dan menyajikan metadata serta <i>lineage</i> untuk fitur, <i>dataset</i> , model, dan <i>pipeline</i> pada katalog DataHub melalui <i>ingestion</i> otomatis dan peristiwa <i>lineage</i> berbasis OpenLineage, sehingga memudahkan <i>discovery</i> , analisis dampak, dan pemenuhan kebutuhan <i>compliance</i> .	Tinggi
KF-09	Pelacakan Eksperimen Terintegrasi	Sistem menyediakan <i>experiment tracking</i> yang terintegrasi untuk mencatat parameter, metrik, artefak, dan versi kode, sehingga proses eksperimen dapat direproduksi dan dibandingkan secara sistematis.	Tinggi
KF-10	<i>Versioning</i> dan Registri Model	Sistem menyediakan <i>model registry</i> terpusat untuk mengelola versi model dan status siklus hidupnya (<i>staging</i> , <i>production</i> , <i>archived</i>) disertai <i>audit trail</i> lengkap untuk mendukung <i>governance</i> .	Tinggi
KF-11	Pemrosesan <i>Batch</i> dan <i>Stream</i>	Sistem mendukung <i>batch processing</i> untuk analisis dan rekayasa fitur historis serta <i>stream processing</i> untuk komputasi fitur <i>real-time</i> , di mana keduanya terhubung dengan <i>feature store</i> sehingga definisi fitur tetap konsisten.	Tinggi
KF-12	Konfigurasi Deklaratif	Sistem memungkinkan konfigurasi komponen seperti fitur, <i>pipeline</i> , dan <i>deployment</i> dalam format deklaratif (manifes Kubernetes, <i>overlay</i> Kustomize, dan definisi Feast berbasis Python) yang dikontrol versinya di Gitea dan disinkronkan oleh Argo CD sebagai <i>single source of truth</i> .	Tinggi
KF-13	API dan SDK Terpadu	Sistem menyediakan kumpulan SDK Python yang seragam (Feast, MLflow, Kubeflow Pipelines, dbt, dan klien KServe v2 <i>inference</i>) sebagai antarmuka standar untuk berinteraksi dengan seluruh komponen, sehingga kompleksitas integrasi menurun dan pola pemakaian menjadi konsisten.	Sedang
KF-14	Manajemen Sumber Daya	Sistem menyediakan kemampuan pengelolaan sumber daya komputasi yang efisien melalui Kueue untuk <i>job queuing</i> , prioritas, dan kuota berbasis ClusterQueue, dipadukan dengan HPA, VPA, dan KEDA untuk <i>dynamic provisioning</i> pada lingkungan <i>multi-tenant</i> agar tidak ada satu <i>workload</i> yang menghabiskan sumber daya klaster.	Sedang

Penomoran KF-01 sampai KF-14 dipakai konsisten pada Bab IV ketika setiap kebutuhan dipetakan ke komponen, pada Bab V ketika setiap kebutuhan dipetakan ke berkas manifes, dan pada Bab VI ketika setiap kebutuhan dipetakan ke skenario uji. Pemetaan satu lawan satu antara kebutuhan fungsional dan sub-sistem dipilih untuk

menghindari tumpang tindih cakupan, sehingga jika satu kebutuhan tidak terpenuhi maka komponen yang bertanggung jawab dapat segera ditemukan tanpa perlu menelusuri rantai panjang. Kebutuhan pada lapisan fitur secara khusus merangkum kontrak *point-in-time correctness* dan dukungan multimoda yang menjadi perbedaan utama dari pendekatan tradisional yang hanya menyajikan fitur tabular dari satu sumber. Tabel kebutuhan fungsional ini juga dijadikan acuan ketika Bab VI menilai apakah platform yang dibangun benar-benar memenuhi cakupan yang dijanjikan, dengan setiap KF ditarik kembali ke tujuan yang melahirkannya.

III.2.3 Kebutuhan Nonfungsional

Tabel III.2 menyajikan dua belas kebutuhan nonfungsional yang berperan sebagai pembatas kualitas. Kinerja latensi rendah dan *throughput* tinggi menyangkut performa penyajian fitur dan *stream processing*. Skalabilitas horizontal, keandalan, dan toleransi kesalahan menyangkut kemampuan platform mengikuti pertumbuhan beban. Konsistensi dan kebenaran menjamin tidak ada kebocoran temporal. *Observability* mengikat metrik, log, dan *trace* pada satu antarmuka. Keamanan mengikat autentikasi, otorisasi, enkripsi, dan *audit logging*. Ekstensibilitas dan modularitas memberi ruang bagi penggantian komponen tanpa mengganggu komponen lain. Kemudahan penggunaan, efisiensi sumber daya, portabilitas, dan kemudahan pemeliharaan menutup daftar dengan menyoroti aspek operasional jangka panjang. Target metrik untuk lapisan inferensi mengikuti panduan Bondi (2000) terkait karakteristik skalabilitas dan pengaruhnya pada kinerja.

Tabel III.2 Kebutuhan Nonfungsional Sistem

ID	Kebutuhan Nonfungsional	Deskripsi	Target Metrik
KNF-01	Kinerja Latensi Rendah	<i>Online feature serving</i> berbasis Valkey dan <i>vector similarity search</i> berbasis Qdrant harus memberikan latensi yang stabil dan rendah untuk mendukung inferensi waktu nyata pada berbagai domain aplikasi, termasuk pada saat beban tinggi.	Latensi p99 sub-detik pada <i>feature serving</i> dan <i>vector search</i> , dengan ambang spesifik per layanan diukur dan dipantau melalui SLO Sloth

ID	Kebutuhan Nonfungsional	Deskripsi	Target Metrik
KNF-02	<i>Throughput</i> Tinggi	Sistem harus mampu menangani <i>workload</i> dengan <i>throughput</i> tinggi yang lazim pada beban kerja <i>streaming</i> dan <i>serving</i> bervolume tinggi, dengan karakteristik skalabilitas mendekati linear ketika sumber daya ditambah.	<i>Throughput ingest</i> dan <i>serving</i> tumbuh proporsional terhadap jumlah <i>partition</i> Kafka serta <i>replica</i> Feast melalui <i>auto-scaling</i> KEDA dan HPA
KNF-03	Skalabilitas Horizontal	Seluruh komponen sistem harus mendukung skala horizontal, sehingga kapasitas dapat ditingkatkan dengan menambah <i>node</i> dan menghasilkan peningkatan kinerja yang linear atau mendekati linear.	Seluruh <i>workload</i> terstandarisasi pada HPA (CPU/memori), VPA <i>recommender</i> , dan KEDA (Kafka <i>lag</i> , <i>custom metrics</i>)
KNF-04	Keandalan & Toleransi Kesalahan	Sistem harus andal dengan komponen <i>stateful</i> (Kafka, ClickHouse, PostgreSQL, MinIO, Valkey, Qdrant) yang dapat berjalan dalam mode <i>multi-replica</i> pada lingkungan produksi multi-node, dan mampu melakukan pemulihan otomatis dari kegagalan sementara melalui mekanisme <i>operator</i> dan <i>snapshot</i> berkala. Pada lingkungan verifikasi node tunggal, <i>replica</i> idle ditetapkan menjadi satu dan ditingkatkan secara dinamis ketika beban nyata terdeteksi.	SLO <i>platform</i> dijaga melalui Sloth (Kafka 99,9%, Postgres 99,95%, MinIO p99 <500ms pada 99%), sedangkan <i>disaster recovery</i> berbasis Velero <i>snapshot</i> harian penuh dan <i>incremental</i> per <i>use case</i>
KNF-05	Konsistensi & Kebenaran	Sistem harus menjaga konsistensi data dan menjamin validitas temporal sehingga tidak terjadi kebocoran data yang merusak kebenaran hasil analisis maupun kinerja model.	<i>Zero tolerance</i> untuk kebocoran data temporal yang terdeteksi
KNF-06	<i>Observability</i>	Sistem harus memiliki <i>observability</i> menyeluruh, mencakup metrik (Prometheus dan Grafana), <i>structured log</i> (Loki), <i>distributed tracing</i> (Grafana Tempo melalui OpenTelemetry), <i>continuous profiling</i> (Pyroscope), dan <i>alerting</i> (Alertmanager dan Sloth) sehingga perilaku sistem dapat dianalisis dari ujung ke ujung.	Seluruh komponen terinstrumentasi metrik dan log standar, <i>trace</i> OpenTelemetry untuk lintasan kritis, serta SLO Sloth untuk indikator layanan

ID	Kebutuhan Nonfungsional	Deskripsi	Target Metrik
KNF-07	Keamanan	Sistem harus menerapkan kontrol keamanan menyeluruh, mencakup autentikasi tunggal melalui dex sebagai <i>identity provider</i> OIDC dan oauth2-proxy, otorisasi berbasis RBAC dan SpiceDB, enkripsi data <i>in transit</i> dengan Istio mTLS <i>strict</i> , enkripsi <i>at rest</i> pada MinIO melalui KES sebagai <i>KMS gateway</i> ke OpenBao, kebijakan admisi Kyverno, serta deteksi ancaman <i>runtime</i> oleh Falco dan pemindaian kerentanan citra oleh Trivy Operator.	TLS 1.3 <i>in transit</i> , AES-256 <i>at rest</i> , dan <i>audit log</i> OpenBao serta Kubernetes <i>audit policy</i> aktif pada seluruh <i>control plane</i>
KNF-08	Ekstensibilitas & Modularitas	Arsitektur harus modular sehingga komponen dapat diperluas atau diganti melalui antarmuka terdefinisi (<i>Custom Resource Definition</i> , helm <i>chart</i> , dan <i>overlay</i> Kustomize) tanpa mengganggu komponen lain.	Setiap komponen disusun pada <i>folder</i> terpisah dengan <i>kustomization.yaml</i> sendiri, sehingga dapat dipasang, diperbarui, atau diganti secara independen
KNF-09	Kemudahan Penggunaan	Antarmuka konsol pengembang dan operator platform harus dapat diakses dari satu titik autentikasi tunggal sehingga konsol Argo CD, Argo Workflows, Kubeflow, MLflow, DataHub, dan Grafana terhubung melalui satu sesi identitas, dengan dokumentasi referensi yang ringkas untuk konfigurasi setiap komponen.	Seluruh antarmuka konsol platform terhubung pada satu jalur autentikasi dex dan oauth2-proxy tanpa konfigurasi identitas tambahan per layanan, serta tiap komponen memiliki dokumentasi referensi pada repositori Gitea
KNF-10	Efisiensi Sumber Daya	Sistem harus memanfaatkan sumber daya komputasi dan penyimpanan secara efisien untuk mengendalikan biaya tanpa mengorbankan kinerja, baik pada lingkungan node tunggal maupun multi-node.	Kompresi kolumnar pada ClickHouse, <i>autoscaling</i> berbasis HPA, VPA, dan KEDA agar utilisasi sumber daya proporsional terhadap beban, serta atribusi biaya per <i>workload</i> melalui OpenCost

ID	Kebutuhan Nonfungsional	Deskripsi	Target Metrik
KNF-11	Portabilitas	Sistem harus dapat dijalankan di berbagai lingkungan <i>deployment</i> dengan perubahan terbatas pada konfigurasi dan tanpa modifikasi kode besar, sebab seluruh komponen berdiri di atas distribusi Kubernetes standar dan citra <i>open source</i> .	Dapat di- <i>deploy</i> pada distribusi Kubernetes <i>managed</i> (AWS EKS, GCP GKE, Azure AKS) maupun <i>on-premise</i> (k3s, kubeadm) hanya dengan penyesuaian <i>overlay</i> Kustomize dan parameter <i>secret</i>
KNF-12	Kemudahan Pemeliharaan	Sistem harus mudah dipelihara, dengan struktur kode dan manifes yang jelas, <i>pipeline</i> CI yang otomatis pada Tekton, serta <i>deployment</i> deklaratif dari sumber kebenaran tunggal pada Gitea melalui Argo CD.	Setiap komponen dapat direkonsiliasi ulang dari <i>repository</i> Gitea tanpa intervensi manual, dan perubahan ditolak otomatis oleh <i>pull request review</i> dan <i>policy</i> Kyverno bila tidak sesuai konvensi

Setiap KNF disertai target metrik yang dapat diuji secara langsung pada Bab VI, sehingga klaim kualitas tidak berhenti pada deskripsi tetapi memiliki ambang batas yang konkret. Pemilihan ambang batas pada KNF latensi dan *throughput* mengikuti panduan industri pada *online feature serving* yang menempatkan latensi P95 di kisaran milisekon untuk menjaga jaminan layanan inferensi yang responsif. Pemilihan ambang batas pada KNF observabilitas mengikuti praktik yang dirangkum oleh Cloud Native Computing Foundation (2024b) bahwa metrik, log, dan *trace* harus dapat dirujuk silang pada satu antarmuka agar diagnosis dapat ditegakkan tanpa berpindah *tool*. Daftar nonfungsional ini bersifat terbuka untuk diperluas pada penelitian lanjutan, terutama pada aspek keamanan rantai pasok perangkat lunak dan kepatuhan terhadap regulasi sektoral, namun cakupan dua belas item ini telah memadai untuk menilai keberhasilan rancangan terhadap empat rumusan masalah pada Bab I.

III.3 Analisis Pemilihan Solusi

Bagian ini menyusun alternatif solusi yang dapat memenuhi kebutuhan fungsional dan nonfungsional, kemudian memilih solusi terbaik dengan menggunakan skor pe-

menuhan kebutuhan. Skor 0 berarti tidak terpenuhi, skor 1 berarti terpenuhi sebagian, dan skor 2 berarti terpenuhi penuh. Penilaian skor dilakukan terhadap empat alternatif dengan menggunakan dua belas KNF dan empat belas KF sebagai kriteria, sehingga total skor maksimum sebuah alternatif adalah lima puluh dua. Alternatif yang memperoleh skor tertinggi akan dirancang lebih lanjut pada Bab IV dan diimplementasikan pada Bab V, dengan justifikasi tertulis pada setiap kebutuhan yang tidak memperoleh skor penuh.

III.3.1 Alternatif Solusi

Empat alternatif solusi dipertimbangkan untuk membangun platform DataOps dan MLOps terintegrasi. Pemilihan empat alternatif didasarkan pada pemetaan pendekatan yang lazim dipakai oleh organisasi yang menerapkan MLOps pada skala produksi, mencakup pendekatan terkelola penuh, pendekatan *open source* di atas Kubernetes, pendekatan kustom dari awal, dan pendekatan hibrida antara terkelola dan *open source*. Setiap alternatif diuraikan dengan kelebihan, kekurangan, dan rujukan pustaka yang relevan agar pertimbangan dapat ditelusuri kembali. Penyajian alternatif menggunakan format sub-paragraf bercetak tebal mengikuti gaya proposal dan menjaga keterbacaan ketika rincian setiap pendekatan cukup banyak.

Alternatif A: Layanan Terkelola dari Penyedia *Cloud*. Pendekatan ini menggunakan rangkaian layanan terkelola pada satu penyedia, mencakup layanan ingestasi, *warehouse*, *feature store*, pelatihan, dan penyajian. Kelebihannya berupa kecepatan adopsi dan integrasi bawaan antar layanan, sebagaimana dibahas oleh Li dkk. (2023). Kekurangannya berupa keterikatan terhadap satu penyedia, biaya operasional pada beban tinggi, dan keterbatasan dalam menyesuaikan komponen pada level rendah, sehingga pertukaran waktu lawan anggaran pada Subbab III.1.2 tetap menjadi beban berjalan.

Alternatif B: Tumpukan *Open Source* pada Kubernetes. Pendekatan ini merangkai komponen *open source* di atas Kubernetes sebagai bidang orkestrasi. Lakka (2025) dan Burgueno-Romero, Cánovas Izquierdo, dan García-García (2025) memperlihatkan rancangan serupa pada lingkungan *multi-cloud* dan *edge*. Kelebihannya berupa fleksibilitas, transparansi versi, portabilitas, dan kemampuan menegakkan pola GitOps. Kekurangannya berupa kebutuhan tata kelola yang lebih ketat dan kurva belajar yang lebih panjang dibandingkan layanan terkelola, sehingga beban konfigurasi dari nol pada Subbab III.1.1 perlu dikurangi melalui rancangan referensi yang utuh.

Alternatif C: Pengembangan Kustom dari Awal. Pendekatan ini membangun setiap lapisan secara mandiri tanpa banyak menggunakan komponen yang sudah jadi. Kelebihannya berupa kontrol penuh atas perilaku setiap komponen. Kekurangannya berupa biaya pengembangan dan pemeliharaan yang sangat besar, ketergantungan pada tim yang sangat kompeten, dan risiko menciptakan ulang masalah yang sudah diselesaikan oleh pustaka *open source* yang teruji.

Alternatif D: Hibrida antara Layanan Terkelola dan *Open Source*. Pendekatan ini menggunakan beberapa layanan terkelola, terutama pada lapisan dasar seperti penyimpanan dan jaringan, dipadukan dengan komponen *open source* pada lapisan DataOps dan MLOps. Kelebihannya berupa keseimbangan antara kecepatan adopsi dan fleksibilitas. Kekurangannya berupa kompleksitas integrasi antara dua jenis komponen dan kebutuhan kontrak antar lapisan yang harus dirancang dengan saksama.

III.3.2 Analisis Penentuan Solusi

Tabel III.3 menyajikan evaluasi pemenuhan kebutuhan fungsional dan Tabel III.4 menyajikan evaluasi pemenuhan kebutuhan nonfungsional. Skor total ditampilkan pada baris terakhir Tabel III.4. Alternatif B memperoleh skor total tertinggi, yaitu 49 dari 52, diikuti oleh Alternatif A dan Alternatif D dengan skor 43, dan Alternatif C dengan skor 30. Selisih skor antara Alternatif B dan dua pesaing terdekatnya sebesar enam poin terutama berasal dari KF jaminan validitas temporal, KF dukungan *vector embeddings*, KNF portabilitas, dan KNF ekstensibilitas yang tidak dipenuhi penuh oleh pendekatan terkelola maupun hibrida.

Tabel III.3 Evaluasi Pemenuhan Kebutuhan Fungsional

Kebutuhan	Cloud	Open-Source	Kustom	Hibrida
KF-01: Manajemen Fitur Terpusat	2	2	1	2
KF-02: Penyajian Fitur Ganda	2	2	1	2
KF-03: Jaminan Validitas Temporal	1	2	2	1
KF-04: Dukungan <i>Vector Embeddings</i>	1	2	2	1
KF-05: Otomatisasi <i>Pipeline</i> Pelatihan	2	2	1	2
KF-06: Otomatisasi Penerapan Model	2	2	1	2
KF-07: Pemantauan <i>Drift</i> Otomatis	1	2	1	1
KF-08: <i>Governance</i> & Penemuan	2	2	1	2
KF-09: Pelacakan Eksperimen	2	2	1	2
KF-10: <i>Versioning</i> Registri Model	2	2	1	2
KF-11: Pemrosesan <i>Batch/Stream</i>	2	2	1	2
KF-12: Konfigurasi Deklaratif	2	2	1	2
KF-13: API dan SDK Terpadu	2	1	1	1
KF-14: Manajemen Sumber Daya	2	2	1	2
Total Skor KF	25	27	16	24

Tabel III.4 Evaluasi Pemenuhan Kebutuhan Nonfungsional

Kebutuhan	Cloud	Open-Source	Kustom	Hibrida
KNF-01: Kinerja Latensi Rendah	2	2	2	2
KNF-02: <i>Throughput</i> Tinggi	2	2	2	2
KNF-03: Skalabilitas Horizontal	2	2	1	2
KNF-04: Keandalan & Toleransi Kesalahan	2	2	1	2
KNF-05: Konsistensi & Kebenaran	1	2	2	1
KNF-06: <i>Observability</i>	2	2	1	2
KNF-07: Keamanan	2	2	1	2
KNF-08: Ekstensibilitas & Modularitas	0	2	2	1
KNF-09: Kemudahan Penggunaan	2	1	0	2
KNF-10: Efisiensi Sumber Daya	1	2	1	1
KNF-11: Portabilitas	0	2	1	1
KNF-12: Kemudahan Pemeliharaan	2	1	0	1
Total Skor KNF	18	22	14	19
TOTAL SKOR (KF+KNF)	43	49	30	43

Alternatif B unggul terutama pada kebutuhan yang menyangkut jaminan validitas temporal, dukungan *vector embeddings*, pemantauan *drift* otomatis, ekstensibilitas, dan portabilitas. Alternatif A unggul pada kemudahan penggunaan dan kemudahan pemeliharaan, tetapi tertinggal pada konsistensi temporal, dukungan *vector embeddings* pada *feature store*, dan portabilitas. Alternatif C unggul pada

kontrol penuh tetapi kalah pada hampir semua kebutuhan operasional. Alternatif D mendekati Alternatif A tetapi tertinggal pada portabilitas dan ekstensibilitas. Berdasarkan skor pemenuhan kebutuhan dan analisis komparatif yang sejalan dengan Burgueno-Romero, Cánovas Izquierdo, dan García-García (2025), Amou Najafabadi dkk. (2024), dan Lakka (2025), Alternatif B dipilih sebagai solusi yang akan dirancang pada Bab IV dan diimplementasikan pada Bab V.

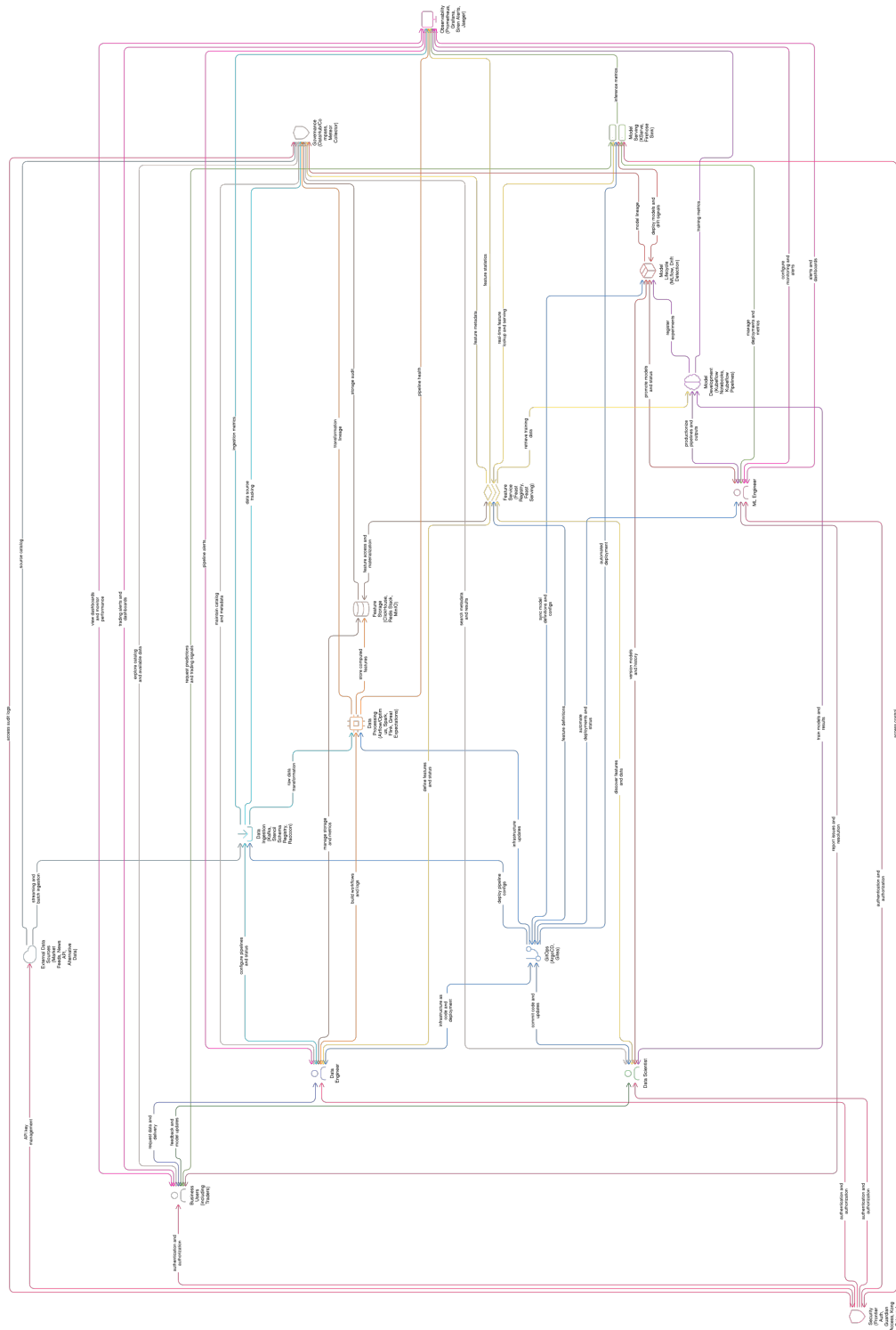
BAB IV

PERANCANGAN ARSITEKTUR PLATFORM DATAOPS DAN MLOPS

Bab ini menyajikan perancangan arsitektur platform yang mengintegrasikan DataOps dan MLOps pada Kubernetes. Pembahasan dimulai dari gambaran umum yang merangkum perbedaan kondisi sebelum dan sesudah integrasi, kemudian dilanjutkan dengan perancangan empat sub-sistem yang masing-masing memenuhi satu tujuan dari Bab I: arsitektur platform terintegrasi sebagai pemenuhan T-1, sub-sistem tata kelola data sebagai pemenuhan T-2, sub-sistem deteksi *drift* dengan pelatihan ulang otomatis sebagai pemenuhan T-3, dan sub-sistem layanan fitur *dual-store* sebagai pemenuhan T-4. Bab ini menutup pembahasan dengan alur kerja *end-to-end* yang menghubungkan keempat sub-sistem dalam satu siklus reproduksibel. Pemetaan satu tujuan menjadi satu subbab utama dipilih agar rantai keterhubungan rumusan masalah, tujuan, perancangan, implementasi, dan evaluasi pada Bab VI dapat ditelusuri dengan satu indeks yang sama. Pemilihan urutan subbab juga mengikuti urutan rumusan masalah pada Bab I sehingga pembaca dapat membandingkan tiap solusi dengan masalah yang menjadi titik berangkatnya.

IV.1 Gambaran Umum Platform

Platform yang dirancang menempatkan Kubernetes sebagai bidang orkestrasi tunggal dan menyatukan komponen DataOps dan MLOps di atasnya. Gambar IV.1 memperlihatkan arsitektur terintegrasi yang menggantikan kondisi fragmentasi pada Gambar III.1 dari Bab III. Komponen yang sebelumnya berdiri sendiri pada perangkat lunak yang berbeda kini terhubung melalui satu bidang kendali, satu sumber kebenaran berbasis Git, dan satu antarmuka observabilitas. Penyatuan bidang kendali memungkinkan tim platform menerapkan perubahan dengan satu mekanisme *deploy* yang sama untuk seluruh komponen, sehingga rantai integrasi yang sebelumnya bercabang dapat dipersingkat.



Gambar IV.1 Pandangan Umum Platform DataOps dan MLOps Terintegrasi

Perbedaan utama terhadap kondisi sebelum integrasi terletak pada tiga aspek. Pertama, bidang kendali yang sebelumnya tersebar pada bermacam *tool* digabungkan ke dalam *control plane* Kubernetes yang sama. Kedua, sumber kebenaran konfigurasi

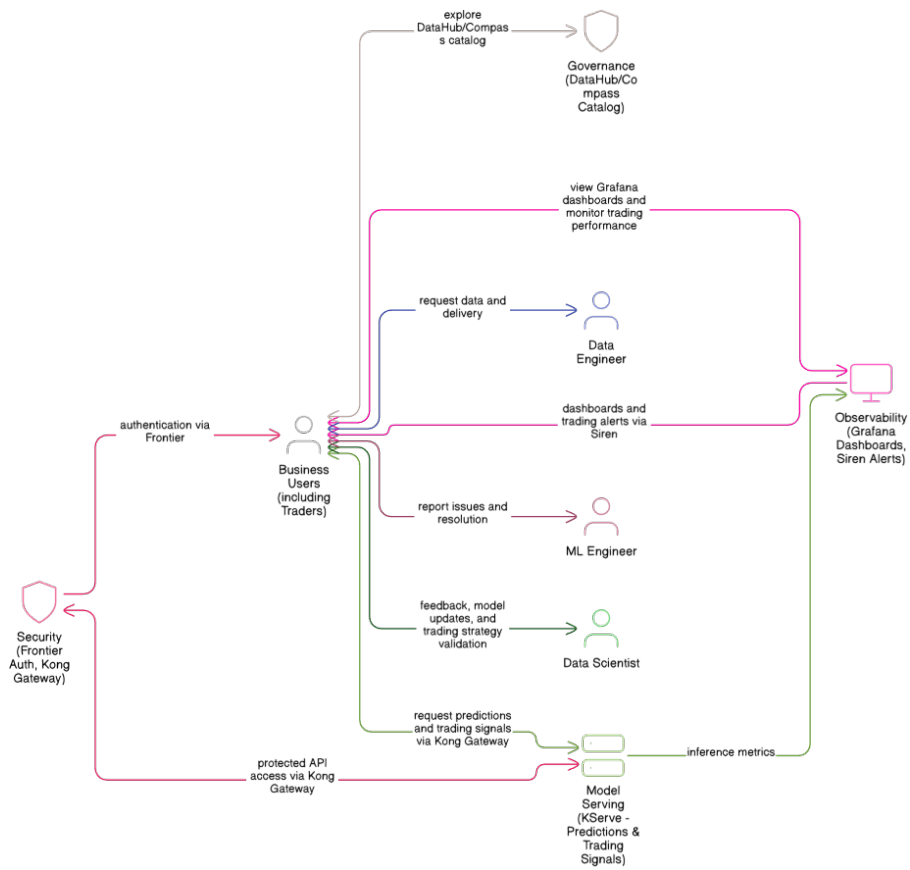
yang sebelumnya berada pada berkas konfigurasi terpisah disatukan pada repositori Git yang dijaga oleh Argo CD (Argo Project 2024). Ketiga, antarmuka observabilitas yang sebelumnya berbeda untuk metrik, log, dan *trace* disatukan di Grafana dengan Prometheus (Prometheus Authors 2024), Loki (Grafana Labs 2024b), dan Grafana Tempo (Grafana Labs 2024c) sebagai sumber tiga pilar observabilitas. Ketiga perbedaan tersebut bersama-sama memutus efek domino fragmentasi yang dibahas pada Subbab III.1.3, karena tiap lapis berbagi kontrak yang sama dan dapat dirujuk silang melalui DataHub.

IV.2 Perancangan Arsitektur Terintegrasi

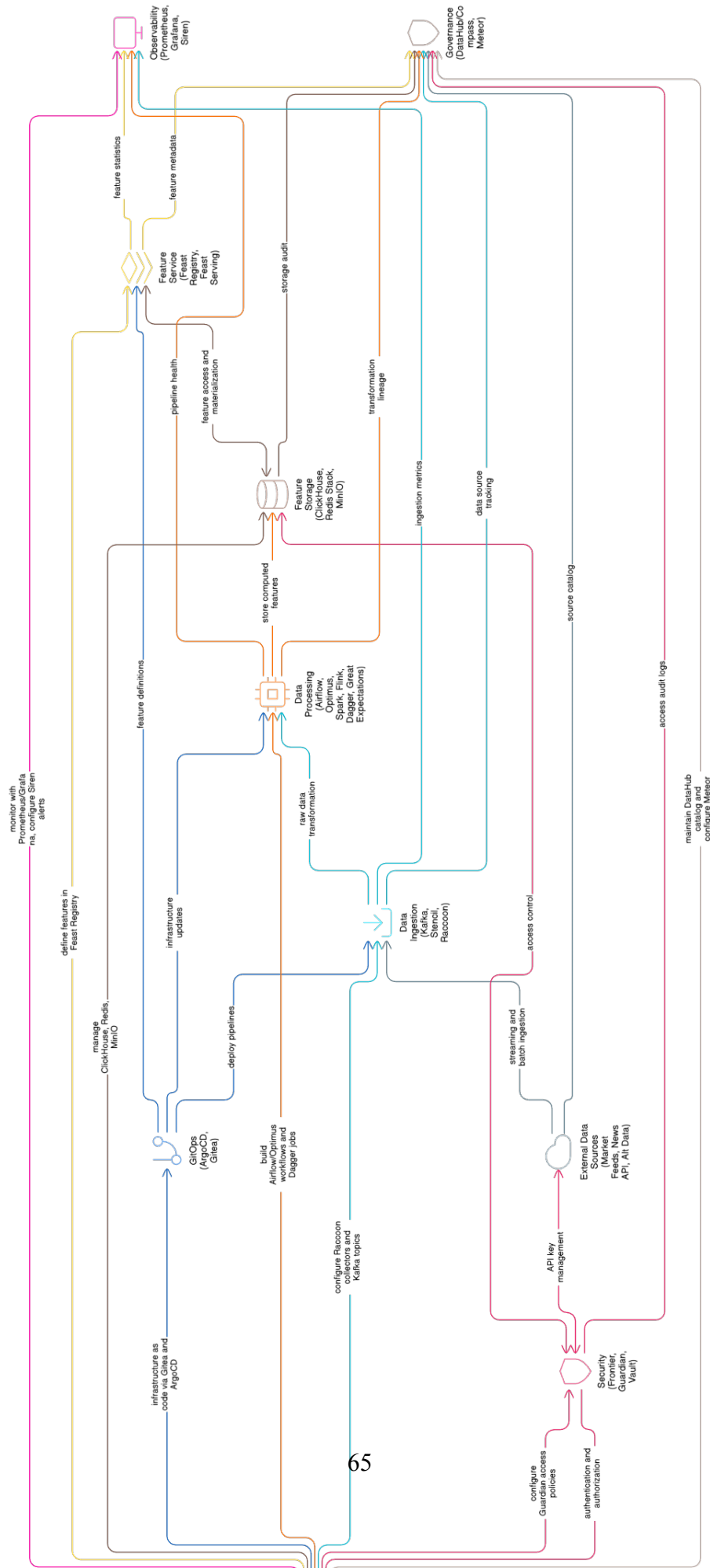
Bagian ini menjawab T-1 dengan merancang arsitektur platform DataOps dan MLOps terintegrasi pada Kubernetes. Arsitektur disusun dalam tujuh lapis yang saling terhubung melalui kontrak antarmuka eksplisit. Tujuh lapis yang dimaksud adalah lapisan infrastruktur, lapisan ingestasi data, lapisan pemrosesan, lapisan penyimpanan fitur, lapisan siklus hidup model, lapisan penyajian model, dan lapisan tata kelola serta observabilitas. Susunan tujuh lapis ini selaras dengan praktik yang dirangkum oleh Kreuzberger, Kühl, dan Hirschl (2023) pada pemetaan rangkaian *tool* MLOps, dan dipertajam dengan tambahan lapisan tata kelola yang menyatukan katalog, *lineage*, dan kontrak data sebagai elemen pertama yang dihubungkan ke lapisan lain.

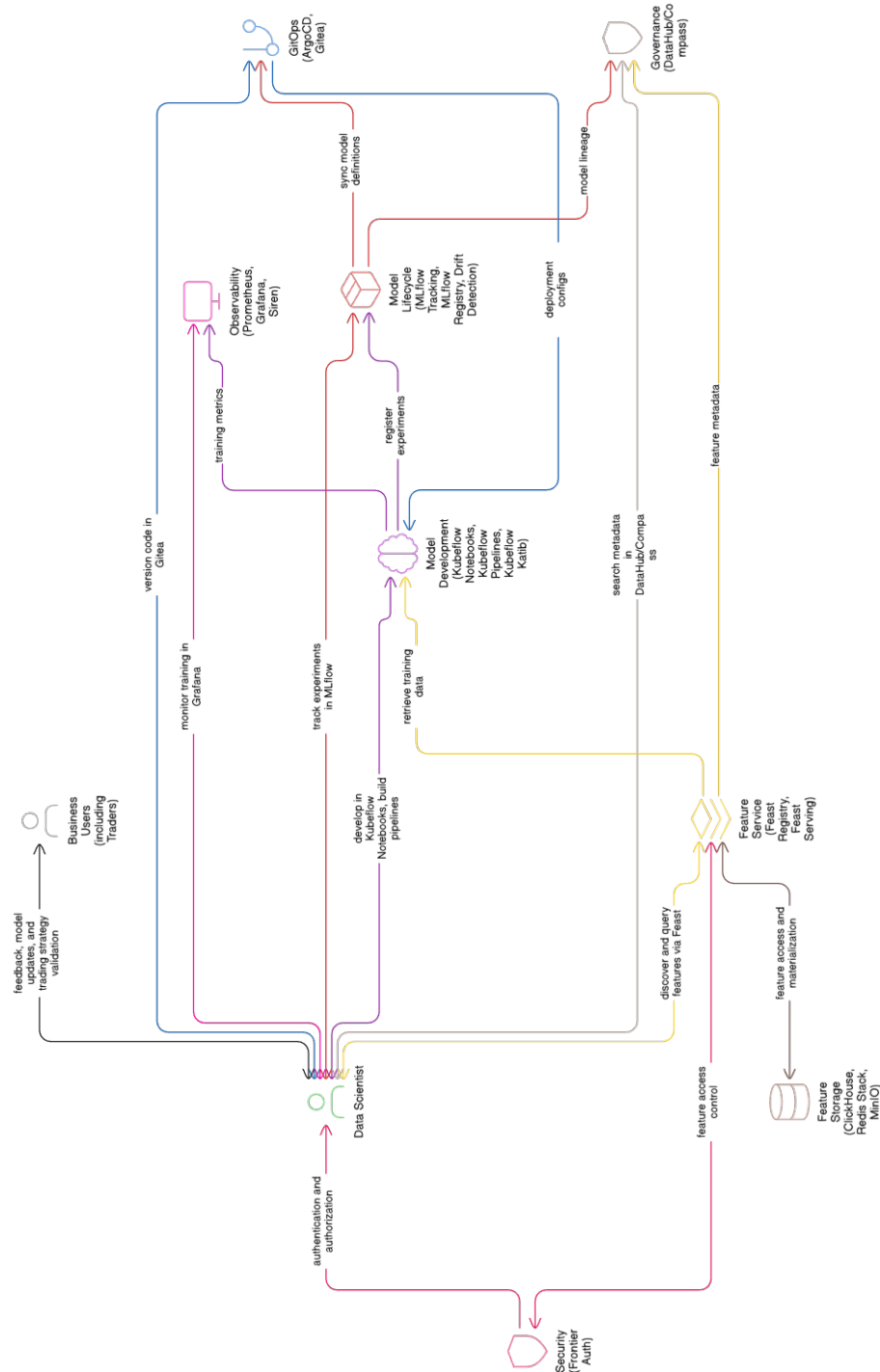
IV.2.1 Pandangan per Peran Pengguna

Empat peran utama mempunyai akses yang berbeda pada platform yang sama. Gambar IV.2 menyajikan pandangan untuk peran *business user*, Gambar IV.3 untuk peran *data engineer*, Gambar IV.4 untuk peran *data scientist*, dan Gambar IV.5 untuk peran *machine learning engineer*. Setiap peran berinteraksi dengan satu antarmuka yang sama, sehingga konteks dapat berpindah dari pelaporan ke katalog, dari katalog ke eksperimen, dan dari eksperimen ke penyebaran model tanpa berganti aplikasi. Pemilihan satu antarmuka pusat ini melindungi pengguna dari kebutuhan menghafal banyak *tool* sekaligus menjaga keterhubungan metadata yang dibagikan antar peran.

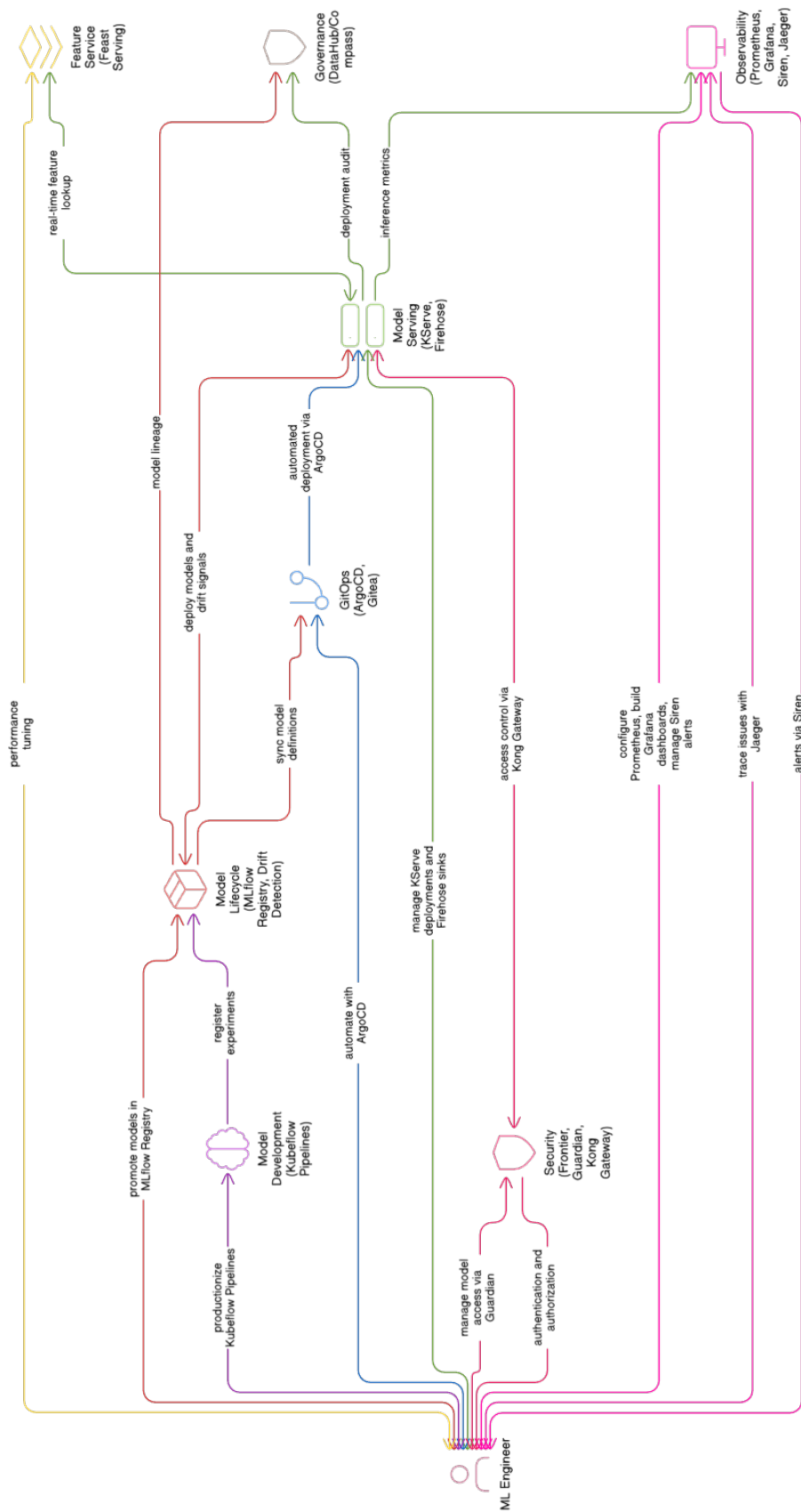


Gambar IV.2 Pandangan Platform Terintegrasi untuk *Business User*





Gambar IV.4 Pandangan Platform Terintegrasi untuk *Data Scientist*



Gambar IV.5 Pandangan Platform Terintegrasi untuk *Machine Learning Engineer*

Business user mengakses platform melalui Apache Superset (Apache Superset Authors 2025) untuk dasbor data dan Grafana untuk dasbor operasional, dengan token OIDC yang sama dari dex (Dex Authors 2025) pada kedua antarmuka. *Data engineer* bekerja dengan DataHub untuk katalog, Argo CD untuk *rollout* konfigurasi, dan Argo Workflows untuk eksekusi *pipeline* di atas *cluster* yang sama. *Data scientist* bekerja dengan Kubeflow Notebooks, MLflow, dan Feast pada *namespace* yang sama dengan tata kelola identitas yang seragam, sehingga eksperimen dan registri model selalu terdokumentasi pada lokasi yang sama. *Machine learning engineer* mengelola pelatihan ulang dan penyajian melalui Kubeflow Pipelines dan KServe, sambil memanfaatkan Argo Rollouts untuk strategi promosi versi yang berbasis metrik.

IV.2.2 Lapisan Infrastruktur dan Bidang Kendali

Lapisan infrastruktur menggunakan Kubernetes (Cloud Native Computing Foundation 2024a) sebagai bidang orkestrasi untuk seluruh komponen platform. Istio (Istio Authors 2024) menyediakan jaringan layanan dengan mTLS yang mengamankan komunikasi antar layanan, sedangkan APISIX (Apache APISIX Authors 2025) berperan sebagai *API gateway* pada perbatasan *mesh*. OpenBao (OpenBao Authors 2025) mengelola rahasia, dan *External Secrets Operator* menyebarkan rahasia dari OpenBao ke *namespace* target tanpa menyalin rahasia ke berkas *manifest*. Argo CD menjadi satu-satunya jalur perubahan, menarik konfigurasi dari Git dan menerapkan ke *cluster* melalui prinsip GitOps (Limoncelli 2022).

Kebijakan keamanan dijalankan dalam dua lapis: Kyverno (Kyverno Authors 2024) pada lapis admisi Kubernetes, dan Falco (Falco Authors 2025) pada lapis *runtime*. Trivy Operator (Aqua Security 2025) memindai citra dan beban kerja secara terus menerus, sedangkan Velero (Velero Authors 2025) melakukan pencadangan sumber daya *cluster* dan *persistent volume* secara terjadwal. Penskalaan otomatis dijalankan oleh tiga komponen yang saling melengkapi yaitu HPA Kubernetes pada level Pod, VPA *recommender* untuk rekomendasi *resource request*, dan KEDA (KEDA Authors 2025) untuk penskalaan berbasis peristiwa dari Kafka atau Prometheus. Cert-manager (cert-manager Contributors 2025) mengelola sertifikat TLS, menutup lapisan infrastruktur dengan kontrak identitas mesin yang dapat diperbarui secara otomatis.

IV.2.3 Lapisan Ingestasi dan Pemrosesan Data

Lapisan ingestasi menggunakan Apache Kafka (Apache Software Foundation 2024c) sebagai bus pesan utama dengan operator Strimzi pada *cluster* Kubernetes. *Kafka Connect* digunakan untuk integrasi dengan sumber data eksternal, dan *Karapace* digunakan sebagai registri skema. Lapisan pemrosesan dibagi menjadi dua jalur, yaitu jalur *batch* dan jalur *stream*, sehingga kebutuhan latensi berbeda dapat dilayani tanpa kompromi. Jalur *batch* menggunakan Apache Spark (Apache Software Foundation 2024d) dan dbt untuk transformasi pada *data warehouse*, sedangkan jalur *stream* menggunakan Apache Flink (Apache Software Foundation 2024b) untuk pemrosesan jendela waktu, agregasi, dan deteksi pola.

Penyimpanan jangka panjang pada *data lake* menggunakan MinIO sebagai *object store* dengan format Apache Iceberg sebagai *table format*, sedangkan Trino (Trino Software Foundation 2024) menyediakan kueri federasi pada beberapa sumber data sekaligus. Katalog Iceberg dikelola oleh Lakekeeper (Lakekeeper Authors 2025) yang memberikan *REST API* kompatibel dengan klien Iceberg standar dan dapat dipakai oleh Spark, Trino, dan Flink secara seragam. lakeFS (Treeverse Ltd. 2025) ditambahkan sebagai lapisan *version control* untuk *data lake* sehingga eksperimen pada cabang data dapat dilakukan tanpa mengganggu cabang utama. ClickHouse (ClickHouse Inc 2024) berperan sebagai *warehouse* berorientasi kolom yang menampung tabel hasil transformasi dan menyediakan kueri analitik dengan latensi rendah untuk dasbor maupun materialisasi fitur.

IV.2.4 Lapisan Siklus Hidup Model dan Penyajian

Lapisan siklus hidup model menggunakan MLflow (MLflow Contributors 2024) untuk *tracking* dan registri model. Kubeflow Pipelines (Kubeflow Contributors 2024) dipakai sebagai *orchestrator* pelatihan, sedangkan Katib digunakan untuk *hyperparameter tuning*. Lapisan penyajian menggunakan KServe (KServe Contributors 2024) sebagai *inference platform* dengan dukungan banyak *runtime*, otomatisasi penskalaan, dan integrasi langsung dengan model yang tersimpan pada registri MLflow. Argo Workflows mengeksekusi *pipeline* yang tidak terkait langsung dengan pelatihan, sedangkan Argo CronWorkflow menjadwalkan *job* berkala untuk pemeliharaan, validasi data, dan pemicu pelatihan ulang.

Kubeflow Trainer (Kubeflow Authors 2025c) menjalankan pelatihan terdistribusi melalui TrainJob yang dipanggil dari *pipeline* Kubeflow Pipelines, sehingga pelatihan *multi-replica* tidak memerlukan *custom controller*. Penyajian model multimo-

da didukung KServe dengan *runtime* bawaan untuk *scikit-learn*, XGBoost, PyTorch, dan ONNX, ditambah *runtime* kustom untuk model yang memerlukan dependensi spesifik. Knative (Knative Authors 2025) berperan sebagai bidang *serverless* bagi KServe, sehingga penskalaan dari nol dapat dilakukan tanpa menahan sumber daya saat trafik rendah. Kueue (Kueue Authors 2024) mengelola antrean pelatihan agar sumber daya GPU atau CPU dapat dibagi adil antara *tenant*, sambil tetap menghormati prioritas *PriorityClass* yang ditetapkan pada level platform.

IV.2.5 Lapisan Tata Kelola dan Observabilitas

Lapisan tata kelola menggunakan DataHub (DataHub Project 2024) sebagai katalog metadata yang mengumpulkan informasi dari komponen lain melalui mekanisme *push* dan *pull*, dengan OpenLineage (OpenLineage Authors 2025) sebagai protokol pengirim peristiwa *lineage* dari komponen *pipeline*. Great Expectations (Great Expectations 2024) menjalankan pengujian kontrak data pada lapisan *batch* dan *stream*. Lapisan observabilitas menggunakan Prometheus untuk metrik, Loki untuk log, dan Grafana Tempo (Grafana Labs 2024c) untuk *trace*, dengan OpenTelemetry sebagai *collector* bersama. Grafana menyatukan ketiganya dan ditambahkan dasbor khusus untuk kualitas data, *drift* fitur, dan kinerja model.

Sloth (Sloth Authors 2025) menurunkan *service-level objective* ke aturan Prometheus, sedangkan Pyroscope menyediakan profil performa berkelanjutan dan OpenCost menutup pemantauan biaya per beban kerja. Superset (Apache Superset Authors 2025) berperan sebagai antarmuka analitik bagi *business user* yang menampilkan dasbor di atas ClickHouse, Trino, dan MySQL. Pushgateway dipakai untuk mengumpulkan metrik *job* berdurasi pendek, antara lain CronJob validasi data dan CronWorkflow pengukur *drift*, agar metrik mereka tidak hilang setelah *job* selesai. Evidently (Evidently AI 2025) melengkapi observabilitas khusus model dengan *report* distribusi fitur dan kinerja model yang dapat diaudit dan disisipkan ke dasbor Grafana.

IV.3 Perancangan Sub-sistem Tata Kelola Data

Bagian ini menjawab T-2 dengan merancang sub-sistem tata kelola data yang konsisten lintas siklus data, fitur, dan model. Tata kelola yang dimaksud mencakup katalog metadata, *lineage*, pengujian kontrak data, dan kontrol akses berbasis identitas. Pemilihan keempat aspek tersebut mengikuti penekanan pada KF-10 dan KNF terkait keamanan serta observabilitas pada Bab III, sekaligus menjawab efek domino fragmentasi pada lapisan tata kelola yang dibahas pada Subbab III.1.3. Sub-sistem ini

menjadi prasyarat bagi tiga sub-sistem lainnya karena katalog metadata berfungsi sebagai indeks bersama yang menghubungkan ingestasi, pelatihan, penyajian, dan pemantauan dalam satu grafik tata kelola.

IV.3.1 Katalog Metadata dan *Lineage*

DataHub menjadi pusat katalog metadata. Sumber metadata datang dari beberapa komponen: Kafka untuk *topic* dan skema, ClickHouse (ClickHouse Inc 2024) untuk tabel pada *warehouse*, MinIO dan Apache Iceberg dengan katalog Lakekeeper (Lakekeeper Authors 2025) untuk *dataset* pada *data lake*, Feast (Feast Project 2024) untuk definisi fitur, MLflow untuk *experiment* dan model, serta Kubeflow Pipelines untuk *pipeline*. Sambungan *lineage* antar komponen dikirim melalui peristiwa OpenLineage (OpenLineage Authors 2025) sehingga DataHub dapat merangkai grafik ketertelusuran yang seragam dari ingestasi sampai prediksi. Identitas pengguna katalog dirambatkan dari *identity provider* pusat yang dipakai bersama oleh seluruh antarmuka platform.

Ingestasi metadata pada DataHub dijalankan secara berkala melalui CronJob yang memanggil `datahub ingest` dengan resep YAML per sumber, sehingga penambahan sumber baru tinggal menambahkan resep tanpa memodifikasi kode pusat. Setiap resep meneruskan token OIDC dari OpenBao, sehingga otentikasi pada DataHub konsisten dengan otentikasi yang dipakai oleh komponen lain. Pengelolaan grafik metadata juga memuat *glossary* domain yang ditarik dari berkas YAML pada repositori Git, memberi lapisan semantik yang dapat ditelusuri oleh *business user*. Praktik DataHub (DataHub Project 2024) menempatkan katalog metadata sebagai sumber utama jawaban ketertelusuran yang lazim muncul pada audit kepatuhan, sehingga kapabilitas tersebut dapat diuji pada Bab VI.

IV.3.2 Kontrak Data dan Kualitas

Great Expectations memberi struktur untuk mendeklarasikan ekspektasi kualitas data, termasuk tipe kolom, batas nilai, dan jumlah baris minimum. Ekspektasi dijalankan pada dua titik: setelah ingestasi pada lapisan *bronze* dan setelah transformasi pada lapisan *silver* dan *gold*. Hasil pengujian dipublikasikan ke DataHub sehingga kualitas data tertampil bersama metadata lain. Chien (February 2022) memberi panduan agar metrik kualitas dipakai sebagai dasar pengambilan keputusan, bukan sekadar indikator pasif.

Bernardo dkk. (2024) dan Stoyanovich, Howe, dan Jagadish (2020) memberi kon-

teks yang melebihi aspek teknis, yakni pertanggungjawaban data sebagai bagian dari tata kelola platform. Kegagalan ekspektasi pada lapisan *bronze* memicu peringatan pada saluran observabilitas tanpa menjatuhkan jalur ingestasi, sementara kegagalan pada lapisan *silver* dan *gold* memicu penghentian *pipeline* agar data tidak berpindah ke lapisan berikutnya dengan kualitas yang tidak memenuhi kontrak. Setiap eksekusi Great Expectations menyimpan *run history* sehingga regresi kualitas dapat dirunut. dbt menjalankan model SQL pada ClickHouse dan menulis hasil ke tabel *gold* dengan kontrak yang divalidasi oleh suite ekspektasi yang dirancang pada *checkpoint* terjadwal.

IV.3.3 Identitas dan Kontrol Akses

Kontrol akses berbasis identitas menggunakan dex (Dex Authors 2025) sebagai *identity provider* OIDC dan oauth2-proxy (OAuth2 Proxy Authors 2025) sebagai *reverse proxy* yang menegakkan otentikasi pada antarmuka aplikasi. Identitas berlaku konsisten dari antarmuka pengguna ke *API* hingga ke akses data pada *warehouse* dan *lake*, dengan token OIDC yang sama dipakai oleh DataHub, MLflow, Grafana, Argo CD, dan Kubeflow Pipelines. Kebijakan otorisasi pada level kontainer dan beban kerja ditegakkan oleh Kyverno (Kyverno Authors 2024) pada lapis admisi Kubernetes, antara lain wajibnya *resource limits*, larangan *privileged container* di luar pengecualian eksplisit, dan kepatuhan label *Pod Security Standards*. Apa yang dibahas oleh Ahmad dkk. (June 2024) tentang strategi keamanan MLOps menjadi rujukan utama, karena banyak ancaman MLOps timbul dari ketidakhadiran kontrol akses yang konsisten di antara lapisan.

SpiceDB (AuthZed 2025) dihadirkan sebagai layanan otorisasi berbasis Zanzibar yang menetapkan hubungan akses pada level objek, sehingga kebijakan akses dataset, fitur, dan model dapat diekspresikan dengan model relasi yang seragam. OpenBao menjadi sumber tunggal rahasia yang disinkronkan ke *namespace* target oleh *External Secrets Operator*, dan *Key Encryption Service* mengenkripsi rahasia tersebut pada lapis MinIO dengan kunci yang dirotasi secara berkala. Falco (Falco Authors 2025) memonitor peristiwa *runtime* dan mengirim peringatan ke Loki ketika perilaku mencurigakan terdeteksi, sehingga lapisan *runtime* ditutup oleh observasi berkelanjutan. Chaos Mesh (Chaos Mesh Authors 2025) dipakai untuk pengujian keandalan terjadwal yang memvalidasi bahwa kontrol akses tetap terjaga pada kondisi kegagalan *partial*.

IV.4 Perancangan Sub-sistem Deteksi *Drift* dan *Continuous Training*

Bagian ini menjawab T-3 dengan merancang sub-sistem deteksi *drift* pada fitur dan label yang otomatis memicu pelatihan ulang pada saat melewati ambang batas. Sub-sistem ini menjawab kombinasi *drift* dan *temporal leakage* yang menjadi salah satu rumusan masalah. Penekanan diberikan pada otomatisasi penuh: pengukuran, pengambilan keputusan, pelatihan ulang, validasi, dan promosi versi dijalankan tanpa intervensi manual. Sub-sistem ini menjadi titik temu antara DataOps dan MLOps karena *drift* terdeteksi pada lapisan data sementara konsekuensinya dieksekusi pada lapisan model.

IV.4.1 Pengukuran *Drift*

Dua indikator utama dipilih sebagai detektor. *Population Stability Index* (PSI) dipakai untuk fitur kategorikal dan kontinu dengan *binning*, sedangkan uji Kolmogorov-Smirnov dipakai untuk distribusi nilai kontinu (Reis dkk. 2016). Pengukuran dilakukan pada jendela waktu yang bergerak dan dibandingkan dengan jendela referensi. Tingkat *drift* dikategorikan menjadi *stable*, *moderate*, dan *severe* dengan ambang yang dapat dikonfigurasi per fitur. Bayram, Ahmed, dan Kassler (2022), Lu dkk. (2019), dan Hinder dkk. (2023) menjadi rujukan untuk penyusunan kebijakan keluarga detektor dan respons platform.

Pengukuran *drift* dijalankan oleh CronJob yang menarik fitur *baseline* dari ClickHouse dan fitur jendela aktif dari Feast, lalu menghitung indikator pada *namespace model-lifecycle* agar terisolasi dari beban kerja produksi lain. Hasil pengukuran dipublikasikan ke tabel `gold.drift_multi_scale` pada ClickHouse sehingga riwayat dapat dianalisis secara historis untuk audit dan pelaporan. Pushgateway menerima metrik tiap eksekusi sehingga Prometheus dapat merangkainya menjadi run-tutan jangka panjang yang ditampilkan pada dasbor Grafana. Evidently menambah lapisan visual *report* HTML yang disimpan pada MinIO dan ditautkan dari DataHub agar dapat dirujuk oleh *data scientist* ketika menelusuri penurunan kinerja model.

IV.4.2 Alur Pelatihan Ulang Otomatis

Alur pelatihan ulang dijalankan oleh Argo CronWorkflow yang berperan sebagai *control loop*. *Workflow* ini menjalankan *job* pengukuran *drift*, kemudian memutuskan apakah pemicu pelatihan diperlukan. Jika diperlukan, *workflow* memicu *pipeline* pelatihan pada Kubeflow Pipelines. *Pipeline* pelatihan menarik fitur dari Feast dengan operasi `get_historical_features` yang menjamin *point-in-time correct-*

ness, kemudian menulis model yang dihasilkan ke MLflow. Evidently (Evidently AI 2025) dijalankan sebagai pelapor pelengkap yang menghasilkan ringkasan distribusi fitur dan kinerja model dalam bentuk *report* yang dapat diaudit. Validasi kualitas model dijalankan sebelum penyebaran.

Apabila lulus validasi, Argo Rollouts (Argo Rollouts Authors 2025) mengelola promosi versi baru pada KServe dengan strategi *canary* berbasis analisis metrik Prometheus, sehingga pengembalian otomatis dapat dilakukan ketika *signal* kinerja menurun. Céspedes Sisniega dkk. (2024) memberi contoh penerapan deteksi *drift* pada lingkungan *serverless*, dan pendekatan yang serupa diadaptasi ke lingkungan Kubernetes pada platform ini. Argo Rollouts membandingkan metrik versi *stable* dan versi *canary* dengan kueri Prometheus terjadwal pada interval lima menit, menghentikan promosi otomatis pada selisih metrik yang melampaui ambang konfigurasi. Riwayat promosi dan pengembalian tercatat pada *namespace* Argo sehingga audit promosi dapat dijalankan langsung dari Argo CD UI tanpa berpindah *tool*.

IV.4.3 Mekanisme Penanganan Kasus Khusus

Sub-sistem ini melayani tiga kasus khusus. Pertama, ketika *drift* terdeteksi pada fitur tetapi label belum tersedia, *workflow* hanya melakukan pelatihan ulang setelah label berjalan cukup lama. Kedua, ketika *drift* terjadi karena perubahan distribusi sementara, ambang dihitung dengan jendela bergulir agar tidak memicu pelatihan ulang yang tidak perlu. Ketiga, ketika model baru gagal lulus validasi, KServe mempertahankan versi lama dan *workflow* mencatat kegagalan ke MLflow.

Penanganan tiga kasus tersebut dirancang sebagai cabang pada DAG CronWorkflow sehingga keputusan dapat ditelusuri pada riwayat eksekusi Argo. Setiap cabang menulis catatan ke MLflow tags agar konteks keputusan tersimpan bersama dengan eksperimen yang dipicu. Kegagalan validasi memicu peringatan ke Alertmanager dengan rute khusus untuk peran *machine learning engineer*, sehingga investigasi dapat dimulai tanpa menunggu pelaporan manual. Riwayat tiga kasus tersebut juga dipakai untuk memperbarui ambang *drift* pada konfigurasi melalui mekanisme *tuning* berkala yang dijalankan pada Argo Workflows.

IV.5 Perancangan Sub-sistem Layanan Fitur *Dual-Store*

Bagian ini menjawab T-4 dengan merancang sub-sistem layanan fitur yang menyajikan fitur tabular, fitur deret waktu, dan *embedding* berdimensi tinggi dalam satu kontrak data dari satu sumber kebenaran. Feast digunakan sebagai *framework* layan-

an fitur yang menyatukan kontrak antara *offline* dan *online*. Pemisahan beban kerja dilakukan secara fisik antara *store* kunci-nilai untuk fitur tabular dan *store* vektor untuk *embedding*, tetapi tetap menyatu pada level kontrak Feast. Penyatuan kontrak ini menghilangkan perlunya klien menyentuh dua API yang berbeda untuk mengambil fitur multimoda pada saat *inference*.

IV.5.1 Penyimpanan *Offline* dan *Online*

Offline store menggunakan ClickHouse untuk fitur tabular dan deret waktu yang dipakai pada fase pelatihan. ClickHouse dipilih karena kemampuan kompresi tinggi, latensi kueri rendah untuk *aggregation*, dan kompatibilitas dengan kontrak Feast. *Online store* menggunakan Valkey (Valkey Contributors 2025) untuk fitur yang dipakai pada fase *inference* dengan target latensi p99 di bawah sepuluh milidetik. Valkey dipilih karena kompatibilitas penuh dengan protokol RESP dan lisensi BSD pasca perubahan model lisensi Redis, sehingga klien Feast yang bersifat *drop-in* dapat dipakai tanpa modifikasi.

Vector search HNSW tidak ditempelkan pada *online store* tabular, melainkan disediakan oleh Qdrant (Qdrant Team 2025) sebagai indeks vektor terpisah, sehingga beban kerja kunci-nilai dan beban kerja pencarian vektor terpisah secara operasional namun tetap berbagi kontrak fitur yang sama melalui Feast. Pemisahan *store* memudahkan tuning *resource* secara mandiri karena beban Valkey berorientasi kunci-nilai dengan akses *point lookup* sedangkan beban Qdrant berorientasi pencarian tetangga terdekat yang membutuhkan memori untuk indeks HNSW. Kedua *store* dideklarasikan pada repositori Feast sebagai *online_store* dan *vector_store* dengan kelas *provider* terpisah, sehingga klien yang menulis dan membaca fitur menggunakan API tunggal Feast. Pembaruan skema fitur diterapkan ke ClickHouse melalui ALTER TABLE terkelola oleh dbt dan secara serempak ke Feast melalui `feast apply` sehingga skema *offline* dan *online* selalu sinkron.

IV.5.2 Dukungan *Vector Embeddings* dan HNSW

Embedding dihasilkan oleh model `sentence-transformers/all-mpnet-base-v2` (Devlin dkk. 2019) berdimensi 768 untuk teks, dan disimpan sebagai fitur dengan dimensi yang konsisten antara fase pelatihan dan fase *inference*. Pencarian tetangga terdekat dijalankan oleh Qdrant (Qdrant Team 2025) dengan indeks HNSW (Malkov dan Yashunin 2020) berparameter `m=16` dan `ef_construct=200` yang memberi keseimbangan baik antara waktu kueri dan kualitas. Qdrant menyediakan kueri *gRPC* berlatensi rendah serta penyaringan *payload* pada saat pencarian, sehingga *embe-*

dding dapat digabung dengan fitur tabular dari Valkey untuk membentuk *request inference* yang lengkap. Bruch, Gai, dan Ingber (2023) dan Campese, Lauriola, dan Moschitti (2024) memberi rujukan untuk fungsi peleburan hasil pencarian pada konteks *hybrid retrieval*.

Parameter HNSW dikonfigurasi melalui *collection* Qdrant yang dideklarasikan oleh *init Job* sehingga penambahan koleksi baru dapat dilakukan dengan menambah berkas YAML, tanpa intervensi langsung pada Qdrant. *Snapshot* koleksi vektor dijalankan secara berkala ke MinIO sebagai bagian dari pencadangan Velero untuk objek terkait. Pemilihan dimensi 768 berdasarkan model *all-mpnet-base-v2* memberi keseimbangan antara kualitas semantik dan biaya penyimpanan, dan dapat disesuaikan dengan model alternatif tanpa mengubah kontrak Feast. Latensi p95 ditargetkan di bawah dua puluh milidetik untuk pencarian top-10 pada koleksi berukuran sampai sepuluh juta vektor, mengikuti panduan kinerja yang dirangkum oleh Malkov dan Yashunin (2020).

IV.5.3 Jaminan *Point-in-Time Correctness*

Feast menjamin *point-in-time correctness* melalui operasi *get_historical_features*. Operasi ini melakukan *point-in-time join* antara entitas dan riwayat fitur dengan menggunakan *event_timestamp* dan *created_timestamp*, sehingga nilai fitur yang dipakai pada saat pelatihan identik dengan nilai yang akan tersedia pada saat *inference* di waktu yang sama. Vela dkk. (2022) dan Wang dan Ruf (2021) memperlihatkan dampak buruk *temporal leakage* pada validitas evaluasi model, dan Feast mencegah kebocoran tersebut secara struktural pada level kontrak data. Setiap *FeatureView* mendeklarasikan *ttl* untuk membatasi usia fitur yang dianggap valid, sehingga fitur usang tidak diam-diam masuk ke set pelatihan.

Mekanisme jaminan ini diuji melalui dua jalur. Pertama, *notebook* reproduksi memanggil *get_historical_features* pada titik waktu historis dan membandingkan keluaran dengan ekstraksi manual yang dijalankan terhadap log peristiwa Kafka, sehingga validitas dapat dibuktikan pada level kasus. Kedua, kontrak antara *Online Store* dan *Offline Store* divalidasi dengan menjalankan kueri yang sama pada Valkey dan ClickHouse pada titik waktu yang sama dan membandingkan keluaran. Kombinasi dua jalur uji ini menutup ruang *temporal leakage* struktural pada lapisan fitur tanpa bergantung pada disiplin manual pengembang *pipeline*.

IV.5.4 Aliran Materialisasi Fitur

Materialisasi fitur dilakukan dengan dua jalur. Jalur *batch* menjalankan transformasi pada ClickHouse atau Spark, kemudian Feast memuat hasilnya ke *online store* secara periodik melalui operasi *materialize*. Jalur *stream* menggunakan Flink untuk menghitung fitur *real-time* yang ditulis langsung ke Valkey untuk fitur tabular dan ke Qdrant untuk *embedding* vektor, serta dipublikasikan ke ClickHouse pada saat yang sama, sehingga *offline* dan *online* tetap konsisten. Jain dan Das (2025) menyoroti pentingnya integrasi DataOps dengan MLOps untuk menjaga konsistensi seperti ini pada skala produksi.

Jadwal materialisasi *batch* dipasang pada Argo CronWorkflow dengan interval per jam untuk fitur agregasi dan per hari untuk fitur *rolling window* berdurasi panjang. Eksekusi *materialize* mengeluarkan metrik durasi dan jumlah baris ke Pushgateway sehingga ketertinggalan dapat terdeteksi melalui *alerting* Prometheus. Jalur *stream* menggunakan *exactly-once* semantik Flink dengan *checkpoint* ke MinIO setiap interval pendek, sehingga kegagalan *TaskManager* tidak menyebabkan duplikasi pada *online store*. Konsistensi antara *offline* dan *online* divalidasi oleh CronJob *feature-store-consistency-check* yang membandingkan jumlah baris dan agregasi pada keduanya, dan memicu peringatan ketika selisih melebihi ambang konfigurasi.

IV.6 Alur Kerja *End-to-End*

Alur kerja menyatukan empat sub-sistem ke dalam satu siklus berkelanjutan. Pertama, data baru masuk melalui Kafka, divalidasi oleh Great Expectations, kemudian disimpan pada *warehouse* ClickHouse dan *data lake* MinIO dengan katalog Lakekeeper. Kedua, fitur dihitung pada Flink atau Spark, lalu diregistrasi pada Feast dan dimaterialisasi ke ClickHouse sebagai *offline store*, Valkey sebagai *online store* tabular, dan Qdrant sebagai indeks vektor terpisah. Ketiga, Argo CronWorkflow mengukur *drift* dan, jika ambang dilewati, memicu *pipeline* pelatihan Kubeflow Pipelines. Pelatihan menggunakan *get_historical_features* untuk menjamin konsistensi temporal.

Keempat, hasil pelatihan ditulis ke MLflow, validasi model dijalankan, lalu Argo Rollouts mempromosikan versi KServe baru dengan strategi *canary* berbasis analisis metrik. Kelima, perubahan pada konfigurasi disinkronkan dari Git oleh Argo CD, sedangkan pengamatan kondisi sistem dipantau oleh Prometheus, Loki, dan Grafana Tempo melalui Grafana. Siklus ini berputar setiap kali data baru masuk atau *drift*

terdeteksi, sehingga platform terus menerus menjaga akurasi model di lingkungan produksi. Rodrigues dkk. (2025) memberi rujukan tambahan untuk arsitektur pembelajaran *near real-time* pada lingkungan yang serupa, dan pendekatan yang dirancang di sini menyesuaikan rujukan tersebut dengan kontrak Feast dan Argo Rollouts agar siklus tetap dapat dijalankan secara deklaratif pada satu *cluster* Kubernetes.

BAB V

IMPLEMENTASI ARSITEKTUR PLATFORM DATAOPS DAN MLOPS

Bab ini membahas implementasi platform DataOps dan MLOps terintegrasi sesuai rancangan pada Bab IV. Pembahasan dimulai dari lingkungan implementasi yang menjadi dasar penyebaran komponen, kemudian dilanjutkan dengan implementasi empat sub-sistem mengikuti urutan pemenuhan tujuan T-1 sampai T-4. Bab ini ditutup dengan verifikasi jalur eksekusi menggunakan satu *use case* sebagai instans pengujian. Penjabaran detail biner dan konfigurasi disimpan pada repositori platform, sehingga bab ini menyajikan pokok-pokok implementasi yang penting untuk membangun pemahaman atas bentuk akhir dari rancangan. Pengelolaan kode dan konfigurasi platform berada pada direktori `platform/`, sedangkan kode dan konfigurasi yang spesifik pada *use case* berada pada direktori *use case* tersendiri yang sepenuhnya terpisah dari `platform/`, sehingga sifat domain-agnostik dari platform tetap terjaga.

V.1 Lingkungan Implementasi

Lingkungan implementasi platform dibangun pada satu *cluster* Kubernetes (Cloud Native Computing Foundation 2024a) berbasis k3s yang berjalan pada satu *node*. Pilihan ini memenuhi kebutuhan eksekusi pada lingkungan terbatas tanpa mengubah pola penyebaran yang dapat dipakai pada *cluster* produksi dengan banyak *node*. Distribusi k3s membawa *control plane* ringan, *containerd* sebagai *runtime*, dan beberapa *add-on* bawaan yang dapat dinonaktifkan agar tidak berbenturan dengan komponen pilihan platform. Konfigurasi *node* memakai satu *disk* lokal dengan *ext4* sebagai *filesystem*, dan tata letak *namespace* pada platform dibakukan agar perpindahan ke *cluster* produksi dengan banyak *node* tinggal mengubah berkas *overlay Kustomize* tanpa mengganti *manifest* dasar.

V.1.1 Konfigurasi Dasar *Cluster*

Cluster menjalankan komponen dasar berikut: Istio (Istio Authors 2024) sebagai *service mesh* dengan mTLS dalam mode *strict* pada lalu lintas internal, APISIX (Apache APISIX Authors 2025) sebagai *API gateway* pada perbatasan *mesh*, kontrol akses jaringan menggunakan *NetworkPolicy* pada level Kubernetes, dan Kyverno (Kyverno Authors 2024) sebagai *policy engine* pada lapis admisi. Falco (Falco Authors 2025) memantau perilaku *runtime* pada level *kernel*, sedangkan Trivy Operator (Aqua Security 2025) memindai citra dan beban kerja secara berkala. *Storage class* bawaan menggunakan *local-path provisioner* untuk *persistent volume* berbasis penyimpanan lokal, dengan Velero (Velero Authors 2025) melakukan pencadangan terjadwal ke MinIO. Chaos Mesh (Chaos Mesh Authors 2025) disiapkan untuk uji ketahanan dengan eksperimen *fault injection* yang terbatas pada *namespace* tertentu.

Konfigurasi sumber daya memberi prioritas pada satu replika *idle* per beban kerja, sementara *Horizontal Pod Autoscaler*, *Vertical Pod Autoscaler* dalam mode rekomendasi, dan KEDA *ScaledObject* disiapkan pada *template* agar penskalaan dapat aktif ketika beban nyata datang. *Resource limits* pada setiap kontainer ditetapkan sesuai panduan Kyverno *require-resource-limits* untuk mencegah kelebihan kuota *node*. Penjadwalan kelas prioritas memakai *PriorityClass* platform agar komponen kritis tidak terusir oleh beban kerja sekali pakai. *External Snapshotter* dipasang sebagai prasyarat untuk *snapshot CSI* yang dipakai oleh Velero, sehingga rekonstruksi *cluster* dari pencadangan dapat dilakukan tanpa intervensi manual.

V.1.2 Manajemen Rahasia dan Identitas

OpenBao (OpenBao Authors 2025) bertindak sebagai penyimpan rahasia. *External Secrets Operator* bertugas menyebarkan rahasia dari OpenBao ke *namespace* target tanpa menyalin nilai rahasia ke dalam berkas *manifest*. Identitas pengguna dikelola oleh dex (Dex Authors 2025) sebagai *identity provider* OIDC, dengan oauth2-proxy (OAuth2 Proxy Authors 2025) sebagai *reverse proxy* yang menegakkan otentikasi pada antarmuka aplikasi seperti DataHub, MLflow, Grafana, Argo CD, dan Kube-flow Pipelines. Kebijakan otorisasi pada *cluster* dijalankan oleh Kyverno (Kyverno Authors 2024) pada lapis admisi, dengan aturan yang mewajibkan *resource limits*, melarang *privileged container* di luar pengecualian eksplisit, dan memaksa kepatuhan label *Pod Security Standards*.

OpenBao dibangkitkan oleh *Job* *openbao-bootstrap* yang melakukan *init* dan *auto-unseal* dengan kunci yang tersimpan di MinIO (MinIO, Inc. 2024) dengan enk-

ripsi sisi server. *External Secrets Operator* dipasang pada *namespace security* dengan *ClusterSecretStore* tunggal yang berisi konfigurasi koneksi ke OpenBao. SpiceDB (AuthZed 2025) dipasang sebagai layanan otorisasi berbasis Zanzibar yang menetapkan hubungan akses pada level objek tata kelola, dengan basis data PostgreSQL melalui CNPG sebagai *datastore*. *Key Encryption Service* dipasang sebagai *sidecar* MinIO untuk enkripsi rahasia, dengan rotasi kunci dijalankan oleh *CronJob* terjadwal.

V.1.3 Pola GitOps

Argo CD (Argo Project 2024) menjadi satu-satunya jalur perubahan pada *cluster*. Repositori Git pada Gitea (Gitea Contributors 2025) bertindak sebagai sumber kebenaran konfigurasi. Setiap perubahan dilakukan melalui *commit* pada repositori dan diterapkan ke *cluster* oleh Argo CD melalui rekonsiliasi. Pola ini menjamin reproduksibilitas dan menghasilkan jejak audit untuk setiap perubahan. *Pipeline* CI yang dijalankan oleh Tekton (Tekton Contributors 2024) membangun *image* dan menjalankan pengujian sebelum *commit* menjadi sumber penyebaran.

ApplicationSet Argo CD dipakai untuk mendeklarasikan banyak *Application* sekaligus dengan satu *template* per kelompok, sehingga penambahan komponen baru tinggal menambahkan *generator* tanpa menambah berkas *Application* satu per satu. Argo CD diatur dengan mode *manual-sync* pada *ApplicationSet template* untuk mencegah *auto-prune* yang dapat menghapus sumber daya secara tidak sengaja. Tekton menjalankan *Pipeline* yang menghasilkan citra ke registri *localhost:5000* yang berfungsi sebagai *mirror* lokal, dan menulis kembali *commit* dengan referensi citra yang sudah diperbarui pada *values.yaml customization*. *Pre-apply* dan *post-apply* hook pada *apply-component.sh* menutup celah *race condition* antara CRD dan CR, sehingga seluruh komponen *cluster* dapat diulang pemasangannya dengan satu perintah *Makefile*.

V.2 Implementasi Arsitektur Terintegrasi

Bagian ini mengimplementasikan rancangan pada Bagian IV.2. Komponen disusun ke dalam *namespace* per lapisan dan dihubungkan melalui kontrak *API* yang stabil. Setiap *namespace* memuat satu kelompok komponen fungsional sehingga kebijakan *NetworkPolicy* dapat ditegakkan dengan model *default-deny* dan pengecualian eksplisit per arah komunikasi. Penyebaran komponen mengikuti pola GitOps melalui Argo CD, sedangkan eksekusi *pipeline* CI mengikuti pola Tekton, dengan setiap komponen tunduk pada *template ApplicationSet* agar penambahan komponen baru

tidak menambah *boilerplate*.

V.2.1 Lapisan Ingestasi

Apache Kafka (Apache Software Foundation 2024c) diterapkan menggunakan operator Strimzi. *Kafka Connect* digunakan untuk integrasi dengan basis data eksternal melalui Debezium dan untuk *sink* ke Apache Iceberg pada MinIO (MinIO, Inc. 2024) dengan katalog Lakekeeper (Lakekeeper Authors 2025). *Karapace* berperan sebagai registri skema dengan dukungan Avro dan JSON Schema. Lalu lintas masuk dari konsumen di luar *mesh* dilewatkan melalui *API gateway* APISIX (Apache APISIX Authors 2025) dengan mTLS yang dijembatani pada perbatasan.

Cluster Kafka diatur dengan *KRaft* murni tanpa ZooKeeper agar topologi tetap ringan. SASL_SSL dipakai sebagai mekanisme otentikasi antar klien dengan sertifikat klien dikelola *Strimzi* melalui *KafkaUser*. *Topic* dideklarasikan sebagai *KafkaTopic* CR dan ACL dideklarasikan sebagai bagian dari *KafkaUser* sehingga keseluruhan kontrak ingestasi tersimpan sebagai sumber daya Kubernetes yang dapat di-*audit*. PeerAuth PERMISSIVE diterapkan pada *broker* agar penyedia data di luar *mesh* masih dapat berkomunikasi tanpa terlebih dahulu menambahkan *sidecar* Istio.

V.2.2 Lapisan Pemrosesan

Pemrosesan *stream* menggunakan operator Flink yang mengelola *FlinkDeployment*. Pemrosesan *batch* menggunakan operator Spark dan dijalankan sebagai *SparkApplication*. Transformasi pada *warehouse* menggunakan dbt yang dijalankan sebagai *task* KubernetesPodOperator pada DAG Airflow (Apache Software Foundation 2024a). Airflow mengorkestrasi *pipeline* data berorientasi *batch*, termasuk pembangunan lapisan *gold*, sedangkan orkestrasi pelatihan model ditangani oleh Kubeflow Pipelines.

Flink dipasang dengan *session cluster* mode *Standalone* dan tugas *stream* dideklarasikan sebagai *FlinkSessionJob* CR sehingga setiap *job* dapat di-*deploy* secara independen melalui Argo CD. Spark Operator memanfaatkan *webhook* mutasi untuk menyisipkan *node selector* dan *toleration* berdasarkan label *queue-name* yang ditetapkan oleh Kueue. dbt menjalankan model SQL pada ClickHouse dengan *profiles.yaml* yang merujuk rahasia dari OpenBao melalui ESO. Airflow memakai *git-sync sidecar* yang menarik DAG dari Gitea sehingga pembaruan DAG tidak memerlukan pembangunan ulang citra.

V.2.3 Lapisan Penyimpanan

Warehouse kolumnar menggunakan ClickHouse (ClickHouse Inc 2024) dengan operator ClickHouse pada satu *shard*. *Data lake* menggunakan MinIO sebagai *object store* dan Apache Iceberg sebagai *table format*, dengan Lakekeeper (Lakekeeper Authors 2025) sebagai katalog Iceberg REST dan Trino (Trino Software Foundation 2024) sebagai *query engine*. Penyimpanan *online* menggunakan Valkey (Valkey Contributors 2025) sebagai penyimpanan utama *online store* fitur tabular dengan protokol RESP, dan Qdrant (Qdrant Team 2025) sebagai indeks vektor terpisah untuk fitur *embedding* berdimensi tinggi. Penyimpanan relasional menggunakan PostgreSQL melalui operator CNPG, dan MySQL melalui operator yang sesuai untuk komponen yang memerlukan *engine* berbeda. Apache Superset (Apache Superset Authors 2025) disiapkan sebagai antarmuka BI untuk peran *business user* dengan koneksi ke ClickHouse, Trino, dan MySQL melalui SQLAlchemy.

ClickHouse memakai ClickHouseKeeper sebagai pengganti ZooKeeper untuk koordinasi metadata replikasi dengan port 9181. MinIO dipasang dengan empat drive pada satu *node* untuk menjaga skema penamaan erasure code yang seragam ketika *cluster* berpindah ke banyak *node*. Lakekeeper menyediakan Warehouse dan Project REST API yang dipakai oleh Spark, Trino, dan Flink dengan menggunakan *header x-project-id* sebagai pemisah antar *warehouse*. lakeFS (Treeverse Ltd. 2025) dipasang sebagai lapisan *version control* di atas MinIO sehingga eksperimen pada cabang data dapat dijalankan tanpa mengganggu data utama.

V.2.4 Lapisan Siklus Hidup Model dan Penyajian

MLflow (MLflow Contributors 2024) dijalankan dengan basis data PostgreSQL dan *artifact store* pada MinIO. Kubeflow Pipelines (Kubeflow Contributors 2024) dijalankan pada *namespace* terpisah dengan dukungan *multi-tenant*, dilengkapi Kubeflow Notebooks untuk lingkungan eksperimen interaktif dan Kubeflow Trainer (Kubeflow Authors 2025c) untuk pelatihan terdistribusi. KServe (KServe Contributors 2024) dipakai untuk *inference* dengan dukungan *runtime* MLflow, scikit-learn, dan ONNX, dan dengan Knative Serving (Knative Authors 2025) sebagai bidang *serverless* bawaan yang menyediakan penskalaan ke nol. Argo Workflows berperan sebagai *orchestrator job* di luar Kubeflow Pipelines, sedangkan Argo Rollouts (Argo Rollouts Authors 2025) mengelola promosi versi model dengan strategi *canary* dan *progressive analysis* berbasis metrik Prometheus.

Kubeflow Pipelines memakai PostgreSQL sebagai *metadata store* dan MinIO se-

bagai *artifact store*, dengan *kfp-launcher* yang membungkus eksekusi langkah *pipeline* agar autentikasi ke MinIO dilakukan menggunakan *secretAsEnv*. *KServe InferenceService* dideklarasikan pada *namespace* *model-serving* dengan label *part-of=kserve* agar Kyverno tidak menolak *Pod mutator* bawaan. Kueue (Kueue Authors 2024) mengelola antrean pelatihan dengan *ClusterQueue* per kelompok beban kerja sehingga sumber daya CPU dan GPU dapat dibagi adil antara *tenant*. Argo Rollouts mengevaluasi metrik dari Prometheus pada interval lima menit untuk menentukan apakah versi *canary* layak dipromosikan menjadi versi *stable*.

V.2.5 Lapisan Observabilitas

Prometheus (Prometheus Authors 2024) mengumpulkan metrik dari semua komponen dengan *ServiceMonitor* dan *PodMonitor*. Loki (Grafana Labs 2024b) mengumpulkan log melalui Alloy. Grafana Tempo (Grafana Labs 2024c) menerima *trace* dari OpenTelemetry Collector yang tersebar pada *node*. Grafana (Grafana Labs 2024a) menggabungkan ketiganya pada satu antarmuka.

Sloth (Sloth Authors 2025) menurunkan definisi *service-level objective* ke dalam aturan Prometheus sehingga *error budget* dapat dipantau bersama metrik lain. Pushgateway berperan sebagai titik kumpul metrik untuk *job* berdurasi pendek yang berjalan di luar siklus *scrape*, termasuk CronJob pengukuran *drift* dan *CronWorkflow* pemicu pelatihan ulang. OpenCost dipakai untuk pemantauan biaya per beban kerja, sedangkan Pyroscope mengumpulkan profil CPU dan memori secara berkelanjutan dari layanan yang menjadi *hotspot*. Evidently (Evidently AI 2025) dijalankan sebagai *job* berkala yang menghasilkan *report* HTML kualitas distribusi fitur dan disimpan pada MinIO sebagai jejak audit yang dapat dirujuk dari DataHub.

V.3 Implementasi Sub-sistem Tata Kelola Data

Bagian ini mengimplementasikan rancangan pada Bagian IV.3. Penekanan implementasi diberikan pada konsistensi katalog metadata, jadwal pengujian kontrak data, dan penegakan kontrol akses pada tiga lapis yang berbeda. Setiap sub-sub-bagian memetakan satu sub-bab perancangan ke berkas konfigurasi yang dapat ditelusuri pada repositori platform. Penomoran KF-09, KF-10, dan kebutuhan KNF terkait keamanan dipakai sebagai acuan untuk menilai apakah implementasi memenuhi seluruh kebutuhan yang dirumuskan pada Bab III.

V.3.1 Katalog Metadata dan Lineage

DataHub (DataHub Project 2024) dijalankan dengan komponen GMS, *frontend*, dan beberapa pekerjaan *ingest*, dengan OpenSearch sebagai *search backend* dan PostgreSQL sebagai *metadata backend*. *Job ingest* berjalan secara berkala sebagai *CronJob* untuk menarik metadata dari Kafka, ClickHouse, MinIO dan Iceberg, Feast, MLflow, dan Kubeflow Pipelines. Hasil *ingest* menyatukan *lineage* antar lapisan, mulai dari *topic* pada Kafka hingga model yang dilayani oleh KServe. OpenLineage (OpenLineage Authors 2025) dipakai sebagai protokol pengirim peristiwa *lineage* dari komponen *pipeline*, baik Airflow maupun Kubeflow Pipelines, agar grafik ketertelusuran di DataHub tersusun dengan kosa kata yang seragam.

Citra `datahub-feast-deps` dibangun secara lokal dengan Kaniko untuk menambahkan *plugin* Feast pada *job ingest* sehingga sumber Feast dapat dipindai secara langsung. Otentikasi ke GMS menggunakan token sistem yang disimpan di OpenBao dan disebarkan ke *namespace* `data-governance` oleh *External Secrets Operator*. Konfigurasi DataHub pada level UI dikelola melalui `datahub-frontend-secret` dengan *checksum annotation* pada GMS dan *frontend* agar rotasi rahasia memicu *auto-roll deployment*. Jadwal *ingest* dapat berkonflik dengan beban kerja besar di *node*, dan untuk itu *native sidecar* Istio diaktifkan pada *CronJob* agar koneksi mTLS ke GMS tidak putus saat sidecar terminasi setelah *job* selesai.

V.3.2 Kontrak Data dan Kualitas

Great Expectations (Great Expectations 2024) mendefinisikan ekspektasi kualitas data pada lapisan *bronze* setelah ingestasi dan pada lapisan *silver* dan *gold* setelah transformasi. Setiap *expectation suite* disimpan pada repositori Git dan dijalankan oleh *CronJob* yang membaca hasil ke DataHub. Pengujian yang gagal akan menutup jalur *promotion* ke lapisan berikutnya dan memunculkan peringatan pada Grafana. Hasil ekspektasi disimpan ke MinIO sebagai *Data Docs* HTML sehingga konteks kegagalan dapat ditelusuri tanpa perlu menjalankan kembali *suite*.

dbt menjalankan model SQL untuk lapisan *silver* dan *gold* pada ClickHouse, dengan `schema.yml` yang memuat *tests* dasar seperti `unique`, `not_null`, dan `accepted_values`. Hasil dbt dirilis ke DataHub melalui *run event* OpenLineage yang dipancarkan oleh *task* `KubernetesPodOperator` dbt pada DAG Airflow. *Checkpoint* Great Expectations dijadwalkan per lapisan dengan jeda yang berbeda agar ekspektasi yang berat tidak berdampak pada beban *warehouse*. *Suite* fakta pelatihan diuji setelah *materialization* pada tabel `gold.fct_training_data` agar set pelatihan tidak masuk

pipeline pelatihan apabila kualitas data tidak memenuhi kontrak.

V.3.3 Kontrol Akses

Kontrol akses pada lapis jaringan menggunakan *NetworkPolicy* dengan kebijakan *default-deny* per *namespace* dan pengecualian yang dideklarasikan secara eksplisit. Kontrol akses pada lapis aplikasi menggunakan dex (Dex Authors 2025) sebagai *identity provider* OIDC dan oauth2-proxy (OAuth2 Proxy Authors 2025) sebagai *reverse proxy* yang menegakkan otentikasi pada antarmuka aplikasi. Kyverno (Kyverno Authors 2024) menerapkan kebijakan pada lapis admisi, antara lain wajibnya *resource limits*, larangan *privileged container* di luar pengecualian eksplisit, kepatuhan label, dan penegakan label *Pod Security Standards*. Falco (Falco Authors 2025) memantau peristiwa *runtime* pada level *kernel* untuk mendeteksi pelanggaran kebijakan keamanan setelah *pod* berjalan.

SpiceDB (AuthZed 2025) berfungsi sebagai layanan otorisasi yang dipakai DataHub dan Kubeflow Pipelines untuk hubungan akses pada level objek, dengan basis data PostgreSQL melalui CNPG. *Policy* Kyverno disusun pada *namespace security* dengan *policy generator* yang menyebar peraturan ke *namespace* aplikasi tanpa duplikasi sumber daya. Kyverno mode *failurePolicy=Ignore* dipakai pada *ValidatingPolicy* agar admisi tidak terblokir saat *webhook* gagal pada kondisi *cluster* yang sibuk. Falco mengirim peringatan ke Loki melalui *falcosidekick* dengan *topic* khusus sehingga regu keamanan dapat menelusuri peristiwa terkait kebijakan dari satu antarmuka Grafana.

V.4 Implementasi Sub-sistem Deteksi *Drift* dan *Continuous Training*

Bagian ini mengimplementasikan rancangan pada Bagian IV.4. Penekanan implementasi diberikan pada otomatisasi penuh *control loop* dari pengukuran sampai promosi versi, dan pada penegakan ambang yang dapat dikonfigurasi per model. Pemicu pelatihan ulang dirancang agar dapat dijalankan secara berkala atau dipicu oleh peristiwa *drift* yang melebihi ambang. Setiap sub-sub-bagian menjelaskan satu lapis implementasi: detektor, *control loop*, dan penanganan kasus khusus.

V.4.1 Detektor *Drift*

Detektor *drift* diimplementasikan sebagai *service* Python yang membaca fitur dari ClickHouse dengan jendela waktu yang dapat dikonfigurasi. Indikator PSI dihitung dengan *binning* adaptif, dan uji Kolmogorov-Smirnov dijalankan dengan

ambang yang dapat disesuaikan per fitur (Reis dkk. 2016; Bayram, Ahmed, dan Kassler 2022; Lu dkk. 2019). Hasil pengukuran ditulis ke ClickHouse sebagai tabel *drift_multi_scale* dan dipublikasikan ke Pushgateway agar dapat diserap oleh Prometheus. Evidently (Evidently AI 2025) dijalankan sebagai pelapor pelengkap yang menghasilkan ringkasan distribusi fitur dan kinerja model dalam bentuk *report HTML* yang dapat diaudit dan dilampirkan pada katalog DataHub.

Service berjalan sebagai *CronJob* pada *namespace* *model-lifecycle* dengan jadwal lima belas menit untuk fitur volatil dan satu jam untuk fitur tabular yang lebih stabil. Konfigurasi ambang per fitur dideklarasikan pada *ConfigMap drift-thresholds* sehingga perubahan ambang dapat dilakukan melalui *commit* pada Git tanpa membangun ulang citra. Hasil tiap eksekusi ditulis dengan *checksum* versi konfigurasi sehingga riwayat ambang dapat ditelusuri saat audit. Citra detektor dibangun dengan *uv* pada *Dockerfile* multi-stage agar model SBERT untuk deteksi *drift embedding* dapat di-*embed* pada lapis terakhir, dan dengan demikian *cold start* tidak memerlukan unduhan model.

V.4.2 Control Loop Pelatihan Ulang

Control loop dijalankan oleh Argo CronWorkflow dengan jadwal yang dapat dikonfigurasi per model. *Workflow* menjalankan dua langkah utama: pengukuran *drift* dan pemicu *pipeline* pelatihan. Pemicu pelatihan mengirim permintaan ke KubeFlow Pipelines dengan parameter versi data dan versi model dasar. *Pipeline* pelatihan menarik fitur dari Feast dengan *get_historical_features*, menjalankan pelatihan pada *job* dengan sumber daya yang dialokasikan oleh Kueue (Kueue Authors 2024), menulis metrik ke MLflow, dan mendaftarkan model baru ke registri MLflow. Validasi model dijalankan sebelum penyebaran dengan membandingkan metrik kandidat terhadap model produksi pada *holdout set*.

Jika lulus validasi, Argo Rollouts (Argo Rollouts Authors 2025) memperbarui *InferenceService* KServe dengan model baru memakai strategi *canary* dan analisis metrik Prometheus, sehingga pengembalian otomatis dapat dilakukan ketika kinerja menurun. *retrain-on-drift* CronWorkflow disimpan pada direktori *workflow* milik *use case* yang terpisah dari *platform/* agar dapat di-*deploy* bersama *use case* ketika pengujian dijalankan. Konfigurasi penonaktifan *cache* KFP diterapkan pada langkah *train* dan *deploy* agar setiap eksekusi memakai data terbaru tanpa dipengaruhi *cache* historis. Pengiriman metrik *workflow* dilakukan melalui *finally task* pada Argo CronWorkflow yang menulis ke Pushgateway, sehingga seluruh metrik eksekusi tetap terkumpul pada Prometheus meskipun *workflow* berdurasi singkat.

V.4.3 Penanganan Kasus Khusus

Tiga kasus khusus ditangani pada implementasi. Pertama, jendela tunggu label diatur per model agar pelatihan ulang tidak dipicu sebelum label tersedia cukup. Kedua, ambang *drift* dihitung dengan jendela bergulir untuk meredam fluktuasi singkat. Ketiga, ketika validasi gagal, KServe menahan versi lama dan *workflow* mencatat kejadian ke MLflow sebagai *run* yang gagal divalidasi.

Pengelolaan ketiga kasus tersebut dilakukan pada cabang DAG Argo CronWorkflow dengan *when* dan *exitHandler* yang memetakan keputusan ke catatan terstruktur. Setiap cabang menulis tag ke MLflow yang menyatakan alasan keputusan dan rujukan ke eksekusi pengukur *drift* yang memicunya. Kegagalan validasi memicu peringatan ke Alertmanager dengan rute khusus untuk peran *machine learning engineer*, sehingga investigasi dapat dimulai tanpa menunggu pelaporan manual. Ambang dan jendela penyesuaian dikelola melalui ConfigMap *retrain-policy* yang dijaga oleh Argo CD agar perubahan kebijakan tetap dapat di-*audit*.

V.5 Implementasi Sub-sistem Layanan Fitur *Dual-Store*

Bagian ini mengimplementasikan rancangan pada Bagian IV.5. Penekanan implementasi diberikan pada satu sumber definisi fitur, satu kontrak Feast, dan dua *store online* yang berperan berbeda sesuai jenis fitur. Pemisahan beban kerja *store* tabular dan *store* vektor dilakukan pada level *provider* Feast sehingga aplikasi pemanggil tetap memakai satu antarmuka. Tiap sub-sub-bagian menjelaskan satu lapis implementasi: konfigurasi Feast, dukungan *vector embedding*, dan jaminan *point-in-time correctness*.

V.5.1 Konfigurasi Feast

Feast (Feast Project 2024) dikonfigurasi dengan *feature_store.yaml* yang menentukan *provider* Kubernetes, *offline store* ClickHouse, *online store* Valkey (Valkey Contributors 2025) melalui protokol RESP, dan registri *file-based* pada MinIO. Definisi fitur ditulis dalam Python pada modul *feature_definitions.py* dan didaftarkan ke registri melalui *feast apply*. Materialisasi fitur ke *online store* dijalankan oleh *job* berkala yang membaca dari ClickHouse dengan rentang waktu yang sesuai. Jain dan Das (2025) memperlihatkan pentingnya integrasi DataOps pada saat materialisasi seperti ini agar konsistensi antar lapisan tidak bergeser.

Registri Feast disimpan sebagai *registry.db* di *bucket* MinIO dengan kunci yang dirotasi melalui ESO sehingga klien Feast pada berbagai *namespace* berbagi sumber

kebenaran yang sama. *Service feast serve* dipasang sebagai *Deployment* yang dijangkau melalui *Service feast-server* pada *namespace* *model-lifecycle*. Pembaruan definisi fitur dilakukan oleh *service* materialisasi yang menjalankan *feast apply* sebagai langkah pada DAG *data_pipeline* Airflow, sehingga registri selalu sinkron dengan definisi pada *feature_repo/*. *Client Feast* pada layanan pemanggil dikonfigurasi dengan *registry_path* URI *s3://*, sehingga sumber kebenaran tunggal tetap terjaga ketika klien berjalan pada *namespace* berbeda.

V.5.2 Dukungan *Vector Embeddings*

Vector embedding disimpan sebagai fitur dengan tipe *Array(Float32)* pada *ClickHouse* dan sebagai *point* berdimensi 768 pada *collection* *Qdrant* (*Qdrant Team 2025*). Pengindeksan *HNSW* diaktifkan pada *Qdrant* dengan parameter *m=16* dan *ef_construct=200* yang dapat dikonfigurasi melalui *HnswConfig*, dan kueri tetangga terdekat dilayani melalui *API gRPC* dengan filter *payload* pada saat pencarian. *Service vector-embedding* pada *platform/services/processing/vector/* membungkus model *sentence-transformers/all-mpnet-base-v2* sebagai penghasil *embedding* dan menyajikan *API* yang dipanggil oleh *stream processor* pada saat *event* masuk (*Malkov dan Yashunin 2020; Devlin dkk. 2019*).

Citra *vector-embedding* dibangun dengan *uv* dan model *SBERT* di-bake ke lapis *COPY* terakhir agar *startup probe* tidak terganggu oleh unduhan model pada pemicu pertama. *Collection Qdrant* dideklarasikan oleh *init Job* yang mengirim *PUT* ke */collections/nama-koleksi* dengan parameter *HNSW* sesuai konfigurasi. *Snapshot* koleksi disimpan ke *MinIO* secara berkala dengan *Job qdrant-snapshot* yang menambah objek pencadangan ke jadwal *Velero*. Klien *Feast* memakai *vector_store provider Qdrant* untuk operasi *get_online_documents* sehingga kueri vektor tetap dipanggil melalui antarmuka *Feast* yang konsisten.

V.5.3 Materialisasi dan *Point-in-Time Correctness*

get_historical_features dijalankan pada *pipeline* pelatihan dengan kolom *event_timestamp* pada *entity dataframe*. *Feast* melakukan *point-in-time join* pada *ClickHouse* menggunakan kueri dengan klausa *ASOF JOIN*, sehingga nilai fitur yang dipakai pada saat pelatihan identik dengan nilai yang tersedia pada saat itu (*Vela dkk. 2022; Wang dan Ruf 2021*). Materialisasi ke *online store* berjalan secara berkala dengan *feast materialize-incremental*, dan *stream materializer* pada *Flink* menulis fitur *real-time* ke *Valkey* untuk fitur tabular dan ke *Qdrant* untuk *embedding* secara bersamaan dengan penulisan ke *ClickHouse*.

`materialize-incremental` dijalankan setiap jam untuk fitur agregasi dan setiap hari untuk fitur *rolling window* berdurasi panjang. Konsistensi antar *store* divalidasi oleh `feature-store-consistency-check CronJob` yang membandingkan jumlah baris pada Valkey dan ClickHouse untuk subset *entity* terpilih. Selisih yang melampaui ambang memicu peringatan ke Alertmanager dan menulis *event* ke DataHub sebagai *assertion failure*. Pengujian *point-in-time* dijalankan sebagai skenario SK-F-03 yang mengaudit `get_historical_features` pada *dataset* pelatihan, memastikan kontrak `event_timestamp` dan `ttl` tetap dipatuhi oleh `FeatureView` yang ditambahkan.

V.6 Verifikasi Implementasi

Verifikasi jalur eksekusi dilakukan dengan satu *use case* domain nyata sebagai instans pengujian, yang struktur layanan, konfigurasi, dan datanya dirinci pada Bab VI. Pemilihan ini tidak mengubah sifat domain-agnostik dari platform, karena seluruh kode dan konfigurasi platform tetap berada pada repositori `platform/`, sedangkan kode dan konfigurasi spesifik domain tinggal pada direktori *use case* tersendiri yang terpisah dari platform. Pemisahan direktori ini memastikan penambahan atau penggantian *use case* tidak menyentuh berkas platform, sehingga lapis kendali tetap dapat dipakai ulang lintas domain. *Manifest* Kubernetes domain mengikuti pola *base* dan *overlays Kustomize* sehingga parameter spesifik lingkungan cukup disetel pada lapis *overlay* tanpa menyentuh definisi dasar.

V.6.1 Lingkup Verifikasi

Verifikasi mencakup lima jalur. Pertama, jalur ingestasi diuji dengan *stream* dari sumber eksternal yang masuk ke Kafka melalui layanan `websocket-collector` pada *use case*. Kedua, jalur pemrosesan diuji dengan `stream-processor` berbasis Flink yang menulis fitur agregasi ke ClickHouse dan Valkey, serta *embedding* ke Qdrant. Ketiga, jalur pelatihan diuji dengan *pipeline* Kubeflow yang membaca fitur dari Feast dan menulis model ke MLflow. Keempat, jalur penyajian diuji dengan *InferenceService* KServe yang memuat model dari MLflow, dengan promosi versi yang dikelola oleh Argo Rollouts. Kelima, jalur pelatihan ulang diuji dengan menjalankan *drift-detector* dan memicu *retrain-on-drift workflow*. Uji ketahanan pendek menggunakan Chaos Mesh (Chaos Mesh Authors 2025) untuk menyuntikkan *fault* pada *pod* terpilih, dan pencadangan dijalankan terjadwal oleh Velero (Velero Authors 2025) ke MinIO sebagai langkah disiplin pemulihan.

Lima jalur tersebut dipilih agar setiap sub-sistem mendapat verifikasi minimal satu

skenario nyata, sehingga ketidaktepatan rancangan dapat ditemukan sebelum penilaian akhir pada Bab VI. Skenario *use case* menetapkan beban yang lebih tinggi pada jalur ingestasi dan pemrosesan dibandingkan jalur pelatihan, karena *use case* *throughput* tinggi memproduksi banyak peristiwa per detik dari beberapa entitas secara paralel. Pengamatan dilakukan dengan dasbor Grafana yang menampilkan metrik latensi, *throughput*, panjang antrean Kafka, dan tingkat keberhasilan *materialization*. Hasil pengamatan dibandingkan dengan target metrik pada KNF Bab III untuk menentukan apakah setiap jalur memenuhi target yang telah ditetapkan.

V.6.2 Pengamatan Hasil Awal

Hasil pengamatan awal memperlihatkan bahwa jalur kendali platform terbentuk sesuai rancangan. Seluruh komponen pada kelima jalur berhasil di-*deploy* dan direkonsiliasi oleh Argo CD, *control loop* pelatihan ulang terpicu otomatis ketika *drift* buatan disuntikkan pada salah satu fitur, dan promosi model dikendalikan oleh Argo Rollouts sesuai konfigurasi kanari. Reprodusibilitas jalur diuji dengan merekonstruksi seluruh komponen platform dari Argo CD pada *cluster* k3s yang baru, dan rekonstruksi tersebut berhasil dilakukan tanpa intervensi manual selain *bootstrap* Argo CD awal. Penilaian kebenaran data *end-to-end*, validitas *point-in-time* pada pelatihan, latensi penyajian, *throughput* pemrosesan, dan ketepatan deteksi *drift* dilakukan secara kuantitatif pada Bab VI setelah seluruh tahap pengujian beban selesai.

Pengamatan tambahan menunjukkan bahwa jalur observabilitas berhasil mempertahankan metrik dan log dari seluruh komponen pada satu antarmuka Grafana, dan bahwa peristiwa *lineage* dari emitter OpenLineage Airflow dan Spark serta konektor *ingestion* DataHub untuk Feast, MLflow, ClickHouse, dan Kafka muncul pada satu graf metadata dengan kosa kata yang seragam. Pencadangan Velero berjalan terjadwal tanpa intervensi, dan pengujian pemulihan dari pencadangan memperlihatkan bahwa konfigurasi seluruh komponen dapat direkonstruksi tanpa langkah manual. Uji *fault injection* Chaos Mesh menunjukkan bahwa *ScaledObject* KEDA memulihkan beban kerja yang dimatikan secara paksa, dan bahwa *Argo Rollouts* otomatis menahan promosi ketika metrik kanari menyentuh ambang. Catatan pengamatan ini menjadi dasar untuk evaluasi kuantitatif pada Bab VI yang akan menilai pemenuhan KF dan KNF terhadap target metrik.

BAB VI

EVALUASI

Bab ini memaparkan evaluasi platform terhadap kebutuhan fungsional KF-01 sampai KF-14 dan kebutuhan nonfungsional KNF-01 sampai KNF-12 yang disusun pada Bab III. Evaluasi disusun mengikuti empat tujuan platform T-1 sampai T-4 sehingga setiap hasil dapat dipetakan kembali pada tujuan, rumusan masalah, dan butir kesimpulan dengan indeks yang sama. Pengujian dilakukan pada satu *use case* domain nyata sebagai instans verifikasi, yaitu pasar aset kripto dengan data Coinbase, agar perilaku platform dapat diamati pada beban dan pola data yang realistis tanpa mengubah sifat *domain-agnostic* platform. Penyajian dibagi menjadi metode evaluasi, hasil evaluasi, dan pembahasan, dengan angka kuantitatif beban penuh dilaporkan menyusul sesuai batasan B-5 pada Subbab I.4.

VI.1 Metode Evaluasi

Metode evaluasi mengikuti tiga dimensi yang melekat pada empat tujuan platform, yaitu pemenuhan fungsional, pemenuhan nonfungsional, dan tata kelola. Pemenuhan fungsional menilai apakah setiap KF terbukti melalui skenario uji yang dapat dijalankan ulang, pemenuhan nonfungsional menilai apakah setiap KNF mencapai target metrik yang ditetapkan, dan tata kelola menilai apakah *lineage* tertelusur, kontrak data ditegakkan, serta kebijakan akses dapat diaudit. Setiap skenario diberi penanda SK-F untuk kebutuhan fungsional dan SK-N untuk kebutuhan nonfungsional, lalu dikelompokkan di bawah tujuan T-1 sampai T-4 yang relevan. Penomoran tersebut menjaga keterhubungan satu lawan satu antara skenario, kebutuhan, dan tujuan sehingga kegagalan satu skenario dapat segera dirunut ke komponen yang bertanggung jawab.

Skenario dijalankan secara manual pada tahap evaluasi dan tidak menjadi bagian dari `make phase-full` maupun rekonsiliasi Argo CD, agar pengukuran tidak mengganggu jalur produksi platform. Pembangkit beban dijalankan dari `namespace platform-test` yang terpisah, sedangkan instrumen pengukuran memanfaatkan komponen observa-

bilitas yang sudah terpasang. Instrumen utama mencakup k6 untuk beban *HTTP* dan *gRPC*, Chaos Mesh untuk injeksi *fault*, Great Expectations untuk kontrak kualitas, serta Prometheus, Grafana, Grafana Tempo, dan OpenCost untuk metrik, *trace*, dan biaya. Berkas skenario disimpan pada direktori `experiments/` yang sejajar dengan `platform/` dan `use-case-crypto/` sebagai lapis verifikasi terpisah, dengan subdirektori `load/`, `chaos/`, dan `drift/`, sehingga prosedur uji dapat ditelusuri ulang bersama kode tanpa mengubah sistem yang diuji.

VI.1.1 Instans Verifikasi dan Lingkungan Uji

Instans verifikasi adalah *use case* pasar aset kripto yang mengalirkan data Coinbase secara *real-time* dengan granularitas per detik sejak 1 Januari 2026. Pemilihan domain ini didasari karakteristiknya yang menuntut *throughput* ingestasi tinggi, latensi penyajian rendah, dan distribusi data nonstasioner, sehingga keempat *sub-sistem* platform tertekan pada kondisi yang menantang. Seluruh kode dan konfigurasi spesifik domain berada pada direktori `use-case-crypto/` yang terpisah dari `platform/`, mencakup layanan `websocket-collector` sebagai produsen *event*, `stream-processor` berbasis Flink, lapis *batch* berbasis dbt, definisi Feast, serta jembatan *inference* dan dasbor. Pemisahan ini menegaskan bahwa platform tetap *domain-agnostic* dan *use case* hanya berperan sebagai muatan uji yang dapat diganti tanpa menyentuh lapis kendali.

Konfigurasi spesifik domain diterapkan melalui *overlay* Kustomize yang menambahkan prefiks `crypto-` pada sumber daya basis platform, sehingga *InferenceService* basis bernama *predictor* menjadi *crypto-predictor* ketika di-*deploy*. Pola *base* dan *overlay* ini membuat parameter sumber data dan registri citra cukup disetel pada lapis *overlay* tanpa mengubah definisi dasar, sebagaimana ditunjukkan pada Listing VI.1. Dengan demikian, penggantian *use case* cukup dilakukan dengan menyiapkan *overlay* baru dan tidak menuntut perubahan pada manifes platform. Pendekatan ini sekaligus menjadi bukti konkret atas pemenuhan KF-12 tentang konfigurasi deklaratif dan KNF-11 tentang portabilitas.

Listing VI.1 Ilustrasi overlay Kustomize use case yang menambahkan prefiks crypto- pada sumber daya basis platform

```
1 # use-case-crypto/manifests/overlays/local/kustomization.yaml
2 namePrefix: crypto-
3 namespace: use-case-crypto
4 resources:
5   - ../../base
6 configMapGenerator:
```

```

7   - name: collector-config
8     literals:
9       - SOURCE=coinbase
10      - GRANULARITY=1s
11      - START=2026-01-01T00:00:00Z
12 images:
13   - name: predictor
14     newName: localhost:5000/predictor

```

VI.1.2 Evaluasi Tujuan T-1: Arsitektur Terintegrasi dan GitOps

Evaluasi T-1 menilai apakah seluruh platform dapat dipasang dari satu sumber Git, dikelola secara deklaratif, dan menyediakan otomatisasi siklus hidup model setara MLOps Level 2. Skenario SK-F-01 mengubah definisi *feature view* pada Gitea lalu mengamati rekonsiliasi Argo CD untuk membuktikan KF-01, sementara SK-F-12 mengubah jumlah *replica* pada manifes untuk membuktikan konfigurasi deklaratif KF-12 yang konvergen tanpa *drift*. Skenario SK-F-06 mempromosikan model melalui Argo Rollouts dengan strategi kanari dan memicu *rollback* pada model bermasalah untuk membuktikan otomatisasi penerapan KF-06, sedangkan SK-F-13 menjalankan satu *notebook* yang memanggil SDK Feast, MLflow, dan klien KServe untuk membuktikan antarmuka terpadu KF-13. Skenario SK-F-14 menjenuhkan kuota *ClusterQueue* Kueue untuk membuktikan manajemen sumber daya KF-14, dengan *job* di luar anggaran berstatus *Pending*.

Kematangan MLOps Level 2 dinilai melalui empat pilar yang saling terhubung pada satu jalur otomatisasi. Pilar pertama adalah *pipeline* pelatihan otomatis pada Kubeflow Pipelines yang dipicu data baru atau ambang *drift*, pilar kedua adalah penerapan otomatisasi melalui KServe dan Argo Rollouts, pilar ketiga adalah pemantauan berkelanjutan oleh Argo Workflows yang melaporkan metrik ke Prometheus, dan pilar keempat adalah pengelolaan metadata terpusat pada DataHub. Reprodusibilitas penyebaran diuji oleh SK-N-11 yang menerapkan ulang seluruh manifes pada *node* k3s baru dan SK-N-12 yang mengukur cakupan uji serta waktu konvergensi *make phase-full*. Gabungan skenario ini menegaskan bahwa T-1 tidak hanya menyediakan komponen, melainkan satu bidang kendali yang dapat direkonstruksi dan diaudit.

VI.1.3 Evaluasi Tujuan T-2: Tata Kelola Data

Evaluasi T-2 menilai apakah katalog metadata, *lineage*, dan kontrol kualitas berjalan otomatis sepanjang *pipeline* data, fitur, dan model. Skenario SK-F-08 me-

nelusuri *lineage* pada DataHub dari topik sumber sampai keluaran prediksi untuk membuktikan KF-08, dengan peristiwa *lineage* dikirim melalui OpenLineage dan *ingestion* terjadwal. Skenario SK-F-09 menjalankan dua *run* pelatihan pada MLflow lalu memutar ulang dari *snapshot* untuk membuktikan pelacakan eksperimen KF-09, sedangkan SK-F-10 memindahkan model melalui status *staging*, *production*, dan *archived* untuk membuktikan registri dan *audit trail* KF-10. Skenario SK-N-05 menjalankan *checkpoint* Great Expectations dengan probe *timestamp* masa depan untuk membuktikan konsistensi dan kebenaran KNF-05 pada sisi kontrak data.

Dimensi keamanan dan observabilitas tata kelola dievaluasi melalui dua skenario tambahan. Skenario SK-N-06 menelusuri satu *trace* OpenTelemetry pada Grafana Tempo dari ingestasi sampai prediksi untuk membuktikan observabilitas KNF-06, sehingga metrik, log, dan *trace* dapat dirujuk silang pada satu antarmuka. Skenario SK-N-07 memindai *cluster* dengan Trivy dan memeriksa konfigurasi TLS untuk membuktikan keamanan KNF-07, mencakup mTLS Istio, enkripsi *at rest* MinIO melalui KES dan OpenBao, serta kebijakan admisi Kyverno. Ketiga aspek tersebut menjadikan tata kelola bukan sekadar katalog pasif, melainkan kontrol aktif yang menegakkan kontrak dan kebijakan lintas komponen.

VI.1.4 Evaluasi Tujuan T-3: Deteksi *Drift* dan Pelatihan Ulang

Evaluasi T-3 menilai apakah deteksi *drift* terotomasi memicu pelatihan ulang sesuai kebijakan dan apakah penyajian fitur menjamin *point-in-time correctness*. Skenario SK-F-07 menyuntikkan *drift* sintesis pada satu fitur melalui `inject_drift.py`, lalu mengamati detektor *drift* dan *CronWorkflow* `retrain-on-drift` untuk membuktikan pemantauan KF-07. Detektor membandingkan distribusi produksi terhadap distribusi pelatihan dengan *Population Stability Index* dan uji Kolmogorov-Smirnov, lalu memicu *pipeline* pelatihan ketika ambang terlampaui. Skenario SK-F-05 menjalankan *pipeline* pelatihan pada Kubeflow Pipelines yang membaca fitur dari Feast dan menulis model ke MLflow untuk membuktikan otomasi pelatihan KF-05.

Validitas temporal diuji secara khusus karena menjadi pembeda utama dari pendekatan tradisional. Skenario SK-F-03 mengaudit `get_historical_features` pada *dataset* pelatihan untuk membuktikan KF-03, dengan memastikan tidak ada baris yang memakai informasi setelah *event timestamp* acuan. Pengujian ini memakai probe *timestamp* masa depan yang sama dengan SK-N-05 sehingga jaminan *zero leakage* ditegakkan pada jalur pelatihan maupun pada kontrak kualitas. Gabungan skenario ini menegaskan bahwa T-3 menutup dua risiko sekaligus, yaitu degradasi model akibat *drift* dan kebocoran temporal akibat penyusunan fitur yang keliru.

VI.1.5 Evaluasi Tujuan T-4: Layanan Fitur *Dual-Store*

Evaluasi T-4 menilai apakah layanan fitur *dual-store* menyajikan fitur tabular, fitur agregat, dan fitur vektor dengan kontrak yang konsisten antara pelatihan dan penyajian. Skenario SK-F-02 membandingkan nilai fitur *online* pada Valkey dan *offline* pada ClickHouse untuk membuktikan penyajian ganda KF-02, sedangkan SK-F-04 menjalankan beban pencarian vektor pada Qdrant melalui `load/vector-search-latency.js` untuk membuktikan dukungan *vector embeddings* KF-04. Skenario SK-F-11 membandingkan fitur hasil *batch* dan hasil *stream* pada indikator yang sama untuk membuktikan pemrosesan ganda KF-11 dengan definisi fitur yang konsisten. Ketiga skenario tersebut menelusuri kontrak fitur dari sisi kebenaran nilai sebelum beban tinggi diterapkan.

Kinerja dan ketahanan layanan fitur diuji oleh kelompok skenario nonfungsional. Skenario SK-N-01 mengukur latensi *feature serving* dan *vector search* dengan k6 dan SLO Sloth untuk KNF-01, SK-N-02 mengukur *throughput serving* dan *stream* untuk KNF-02, serta SK-N-03 menaikkan beban dengan `load/hpa-keda-rampup.js` untuk membuktikan skalabilitas horizontal KNF-03. Skenario SK-N-04 menyuntikkan *fault* dengan Chaos Mesh untuk membuktikan keandalan KNF-04, SK-N-08 mengganti satu komponen melalui antarmuka untuk membuktikan ekstensibilitas KNF-08, SK-N-09 mengukur kemudahan penggunaan melalui sesi identitas tunggal dex untuk KNF-09, dan SK-N-10 mengukur efisiensi sumber daya melalui OpenCost untuk KNF-10. Cakupan ini menempatkan T-4 sebagai tujuan dengan beban pengujian terbanyak karena lapis fitur menjadi titik temu seluruh jalur data.

VI.1.6 Matriks Cakupan dan Uji Penerimaan

Pemetaan antara skenario dan kebutuhan bersifat banyak-ke-banyak, sebab satu skenario kerap menegaskan lebih dari satu kebutuhan dan satu kebutuhan dapat diperkuat oleh beberapa skenario. Sebagai contoh, SK-F-04 yang menguji latensi pencarian vektor sekaligus menegaskan KNF-01, sedangkan SK-N-05 yang menguji kontrak kualitas turut memperkuat KF-03 pada sisi validitas temporal. Tabel VI.1 merangkum prosedur uji dan kriteria penerimaan untuk setiap skenario yang dikelompokkan di bawah tujuan T-1 sampai T-4. Susunan tersebut memastikan seluruh empat belas kebutuhan fungsional dan dua belas kebutuhan nonfungsional memiliki sekurang-kurangnya satu skenario uji yang konkret.

Tabel VI.1 Skenario Uji Penerimaan per Tujuan Platform

Skenario	Kebutuhan	Prosedur Uji	Kriteria Penerimaan
Tujuan T-1: arsitektur platform terintegrasi dan praktik GitOps			
SK-F-01	KF-01	Mengubah definisi <i>feature view</i> pada repositori Gitea lalu menunggu rekonsiliasi Argo CD, kemudian memverifikasi <i>feast feature-views list</i> .	<i>Feature view</i> baru aktif tanpa intervensi manual dan perubahan terlacak pada riwayat Git.
SK-F-06	KF-06	Mempromosikan model melalui Argo Rollouts dengan strategi kanari, lalu menyuntikkan model bermasalah untuk memicu <i>rollback</i> .	Model valid dipromosikan; model bermasalah memicu <i>rollback</i> otomatis tanpa kehilangan permintaan.
SK-F-12	KF-12	Mengubah jumlah <i>replica</i> pada manifes YAML di Gitea, lalu mengamati rekonsiliasi Argo CD.	Status <i>cluster</i> konvergen ke definisi Git dan tidak menyisakan <i>drift</i> .
SK-F-13	KF-13	Menjalankan satu <i>notebook</i> UV yang memanggil SDK Feast, MLflow, dan klien KServe v2 secara berurutan.	Ketiga SDK dipanggil dari satu sesi tanpa konfigurasi tambahan per layanan.
SK-F-14	KF-14	Mengirim sejumlah <i>job</i> pelatihan melebihi kuota <i>ClusterQueue</i> Kueue, lalu mengamati status admisi.	<i>Job</i> di luar anggaran kuota bers-tatus <i>Pending</i> ; admisi mengikuti <i>nominalQuota</i> dan <i>borrowingLimit</i> .
SK-N-11	KNF-11	Menerapkan ulang seluruh manifes GitOps pada <i>node k3s</i> baru dari satu sumber Git.	Seluruh komponen terekonstruksi tanpa intervensi manual selain <i>bootstrap</i> Argo CD.
SK-N-12	KNF-12	Menjalankan <i>make test</i> untuk cakupan uji dan <i>make phase-full</i> pada <i>cluster</i> bersih sambil mengukur waktu konvergensi.	Cakupan uji melewati ambang yang ditetapkan dan <i>phase-full</i> konvergen tanpa langkah manual.
Tujuan T-2: sub-sistem tata kelola data			
SK-F-08	KF-08	Menelusuri <i>lineage</i> pada DataHub dari topik sumber hingga keluaran prediksi melalui antarmuka katalog.	Rantai <i>lineage</i> utuh dari sumber data sampai prediksi tersaji pada satu graf metadata.
SK-F-09	KF-09	Menjalankan dua <i>run</i> pelatihan pada MLflow, lalu memutar ulang <i>run</i> dari <i>snapshot</i> data dan parameter tercatat.	<i>Run</i> tercatat lengkap dan pemutaran ulang menghasilkan selisih metrik di bawah satu persen.
SK-F-10	KF-10	Memindahkan model melalui status <i>none</i> , <i>staging</i> , <i>production</i> , dan <i>archived</i> pada registri MLflow.	Transisi mengikuti aturan yang ditetapkan dan <i>audit trail</i> terekam lengkap.
SK-N-05	KNF-05	Menjalankan <i>checkpoint</i> Great Expectations dan menyuntikkan probe <i>timestamp</i> masa depan untuk menguji kebocoran temporal.	Pelanggaran kontrak kualitas terdeteksi dan tidak ada kebocoran data temporal yang lolos.
SK-N-06	KNF-06	Menelusuri satu <i>trace</i> OpenTelemetry pada Grafana Tempo dari ingestasi hingga prediksi.	Satu <i>trace</i> menautkan lintasan kritis dan metrik, log, serta <i>trace</i> dapat dirujuk silang.

Skenario	Kebutuhan	Prosedur Uji	Kriteria Penerimaan
SK-N-07	KNF-07	Memindai <i>cluster</i> dengan Trivy dan memeriksa konfigurasi TLS pada <i>endpoint</i> layanan.	TLS 1.3 <i>in transit</i> dan AES-256 <i>at rest</i> terverifikasi tanpa kerentanan kritis terbuka.
Tujuan T-3: deteksi <i>drift</i> dan pelatihan ulang			
SK-F-03	KF-03	Mengaudit <code>get_historical_features</code> pada <i>dataset</i> pelatihan untuk memastikan jendela <i>point-in-time</i> dipatuhi.	Tidak ada baris yang memakai informasi setelah <i>event timestamp</i> acuan.
SK-F-05	KF-05	Menjalankan <i>pipeline</i> pelatihan pada Kubeflow Pipelines yang membaca fitur dari Feast dan menulis model ke MLflow.	<i>Pipeline</i> berjalan dari ujung ke ujung dan model terdaftar pada registri MLflow.
SK-F-07	KF-07	Menyuntikkan <i>drift</i> sintetis pada satu fitur, lalu mengamati detektor <i>drift</i> dan <i>CronWorkflow</i> <i>retrain-on-drift</i> .	<i>Drift</i> terdeteksi saat ambang PSI atau KS terlampaui dan <i>pipeline</i> pelatihan ulang terpicu otomatis.
Tujuan T-4: layanan fitur <i>dual-store</i>			
SK-F-02	KF-02	Membandingkan nilai fitur <i>online</i> (Valkey) dan <i>offline</i> (ClickHouse) untuk <i>entity</i> yang sama.	Nilai <i>online</i> dan <i>offline</i> konsisten untuk <i>entity</i> dan waktu acuan yang sama.
SK-F-04	KF-04	Menjalankan beban pencarian vektor pada Qdrant melalui <code>load/vector-search-latency.js</code> dengan k6.	<i>Recall</i> memenuhi target dan latensi pencarian berada pada orde sub-detik.
SK-F-11	KF-11	Membandingkan fitur hasil <i>batch</i> (dbt) dan hasil <i>stream</i> (Flink) untuk indikator yang sama.	Nilai fitur <i>batch</i> dan <i>stream</i> setara dalam toleransi yang ditetapkan.
SK-N-01	KNF-01	Menjalankan <code>load/feast-online-latency.js</code> dan beban pencarian vektor dengan k6 sambil memantau SLO Sloth.	Latensi p99 berada pada orde sub-detik pada <i>feature serving</i> dan <i>vector search</i> .
SK-N-02	KNF-02	Menjalankan <code>load/feast-throughput.js</code> untuk <i>serving</i> dan <code>kafka-producer-perf-test</code> untuk <i>stream</i> , sambil memantau <i>lag</i> .	<i>Throughput</i> mencapai target dan <i>lag</i> konsumen tetap terkendali pada beban puncak.
SK-N-03	KNF-03	Menaikkan beban dengan <code>load/hpa-keda-rampup.js</code> sambil mengamati HPA, KEDA <i>ScaledObject</i> , dan jumlah <i>pod</i> .	Jumlah <i>replica</i> naik mengikuti beban dan <i>throughput</i> bertambah mendekati linear.
SK-N-04	KNF-04	Menyuntikkan <i>fault</i> dengan Chaos Mesh (<code>pod-chaos</code> dan <code>network-chaos</code>) lalu mengamati pemulihan.	Beban kerja pulih otomatis dengan RTO dan RPO di bawah ambang yang ditetapkan.
SK-N-08	KNF-08	Mengganti satu komponen (misalnya <i>runtime</i> penyajian) melalui antarmuka CRD dan <i>overlay</i> Kustomize.	Penggantian terisolasi pada antarmuka tanpa mengubah komponen lain.

Skenario	Kebutuhan	Prosedur Uji	Kriteria Penerimaan
SK-N-09	KNF-09	Mengakses konsol platform melalui satu sesi identitas dex dan mengumpulkan survei kemudahan penggunaan.	Seluruh konsol terhubung pada satu jalur autentikasi dan skor kemudahan melewati ambang survei.
SK-N-10	KNF-10	Mengukur rasio kompresi ClickHouse, utilisasi sumber daya, dan atribusi biaya <i>workload</i> pada OpenCost.	Rasio kompresi dan utilisasi memenuhi target serta biaya per <i>workload</i> dapat diatribusikan.

VI.2 Hasil Evaluasi

Hasil evaluasi dilaporkan dalam dua tingkat, yaitu hasil yang telah terverifikasi secara struktural pada lingkungan node tunggal dan hasil kuantitatif beban yang dilaporkan menyusul. Verifikasi struktural memeriksa apakah jalur kendali terbentuk sesuai rancangan, mencakup penyebaran dan rekonsiliasi seluruh komponen, pemuncuan otomatis pelatihan ulang, pengendalian promosi model, serta rekonstruksi penuh dari Git. Pengukuran kuantitatif beban penuh, seperti latensi p99, *throughput* puncak, dan ketepatan deteksi *drift*, ditunda sesuai batasan B-5 karena menuntut periode pengamatan beban yang stabil pada *use case*. Tabel VI.2 merangkum target, instrumen ukur, dan status setiap kebutuhan agar pembaca dapat membedakan capaian yang sudah terverifikasi dari yang masih menunggu pengukuran.

Pada T-1, verifikasi struktural menunjukkan seluruh komponen berhasil dipasang dan direkonsiliasi oleh Argo CD dari satu repositori Gitea, perubahan *replica* dan definisi fitur konvergen tanpa *drift*, dan rekonstruksi pada *node* k3s baru berhasil tanpa intervensi manual selain *bootstrap* awal. Pada T-2, peristiwa *lineage* dari emitter OpenLineage Airflow dan Spark serta konektor *ingestion* DataHub untuk Feast, MLflow, ClickHouse, dan Kafka muncul pada satu graf metadata dengan kosa kata seragam, transisi status model tercatat pada MLflow, dan kontrol keamanan mTLS serta enkripsi *at rest* aktif sesuai konfigurasi. Pada T-3, *control loop* pelatihan ulang terpicu otomatis ketika *drift* sintetis disuntikkan, sedangkan ketepatan deteksi pada beban data penuh diukur menyusul. Pada T-4, jalur penyajian fitur *online*, *offline*, dan vektor berhasil dibentuk, sementara angka latensi dan *throughput* pada beban tinggi dilaporkan setelah pengujian beban selesai.

Tabel VI.2 Target Metrik, Instrumen Ukur, dan Status Evaluasi per Kebutuhan

Kebutuhan	Target atau Kriteria	Instrumen Ukur	Status
Kebutuhan Fungsional			

Kebutuhan	Target atau Kriteria	Instrumen Ukur	Status
KF-01	<i>Feature view</i> terversion dan rekonsiliasi otomatis	Feast <i>registry</i> , Argo CD, Gitea	Terverifikasi struktural
KF-02	Konsistensi nilai <i>online</i> dan <i>offline</i>	Feast SDK, k6	Menunggu pengukuran beban
KF-03	<i>Zero leakage</i> pada <i>point-in-time join</i>	Feast PIT, probe <i>leakage</i>	Menunggu pengukuran beban
KF-04	<i>Recall</i> memenuhi target, latensi sub-detik	Qdrant, k6	Menunggu pengukuran beban
KF-05	<i>Pipeline end-to-end</i> , model terdaftar	Kubeflow Pipelines, MLflow	Terverifikasi struktural
KF-06	Promosi kanari dan <i>rollback</i> otomatis	KServe, Argo Rollouts, Prometheus	Terverifikasi struktural
KF-07	<i>Drift</i> memicu pelatihan ulang	<i>drift-detector</i> , Argo CronWorkflow	Struktural; ketepatan menyusul
KF-08	<i>Lineage</i> data ke fitur ke model utuh	DataHub, OpenLineage	Terverifikasi struktural
KF-09	<i>Run</i> reproducible, selisih ulang kecil	MLflow	Menunggu pengukuran beban
KF-10	Transisi status dan <i>audit trail</i>	MLflow Model Registry	Terverifikasi struktural
KF-11	Paritas fitur <i>batch</i> dan <i>stream</i>	dbt, Flink	Menunggu pengukuran beban
KF-12	Konvergensi deklaratif tanpa <i>drift</i>	Kustomize, Argo CD, Gitea	Terverifikasi struktural
KF-13	Satu sesi memanggil seluruh SDK	SDK Feast, MLflow, KServe	Terverifikasi struktural
KF-14	<i>Job</i> luar kuota <i>Pending</i> , admisi sesuai	Kueue <i>ClusterQueue</i> , HPA, VPA, KEDA	Terverifikasi struktural
Kebutuhan Nonfungsional			
KNF-01	Latensi p99 sub-detik <i>serving</i> dan vektor	k6, SLO Sloth	Menunggu pengukuran beban
KNF-02	<i>Throughput serving</i> dan <i>stream</i> tinggi	k6, kafka-producer-perf-test, KEDA	Menunggu pengukuran beban
KNF-03	<i>Replica</i> dan <i>throughput</i> naik mendekati linear	HPA, KEDA, k6	Menunggu pengukuran beban
KNF-04	Pemulihan otomatis, RTO dan RPO terjaga	Chaos Mesh, Velero, Sloth	Struktural; RTO/RPO menyusul
KNF-05	<i>Zero</i> kebocoran data temporal	Great Expectations, probe	Struktural; kebenaran data menyusul
KNF-06	Metrik, log, dan <i>trace</i> silang-rujuk	Prometheus, Loki, Tempo, OTel	Terverifikasi struktural

Kebutuhan	Target atau Kriteria	Instrumen Ukur	Status
KNF-07	TLS 1.3, AES-256, dan <i>audit log</i> aktif	Istio mTLS, KES, OpenBao, Trivy	Terverifikasi struktural
KNF-08	Penggantian komponen terisolasi	CRD, Helm, Kustomize	Terverifikasi struktural
KNF-09	SSO satu sesi, skor survei lewat ambang	dex, oauth2-proxy, survei	Menunggu pengukuran survei
KNF-10	Kompresi, utilisasi, dan atribusi biaya	OpenCost, ClickHouse	Menunggu pengukuran beban
KNF-11	Rekonstruksi penuh dari Git	Argo CD, Kustomize	Terverifikasi struktural
KNF-12	Cakupan uji dan <i>phase-full</i> konvergen	Tekton CI, Argo CD	Terverifikasi struktural

VI.3 Pembahasan Hasil Evaluasi

Pembahasan mengaitkan setiap hasil dengan tujuan yang dijawab dan dengan rumusan masalah yang menjadi titik berangkat. Verifikasi struktural pada T-1 menjawab RM-1 dengan menunjukkan bahwa fragmentasi tumpukan teknologi dapat diganti oleh satu bidang kendali GitOps yang dapat direkonstruksi. Verifikasi pada T-2 menjawab RM-2 dengan menunjukkan tata kelola berjalan sebagai layanan bersama, bukan pekerjaan pasca-fakta, sehingga *lineage* dan kontrak kualitas tertanam pada *pipeline*. Verifikasi pada T-3 dan T-4 menjawab RM-3 dan RM-4 dengan menunjukkan bahwa deteksi *drift* dengan pelatihan ulang dan layanan fitur *dual-store* telah terbentuk pada jalur kendali, meskipun penilaian kuantitatif beban masih menyusul.

Cakupan evaluasi telah memenuhi syarat keterlacakan, karena seluruh empat belas kebutuhan fungsional dan dua belas kebutuhan nonfungsional memiliki sekurang-kurangnya satu skenario uji pada Tabel VI.1 dan satu baris status pada Tabel VI.2. Pemetaan banyak-ke-banyak memperkuat keyakinan karena beberapa kebutuhan kritis, seperti KNF-01 dan KNF-05, ditegaskan oleh lebih dari satu skenario. Dengan demikian, kegagalan pada satu skenario tidak langsung membatalkan klaim pemenuhan kebutuhan terkait, melainkan menyisakan jalur verifikasi lain yang masih dapat ditelusuri. Susunan ini sekaligus menyiapkan struktur kesimpulan pada Bab VII yang menjawab tepat satu tujuan untuk setiap butir.

Evaluasi ini memiliki sejumlah keterbatasan yang perlu dicatat secara terbuka. Pertama, lingkungan pengujian adalah k3s node tunggal sesuai batasan B-2, sehingga klaim skalabilitas horizontal diukur melalui *multi-replica* dan *autoscaling* pada satu

node, bukan pada kluster multi-node. Kedua, deteksi *drift* diverifikasi dengan *drift* sintetis sehingga ketepatan pada pergeseran distribusi alami masih perlu diamati lebih lama, dan angka beban penuh seperti latensi p99 dan *throughput* puncak dilaporkan menyusul sesuai batasan B-5. Ketiga, sebagian skenario bergantung pada kelengkapan jalur data *use case* sehingga penilaian kebenaran nilai fitur dilakukan setelah jalur ingestasi sampai *gold* terisi penuh. Keterbatasan ini menjadi dasar saran tindak lanjut pada Bab VII, terutama untuk pengujian beban pada lingkungan multi-node dan pengamatan *drift* pada data nyata berdurasi panjang.

BAB VII

PENUTUP

Bab ini akan diisi setelah hasil evaluasi pada Bab VI terkonsolidasi. Kerangka kesimpulan dijaga mengikuti aturan rantai kunci: setiap poin kesimpulan menjawab tepat satu tujuan T-1 sampai T-4 dengan urutan yang sama, tanpa menambahkan klaim baru di luar yang dibahas pada bab-bab sebelumnya.

VII.1 Kesimpulan

Kesimpulan akan dirangkum dalam empat poin yang bersesuaian dengan empat tujuan platform. Poin K1 akan merangkum bukti bahwa arsitektur DataOps dan MLOps terintegrasi pada Kubernetes dapat dipasang dari satu sumber Git dengan praktik GitOps. Poin K2 akan merangkum bukti bahwa katalog metadata, *lineage*, dan kontrol kualitas data berjalan otomatis sepanjang *pipeline* data, fitur, dan model. Poin K3 akan merangkum bukti bahwa deteksi *drift* terotomasi memicu pelatihan ulang sesuai kebijakan dan bahwa penyajian fitur memenuhi *point-in-time correctness*. Poin K4 akan merangkum bukti bahwa layanan fitur *dual-store* menyajikan fitur tabular, fitur agregat, dan fitur vektor dengan kontrak yang konsisten antara fase pelatihan dan fase *inference*.

VII.2 Saran

Saran akan diarahkan pada tindak lanjut untuk tujuan yang belum tertutup penuh oleh evaluasi awal serta pada arah pengembangan platform berikutnya. Beberapa arah yang telah teridentifikasi mencakup perluasan dukungan multi-*tenancy* pada lapis tata kelola, otomasi kebijakan kuota pada *Kueue* untuk beban pelatihan yang lebih besar, integrasi metrik biaya *OpenCost* dengan kebijakan *retraining*, dan adopsi profil keamanan yang lebih ketat melalui *Kyverno* setelah profil dasar matang.

DAFTAR PUSTAKA

- Adusumilli, Lakshmi Vara Prasad. 2025. "Serverless Kubernetes: The Evolution of Container Orchestration". *European Journal of Computer Science and Information Technology* 13 (30): 20–36. <https://doi.org/10.37745/ejcsit.2013/vol13n302036>. <https://doi.org/10.37745/ejcsit.2013/vol13n302036>.
- Ahmad, Tazeem, Mohd Adnan, Saima Rafi, Muhammad Azeem Akbar, dan Ayesha Anwar. June 2024. "MLOps-Enabled Security Strategies for Next-Generation Operational Technologies". Dalam *Proceedings of the 28th International Conference on Evaluation and Assessment in Software Engineering (EASE 2024)*, 662–670. <https://doi.org/10.1145/3661167.3661283>. <https://doi.org/10.1145/3661167.3661283>.
- Aiven Karapace Authors. 2025. *Karapace: Schema Registry and REST Proxy for Apache Kafka*. Diakses 25 Mei 2026. <https://www.karapace.io/>.
- Altinity. 2025. *Altinity Kubernetes Operator for ClickHouse*. Diakses 25 Mei 2026. <https://github.com/Altinity/clickhouse-operator>.
- Amazon Web Services. 2024. *What is an Event-Driven Architecture?* Diakses 15 Oktober 2025. <https://aws.amazon.com/event-driven-architecture/>.
- Amershi, Saleema, Andrew Begel, Christian Bird, Robert DeLine, Harald Gall, Ece Kamar, Nachiappan Nagappan, Besmira Nushi, dan Thomas Zimmermann. 2019. "Software Engineering for Machine Learning: A Case Study". Dalam *2019 IEEE/ACM 41st International Conference on Software Engineering: Software Engineering in Practice (ICSE-SEIP)*, 291–300. IEEE. <https://doi.org/10.1109/ICSE-SEIP.2019.00042>. <https://doi.org/10.1109/ICSE-SEIP.2019.00042>.
- Amou Najafabadi, Faezeh, Justus Bogner, Ilias Gerostathopoulos, dan Patricia Lago. 2024. "An Analysis of MLOps Architectures: A Systematic Mapping Study". Dalam *Software Architecture: 18th European Conference, ECSA 2024, Luxembourg City, Luxembourg, 3–6 September 2024, Proceedings*, 14889:69–85. Lecture Notes in Computer Science. Springer. https://doi.org/10.1007/978-3-031-70797-1_5. https://doi.org/10.1007/978-3-031-70797-1_5.

- Anaconda, Inc. July 2020. *2020 State of Data Science: Moving From Hype Toward Maturity*. Diakses 1 November 2025. <https://know.anaconda.com/rs/387-XNW-688/images/Anaconda-SODS-Report-2020-Final.pdf>.
- Apache APISIX Authors. 2025. *Apache APISIX: Cloud-Native API Gateway*. Diakses 10 November 2025. <https://apisix.apache.org/docs/>.
- Apache Flink Authors. 2025. *Flink Kubernetes Operator*. Diakses 25 Mei 2026. <https://nightlies.apache.org/flink/flink-kubernetes-operator-docs-stable/>.
- Apache Software Foundation. 2024a. *Apache Airflow Documentation*. Diakses 5 November 2025. <https://airflow.apache.org/docs/>.
- . 2024b. *Apache Flink Documentation*. Diakses 30 September 2025. <https://flink.apache.org/documentation/>.
- . 2024c. *Apache Kafka Documentation*. Diakses 15 Oktober 2025. <https://kafka.apache.org/documentation/>.
- . 2024d. *Apache Spark Documentation*. Diakses 2 November 2025. <https://spark.apache.org/docs/latest/>.
- Apache Spark Authors. 2025. *Apache Spark on Kubernetes Operator*. Diakses 25 Mei 2026. <https://github.com/kubeflow/spark-operator>.
- Apache Superset Authors. 2025. *Apache Superset Documentation*. Diakses 10 November 2025. <https://superset.apache.org/docs/intro>.
- Aqua Security. 2025. *Trivy Operator: Kubernetes Native Security Scanner*. Diakses 10 November 2025. <https://aquasecurity.github.io/trivy-operator/>.
- Argo Project. 2024. *Argo CD - Declarative GitOps CD for Kubernetes*. Diakses 8 November 2025. <https://argo-cd.readthedocs.io/>.
- . 2025. *Argo Workflows: Container-Native Workflow Engine for Kubernetes*. Diakses 25 Mei 2026. <https://argo-workflows.readthedocs.io/>.
- Argo Rollouts Authors. 2025. *Argo Rollouts: Progressive Delivery for Kubernetes*. Diakses 10 November 2025. <https://argoproj.github.io/argo-rollouts/>.
- AuthZed. 2025. *SpiceDB: Fine-Grained Authorization Database*. Diakses 25 Mei 2026. <https://authzed.com/docs/spicedb/>.

- Bayram, Firas, Bestoun S. Ahmed, dan Andreas Kassler. 2022. “From Concept Drift to Model Degradation: An Overview on Performance-Aware Drift Detectors”. *Knowledge-Based Systems* 245:108632. <https://doi.org/10.1016/j.knosys.2022.108632>. <https://doi.org/10.1016/j.knosys.2022.108632>.
- Bernardo, B. M. V., H. S. Mamede, J. M. P. Barroso, dan V. M. P. D. dos Santos. 2024. “Data Governance and Quality Management: Innovation and Breakthroughs across Different Fields”. *Journal of Innovation and Knowledge* 9 (4): 100598. <https://doi.org/10.1016/j.jik.2024.100598>. <https://doi.org/10.1016/j.jik.2024.100598>.
- Bondi, Andre B. 2000. “Characteristics of Scalability and Their Impact on Performance”. Dalam *Proceedings of the 2nd International Workshop on Software and Performance*, 195–203. ACM. <https://doi.org/10.1145/350391.350432>.
- Breck, Eric, Shanqing Cai, Eric Nielsen, Michael Salib, dan D. Sculley. 2017. “The ML Test Score: A Rubric for ML Production Readiness and Technical Debt Reduction”. Dalam *2017 IEEE International Conference on Big Data (Big Data)*, 1123–1132. IEEE. <https://doi.org/10.1109/BigData.2017.8258038>. <https://doi.org/10.1109/BigData.2017.8258038>.
- Bruch, Sebastian, Siyu Gai, dan Amir Ingber. 2023. “An Analysis of Fusion Functions for Hybrid Retrieval”. Dalam *ACM Transactions on Information Systems*. <https://doi.org/10.1145/3596512>.
- Burgueno-Romero, Anabel M., José L. Cánovas Izquierdo, dan José García-García. 2025. “Toward an Open Source MLOps Architecture”. *IEEE Software* 42 (1): 28–35. <https://doi.org/10.1109/MS.2024.3421675>. <https://doi.org/10.1109/MS.2024.3421675>.
- Campese, Stefano, Ivano Lauriola, dan Alessandro Moschitti. 2024. “Improving Document Retrieval Coherence for Semantically Equivalent Queries”. *arXiv preprint arXiv:2404.12338*.
- Carbone, Paris, Stephan Ewen, Kostas Tzoumas, Volker Markl, dan Fabian Hueske. 2015. “Apache Flink: Stream and Batch Processing in a Single Engine”. *Bulletin of the IEEE Computer Society Technical Committee on Data Engineering* 36 (4): 28–38. <https://asterios.katsifodimos.com/assets/publications/flink-deb.pdf>.

- cert-manager Contributors. 2025. *cert-manager: Automated TLS Certificate Management for Kubernetes*. Diakses 25 Mei 2026. <https://cert-manager.io/docs/>.
- Céspedes Sisniega, Jaime, Vicente Rodríguez, Germán Moltó, dan Álvaro López García. 2024. “Efficient and Scalable Covariate Drift Detection in Machine Learning Systems with Serverless Computing”. *Future Generation Computer Systems* 161:174–188. <https://doi.org/10.1016/j.future.2024.07.010>. <https://doi.org/10.1016/j.future.2024.07.010>.
- Chaos Mesh Authors. 2025. *Chaos Mesh: A Cloud-Native Chaos Engineering Platform*. Diakses 10 November 2025. <https://chaos-mesh.org/docs/>.
- Chien, Melody. February 2022. *How to Improve Your Data Quality*. Diakses 29 September 2025. Gartner. <https://www.gartner.com/smarterwithgartner/how-to-improve-your-data-quality>.
- ClickHouse Inc. 2024. *ClickHouse Documentation*. Diakses 5 November 2025. <https://clickhouse.com/docs/>.
- Cloud Native Computing Foundation. 2024a. *Kubernetes Documentation*. <https://kubernetes.io/docs/>.
- . 2024b. *Observability Whitepaper*. Diakses 3 November 2025. <https://github.com/cncf/tag-observability/blob/main/whitepaper.md>.
- CloudNativePG Contributors. 2025. *CloudNativePG: Kubernetes Operator for PostgreSQL*. Diakses 25 Mei 2026. <https://cloudnative-pg.io/documentation/>.
- DataHub Project. 2024. *DataHub Documentation*. Diakses 5 November 2025. <https://datahubproject.io/docs/>.
- dbt Labs. 2025. *dbt: Data Build Tool Documentation*. Diakses 25 Mei 2026. <https://docs.getdbt.com/>.
- Devlin, Jacob, Ming-Wei Chang, Kenton Lee, dan Kristina Toutanova. 2019. “BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding”. Dalam *Proceedings of the 2019 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies, Volume 1 (Long and Short Papers)*, 4171–4186. Association for Computational Linguistics. <https://doi.org/10.18653/v1/N19-1423>. <https://doi.org/10.18653/v1/N19-1423>.

- Dex Authors. 2025. *Dex: A Federated OpenID Connect Provider*. Diakses 10 November 2025. <https://dexidp.io/docs/>.
- Evidently AI. 2025. *Evidently: Open-Source ML Observability Platform*. Diakses 10 November 2025. <https://docs.evidentlyai.com/>.
- External Secrets Authors. 2025. *External Secrets Operator Documentation*. Diakses 25 Mei 2026. <https://external-secrets.io/latest/>.
- Falco Authors. 2025. *Falco: Cloud-Native Runtime Security*. Diakses 10 November 2025. <https://falco.org/docs/>.
- Feast Project. 2024. *Feast Documentation*. Diakses 5 November 2025. <https://docs.feast.dev/>.
- Fowler, Martin, dan James Lewis. 2014. *Microservices*. Diakses November 2025. <https://martinfowler.com/articles/microservices.html>.
- Gitea Contributors. 2025. *Gitea Documentation*. Diakses 10 November 2025. <https://docs.gitea.com/>.
- Google Cloud. 2024. *MLOps: Continuous Delivery and Automation Pipelines in Machine Learning*. Cloud Architecture Center. Diakses 2 November 2025. <https://cloud.google.com/architecture/mlops-continuous-delivery-and-automation-pipelines-in-machine-learning>.
- Grafana Labs. 2024a. *Grafana Documentation*. Diakses 5 November 2025. <https://grafana.com/docs/grafana/latest/>.
- . 2024b. *Grafana Loki Documentation*. Diakses 10 November 2025. <https://grafana.com/docs/loki/latest/>.
- . 2024c. *Grafana Tempo: High-Volume Distributed Tracing Backend*. Diakses 10 November 2025. <https://grafana.com/docs/tempo/latest/>.
- . 2025. *Grafana Pyroscope: Continuous Profiling*. Diakses 25 Mei 2026. <https://grafana.com/docs/pyroscope/latest/>.
- Great Expectations. 2024. *Great Expectations Documentation*. Diakses 10 November 2025. <https://docs.greatexpectations.io/docs/>.

- Hamza, Muhammad, Muhammad Azeem Akbar, dan Rafael Capilla. 2024. “Understanding Cost Dynamics of Serverless Computing: An Empirical Study”. Dalam *Software Business: 14th International Conference, ICSOB 2023, Proceedings*, 500:456–470. Lecture Notes in Business Information Processing. Springer. https://doi.org/10.1007/978-3-031-53227-6_32. https://doi.org/10.1007/978-3-031-53227-6_32.
- Hevner, Alan R., Salvatore T. March, Jinsoo Park, dan Sudha Ram. 2004. “Design Science in Information Systems Research”. *MIS Quarterly* 28 (1): 75–105. <https://doi.org/10.2307/25148625>. <https://doi.org/10.2307/25148625>.
- Hinder, Fabian, Valerie Vaquet, Johannes Brinkrolf, dan Barbara Hammer. 2023. “Model-Based Explanations of Concept Drift”. *Neurocomputing* 555:126640. <https://doi.org/10.1016/j.neucom.2023.126640>. <https://doi.org/10.1016/j.neucom.2023.126640>.
- Hsu, C. H., P. W. Liu, Y. C. Fan, dan C. Y. Lin. 2024. “End-to-End Automation of ML Model Lifecycle Management Using Machine Learning Operations Platforms”. Dalam *2024 International Conference on Consumer Electronics-Taiwan (ICCE-Taiwan)*, 1–6. IEEE. <https://doi.org/10.1109/ICCE-Taiwan62264.2024.10674445>. <https://doi.org/10.1109/ICCE-Taiwan62264.2024.10674445>.
- IBM. 2024. *What is Observability? Logs, Metrics, and Distributed Tracing*. Diakses 2 November 2025. <https://www.ibm.com/topics/observability>.
- Istio Authors. 2024. *Istio Service Mesh*. <https://istio.io/>.
- Jain, Souratn, dan Jyotipriya Das. 2025. “Integrating Data Engineering and MLOps for Scalable and Resilient Machine Learning Pipelines: Frameworks, Challenges, and Future Trends”. *World Journal of Advanced Engineering Technology and Sciences* 14 (1): 241–253. <https://doi.org/10.30574/wjaets.2025.14.1.0020>. <https://doi.org/10.30574/wjaets.2025.14.1.0020>.
- Jegou, Herve, Matthijs Douze, dan Cordelia Schmid. 2011. “Product Quantization for Nearest Neighbor Search”. *IEEE Transactions on Pattern Analysis and Machine Intelligence* 33 (1): 117–128. <https://doi.org/10.1109/TPAMI.2010.57>. <https://doi.org/10.1109/TPAMI.2010.57>.

- Johnson, Jeff, Matthijs Douze, dan Hervé Jégou. 2021. “Billion-Scale Similarity Search with GPUs”. *IEEE Transactions on Big Data* 7 (3): 535–547. <https://doi.org/10.1109/TBDATA.2019.2921572>. <https://doi.org/10.1109/TBDATA.2019.2921572>.
- Jourand, Nicolas, Tom Bayer, Tobias Biegel, dan Joachim Metternich. 2023. “Handling Concept Drift in Deep Learning Applications for Process Monitoring”. *Procedia CIRP* 120:42–47. <https://doi.org/10.1016/j.procir.2023.08.007>. <https://doi.org/10.1016/j.procir.2023.08.007>.
- Kapoor, Sayash, dan Arvind Narayanan. 2023. “Leakage and the Reproducibility Crisis in Machine-Learning-Based Science”. *Patterns* 4 (9): 100804. <https://doi.org/10.1016/j.patter.2023.100804>. <https://doi.org/10.1016/j.patter.2023.100804>.
- Kartakis, Sokratis, Giuseppe Angelo Porcelli, Georgios Schinas, dan Shelbee Eigenbrode. June 2022. “MLOps Foundation Roadmap for Enterprises with Amazon SageMaker”. Diakses 2 November 2025, *AWS Machine Learning Blog* (). <https://aws.amazon.com/blogs/machine-learning/mlops-foundation-roadmap-for-enterprises-with-amazon-sagemaker/>.
- KEDA Authors. 2025. *KEDA: Kubernetes Event-Driven Autoscaling*. Diakses 25 Mei 2026. <https://keda.sh/docs/>.
- Kim, Gene, Jez Humble, Patrick Debois, dan John Willis. 2016. *The DevOps Handbook: How to Create World-Class Agility, Reliability, and Security in Technology Organizations*. 1st. IT Revolution Press. ISBN: 978-1942788003. <https://itrevolution.com/product/the-devops-handbook/>.
- Kleppmann, Martin. 2017. *Designing Data-Intensive Applications*. O’Reilly Media. ISBN: 978-1491903063. <https://www.oreilly.com/library/view/designing-data-intensive-applications/9781491903063/>.
- Knative Authors. 2025. *Knative Serving: Serverless on Kubernetes*. Diakses 10 November 2025. <https://knative.dev/docs/serving/>.
- Kreps, Jay. 2014. *Questioning the Lambda Architecture*. Diakses 8 November 2025. O’Reilly Radar. <https://www.oreilly.com/radar/questioning-the-lambda-architecture/>.

- Kreuzberger, Dominik, Niklas Kühl, dan Sebastian Hirschl. 2023. “Machine Learning Operations (MLOps): Overview, Definition, and Architecture”. *IEEE Access* 11:31866–31879. <https://doi.org/10.1109/ACCESS.2023.3262138>. <https://doi.org/10.1109/ACCESS.2023.3262138>.
- KServe Contributors. 2024. *KServe Documentation*. Diakses 12 Oktober 2025. <https://kserve.github.io/website/>.
- Kubeflow Authors. 2025a. *Katib: Kubernetes-Native Hyperparameter Tuning*. Diakses 25 Mei 2026. <https://www.kubeflow.org/docs/components/katib/>.
- . 2025b. *Kubeflow Notebooks*. Diakses 25 Mei 2026. <https://www.kubeflow.org/docs/components/notebooks/>.
- . 2025c. *Kubeflow Trainer: Distributed Training Operators*. Diakses 25 Mei 2026. <https://www.kubeflow.org/docs/components/trainer/>.
- Kubeflow Contributors. 2024. *Kubeflow Documentation*. Diakses 29 September 2025. <https://www.kubeflow.org/docs/>.
- Kubernetes Autoscaling SIG. 2025. *Vertical Pod Autoscaler*. Diakses 25 Mei 2026. <https://github.com/kubernetes/autoscaler/tree/master/vertical-pod-autoscaler>.
- Kueue Authors. 2024. *Kueue: Job Queueing for Kubernetes*. Diakses 10 November 2025. <https://kueue.sigs.k8s.io/docs/>.
- Kyverno Authors. 2024. *Kyverno: Kubernetes Native Policy Management*. Diakses 10 November 2025. <https://kyverno.io/docs/>.
- Lakekeeper Authors. 2025. *Lakekeeper: Apache Iceberg REST Catalog*. Diakses 10 November 2025. <https://docs.lakekeeper.io/>.
- Lakka, Mounika. 2025. “Kubernetes for Enterprise: Mastering Cloud-Native Container Orchestration at Scale”. *European Modern Studies Journal* 9 (3): 358–367. [https://doi.org/10.59573/emsj.9\(3\).2025.31](https://doi.org/10.59573/emsj.9(3).2025.31). [https://doi.org/10.59573/emsj.9\(3\).2025.31](https://doi.org/10.59573/emsj.9(3).2025.31).
- Lamer, Antoine, Chloé Saint-Dizier, Nicolas Paris, dan Emmanuel Chazard. 2024. “Data Lake, Data Warehouse, Datamart, and Feature Store: Their Contributions to the Complete Data Reuse Pipeline”. *JMIR Medical Informatics* 12:e54590. <https://doi.org/10.2196/54590>. <https://doi.org/10.2196/54590>.

- Li, Anya, Bhala Ranganathan, Feng Pan, Mickey Zhang, Qianjun Xu, Runhan Li, Sethu Raman, Shail Paragbhai Shah, dan Vivienne Tang. 2023. “Managed Geo-Distributed Feature Store: Architecture and System Design”. *arXiv preprint arXiv:2305.20077*, <https://doi.org/10.48550/arXiv.2305.20077>. <https://arxiv.org/abs/2305.20077>.
- Liang, Penghao, Wei Chen, Yifan Zhang, dan Jun Liu. 2024. “Automating the Training and Deployment of Models in MLOps by Integrating Systems with Machine Learning”. *arXiv preprint arXiv:2405.09819*, <https://doi.org/10.48550/arXiv.2405.09819>. <https://arxiv.org/abs/2405.09819>.
- Limoncelli, Thomas. 2022. “GitOps: A Path to More Self-Service IT”. *Communications of the ACM* 65 (8): 43–49. <https://doi.org/10.1145/3233241>. <https://dl.acm.org/doi/10.1145/3233241>.
- Lu, Jie, Anjin Liu, Fan Dong, Feng Gu, Joao Gama, dan Guangquan Zhang. 2019. “Learning Under Concept Drift: A Review”. Versi cetak: *arXiv:2004.05785* (2020), *IEEE Transactions on Knowledge and Data Engineering* 31 (12): 2346–2363. <https://arxiv.org/abs/2004.05785>.
- Malkov, Yury A., dan D. A. Yashunin. 2020. “Efficient and Robust Approximate Nearest Neighbor Search Using Hierarchical Navigable Small World Graphs”. *IEEE Transactions on Pattern Analysis and Machine Intelligence* 42 (4): 824–836. <https://doi.org/10.1109/TPAMI.2018.2889473>. <https://doi.org/10.1109/TPAMI.2018.2889473>.
- Marz, Nathan, dan James Warren. 2015. *Big Data: Principles and Best Practices of Scalable Realtime Data Systems*. Manning Publications. ISBN: 978-1617290343. <https://www.manning.com/books/big-data>.
- McKnight, William. January 2020. *Delivering on the Vision of MLOps: A Maturity-Based Approach*. Diakses 2 November 2025. GigaOm. <https://gigaom.com/report/delivering-on-the-vision-of-mlops/>.
- Microsoft Azure. 2023. *Machine Learning Operations Maturity Model*. Azure Architecture Center. Diakses 2 November 2025. <https://learn.microsoft.com/en-us/azure/architecture/ai-ml/guide/mlops-maturity-model>.
- MinIO, Inc. 2024. *MinIO Documentation*. Diakses 5 November 2025. <https://min.io/docs/>.

- MinIO, Inc. 2025. *KES: Stateless Key Management Server*. Diakses 25 Mei 2026. <https://github.com/minio/kcs>.
- MLflow Contributors. 2024. *MLflow Documentation*. Diakses 1 November 2025. <https://mlflow.org/docs/latest/index.html>.
- Mo, Di, Robert Cordingley, Donald Chinn, dan Wes Lloyd. 2023. “Addressing Serverless Computing Vendor Lock-In through Cloud Service Abstraction”. Dalam *2023 IEEE International Conference on Cloud Computing Technology and Science (CloudCom)*, 193–199. IEEE. <https://doi.org/10.1109/CloudCom59040.2023.00040>. <https://doi.org/10.1109/CloudCom59040.2023.00040>.
- Munappy, Aiswarya Raj, Jan Bosch, Helena Holmström Olsson, Anders Arpteg, dan Björn Brinne. 2022. “Data Management for Production Quality Deep Learning Models: Challenges and Solutions”. *Journal of Systems and Software* 191:111359. <https://doi.org/10.1016/j.jss.2022.111359>. <https://doi.org/10.1016/j.jss.2022.111359>.
- OAuth2 Proxy Authors. 2025. *OAuth2 Proxy Documentation*. Diakses 10 November 2025. <https://oauth2-proxy.github.io/oauth2-proxy/>.
- Open Policy Agent Authors. 2024. *Open Policy Agent*. <https://www.openpolicyagent.org/>.
- OpenAI. December 2022. *New and Improved Embedding Model*. OpenAI Blog. Diakses 2 November 2025. <https://openai.com/index/new-and-improved-embedding-model/>.
- OpenBao Authors. 2025. *OpenBao: Open Source Secrets Management*. Diakses 10 November 2025. <https://openbao.org/docs/>.
- OpenCost Authors. 2025. *OpenCost: Open Source Cost Monitoring for Kubernetes*. Diakses 25 Mei 2026. <https://www.opencost.io/docs/>.
- OpenLineage Authors. 2025. *OpenLineage: An Open Standard for Lineage Metadata Collection*. Diakses 10 November 2025. <https://openlineage.io/docs/>.
- OpenSearch Foundation. 2025. *OpenSearch: Open Source Search and Analytics Suite*. Diakses 25 Mei 2026. <https://opensearch.org/docs/latest/>.
- Oracle Corporation. 2025. *MySQL Reference Manual*. Diakses 25 Mei 2026. <https://dev.mysql.com/doc/>.

- Paley, Andrei, Raoul-Gabriel Urma, dan Neil D. Lawrence. 2022. "Challenges in Deploying Machine Learning: A Survey of Case Studies". *ACM Computing Surveys* 55 (6): 1–29. <https://doi.org/10.1145/3533378>. <https://doi.org/10.1145/3533378>.
- Peffer, Ken, Tuure Tuunanen, Marcus A. Rothenberger, dan Samir Chatterjee. 2007. "A Design Science Research Methodology for Information Systems Research". *Journal of Management Information Systems* 24 (3): 45–77. <https://doi.org/10.2753/MIS0742-1222240302>. <https://doi.org/10.2753/MIS0742-1222240302>.
- Pimentel, João Felipe, Leonardo Murta, Vanessa Braganholo, dan Juliana Freire. 2019. "A Large-Scale Study About Quality and Reproducibility of Jupyter Notebooks". Dalam *2019 IEEE/ACM 16th International Conference on Mining Software Repositories (MSR)*, 507–517. IEEE. <https://doi.org/10.1109/MSR.2019.00077>. <https://doi.org/10.1109/MSR.2019.00077>.
- Polyzotis, Neoklis, Sudip Roy, Steven Euijong Whang, dan Martin Zinkevich. 2018. "Data Lifecycle Challenges in Production Machine Learning: A Survey". *ACM SIGMOD Record* 47 (2): 17–28. <https://doi.org/10.1145/3299887.3299891>. <https://doi.org/10.1145/3299887.3299891>.
- PostgreSQL Global Development Group. 2025. *PostgreSQL: The World's Most Advanced Open Source Relational Database*. Diakses 25 Mei 2026. <https://www.postgresql.org/docs/>.
- Prometheus Authors. 2024. *Prometheus Documentation*. Diakses 5 November 2025. <https://prometheus.io/docs/>.
- . 2025. *Prometheus Pushgateway*. Diakses 25 Mei 2026. <https://prometheus.io/docs/practices/pushing/>.
- Qdrant Team. 2025. *Qdrant: High-Performance Vector Search Engine*. Diakses 25 Mei 2026. <https://qdrant.tech/documentation/>.
- Reis, Denis Moreira dos, Peter Flach, Stan Matwin, dan Gustavo Batista. 2016. "Fast Unsupervised Online Drift Detection Using Incremental Kolmogorov-Smirnov Test". Dalam *Proceedings of the 22nd ACM SIGKDD International Conference on Knowledge Discovery and Data Mining*, 1545–1554. ACM. <https://doi.org/10.1145/2939672.2939836>. <https://doi.org/10.1145/2939672.2939836>.

- Rella, Bhanu Prakash Reddy. 2022. "MLOps and DataOps Integration for Scalable Machine Learning Deployment". *International Journal for Multidisciplinary Research* 4 (1). <https://www.ijfmr.com/papers/2022/1/39278.pdf>.
- Rodrigues, Miguel G., Eduardo K. Viegas, Altair O. Santin, dan Fabrício Enembreck. 2025. "A MLOps Architecture for Near Real-Time Distributed Stream Learning Operation Deployment". *Journal of Network and Computer Applications* 238:104169. <https://doi.org/10.1016/j.jnca.2025.104169>. <https://doi.org/10.1016/j.jnca.2025.104169>.
- Sculley, D., Gary Holt, Daniel Golovin, Eugene Davydov, Todd Phillips, Dietmar Ebner, Vinay Chaudhary, Michael Young, Jean-Francois Crespo, dan Dan Dennison. 2015. "Hidden Technical Debt in Machine Learning Systems". Dalam *Advances in Neural Information Processing Systems 28 (NIPS 2015)*, disunting oleh C. Cortes, N. D. Lawrence, D. D. Lee, M. Sugiyama, dan R. Garnett, 2503–2511. Red Hook, NY: Curran Associates, Inc. <https://proceedings.neurips.cc/paper/2015/file/86df7dcfd896fcdf2674f757a2463eba-Paper.pdf>.
- Shankar, Shreya, Rolando Garcia, Joseph M. Hellerstein, dan Aditya G. Parameswaran. 2022. *Operationalizing Machine Learning: An Interview Study*. ArXiv preprint arXiv:2209.09125. <https://doi.org/10.48550/arXiv.2209.09125>. <https://arxiv.org/abs/2209.09125>.
- Sloth Authors. 2025. *Sloth: Easy and Reliable SLOs for Prometheus*. Diakses 10 November 2025. <https://sloth.dev/>.
- Stopford, Ben. 2018. *Designing Event-Driven Systems: Concepts and Patterns for Streaming Services with Apache Kafka*. O'Reilly Media. ISBN: 978-1-4920-3819-5.
- Stoyanovich, Julia, Bill Howe, dan H. V. Jagadish. 2020. "Responsible Data Management". *Proceedings of the VLDB Endowment* 13 (12): 3474–3488. <https://doi.org/10.14778/3415478.3415570>. <https://doi.org/10.14778/3415478.3415570>.
- Strimzi Authors. 2025. *Strimzi: Apache Kafka on Kubernetes*. Diakses 25 Mei 2026. <https://strimzi.io/documentation/>.
- Tekton Contributors. 2024. *Tekton Pipelines Documentation*. Diakses 10 November 2025. <https://tekton.dev/docs/>.

- Treeverse Ltd. 2025. *lakeFS: Data Version Control for Object Storage*. Diakses 25 Mei 2026. <https://docs.lakefs.io/>.
- Trino Software Foundation. 2024. *Trino: Distributed SQL Query Engine*. <https://trino.io/>.
- Valkey Contributors. 2025. *Valkey: An Open Source In-Memory Data Store*. Diakses 25 Mei 2026. <https://valkey.io/docs/>.
- Vangala, Rajesh, Priya Mehta, dan Arjun Singh. 2022. *MLOps in Practice: A Framework for Scalable AI Model Deployment, Monitoring, and Retraining*. Technical Report. Diakses 1 November 2025. https://www.researchgate.net/publication/393096121_MLOps_in_Practice_A_Framework_for_Scalable_AI_Model_Deployment_Monitoring_and_Retraining.
- Vela, Daniel, Andrew Sharp, Richard Zhang, Trang Nguyen, An Hoang, dan Oleg S. Pianykh. 2022. "Temporal Quality Degradation in AI Models". *Scientific Reports* 12:11654. <https://doi.org/10.1038/s41598-022-15245-z>. <https://doi.org/10.1038/s41598-022-15245-z>.
- Velero Authors. 2025. *Velero: Backup and Migrate Kubernetes Resources and Persistent Volumes*. Diakses 10 November 2025. <https://velero.io/docs/>.
- Wang, Weiquan, dan Johannes Ruf. 2021. "Information Leakage in Backtesting". *SSRN Electronic Journal*, <https://doi.org/10.2139/ssrn.3836631>. <https://doi.org/10.2139/ssrn.3836631>.
- Zaharia, Matei, Reynold S. Xin, Patrick Wendell, Tathagata Das, Michael Armbrust, Ankur Dave, Xiangrui Meng, dkk. 2016. "Apache Spark: A Unified Engine for Big Data Processing". *Communications of the ACM* 59 (11): 56–65. <https://doi.org/10.1145/2934664>. <https://doi.org/10.1145/2934664>.

LAMPIRAN A

DAFTAR ARTEFAK PLATFORM

Lampiran ini merangkum lokasi dan tata letak artefak yang dirujuk oleh Bab IV dan Bab V. Kode program lengkap, *manifest* Kubernetes, *chart* Helm, dan berkas konfigurasi disimpan pada repositori Gitea internal yang menjadi sumber kebenaran bagi Argo CD. Berkas terkait *use case* pasar aset kripto sebagai instans uji disimpan pada repositori terpisah agar tata kelola domain tidak bercampur dengan *control plane* platform.

A.1 Tata Letak Repositori Platform

Repositori platform disusun mengikuti pemisahan komponen menjadi tujuh lapisan sebagaimana digambarkan pada Subbab IV.1. Direktori `platform/components/` memuat *manifest* setiap komponen dasar yang dipasang melalui Argo CD. Direktori `platform/services/` memuat layanan kustom yang ditulis untuk kebutuhan tata kelola, deteksi *drift*, dan layanan fitur. Direktori `platform/scripts/` memuat *script* bantu untuk bootstrap, nuke, dan penskalaan beban.

A.2 Tata Letak Repositori Use Case

Repositori *use case* pasar aset kripto memuat berkas *overlay* Kustomize yang mengikat *base manifest* platform dengan konfigurasi domain. Berkas `config/project.yaml` menyimpan parameter spesifik domain yang dirujuk oleh Tekton, Argo Workflows, dan Feast. Direktori `services/` memuat kode produsen data, transformator fitur, dan *trainer* yang dipanggil oleh *pipeline* pelatihan.

A.3 Contoh Berkas Konfigurasi Feast

Berkas `feature_store.yaml` mendeklarasikan *registry file-based* pada MinIO, *offline store* pada ClickHouse, *online store* fitur tabular pada Valkey melalui protokol RESP, dan indeks vektor pada Qdrant secara terpisah. Berkas ini diolah oleh layanan

fitur pada saat *materialization* batch maupun *streaming*. Rincian sintaks dijelaskan pada Subbab V.5.

LAMPIRAN B

HASIL VERIFIKASI USE CASE

Lampiran ini akan diisi dengan tangkapan layar dan keluaran perintah yang membuktikan jalur ujung ke ujung pada *use case* pasar aset kripto setelah evaluasi pada Bab VI terkonsolidasi. Bukti yang direncanakan meliputi:

1. Status sinkronisasi Argo CD untuk seluruh *Application* platform dan *use case* pada kondisi *Synced/Healthy*.
2. Hasil eksekusi *pipeline* pelatihan dari Kubeflow Pipelines beserta *run id* pada MLflow.
3. Hasil eksekusi *CronWorkflow retrain-on-drift* ketika ambang batas dilewati.
4. Hasil panggilan `get_online_features` terhadap Feast untuk fitur tabular dan fitur vektor.
5. Hasil panggilan prediksi terhadap *InferenceService* KServe pada *model-serving* dengan respons HTTP yang valid.
6. Cuplikan dasbor Grafana yang menunjukkan kelengkapan jejak *OpenTelemetry* dari ingestasi hingga prediksi.

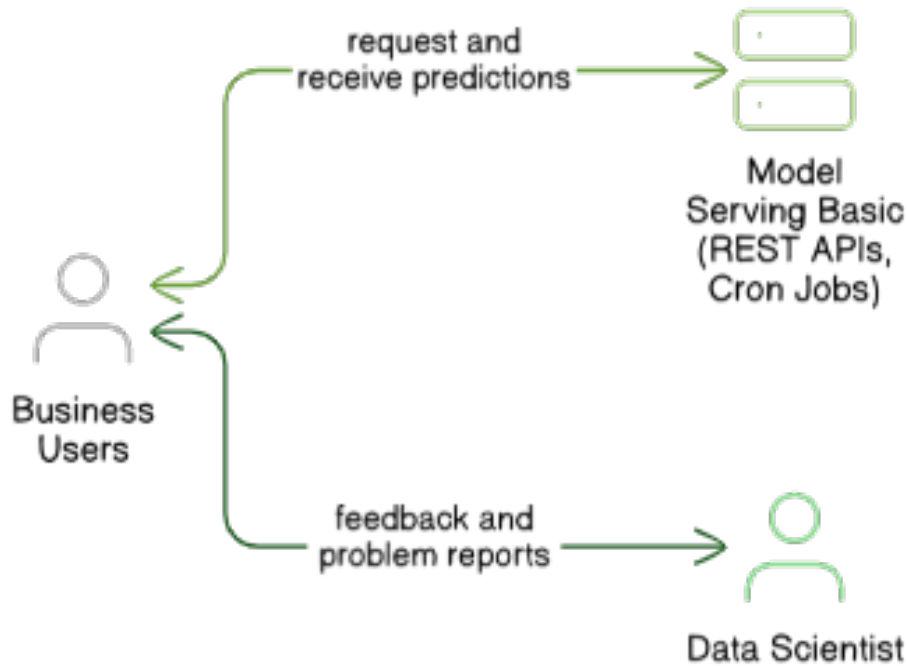
LAMPIRAN C

FRAGMENTASI ARSITEKTUR PER PERAN

Lampiran ini menyajikan rincian fragmentasi arsitektur DataOps dan MLOps pada empat peran utama yang dirujuk dari Subbab III.1.3. Pembahasan pada bab tersebut diringkas pada pandangan umum agar fokus tetap pada tiga sumber tekanan, yaitu beban konfigurasi platform dari nol, biaya berlangganan layanan terkelola, dan efek domino fragmentasi lintas peran. Detail per peran disajikan di sini sebagai bukti pendukung pemetaan kondisi saat ini.

C.1 Fragmentasi pada Peran *Business User*

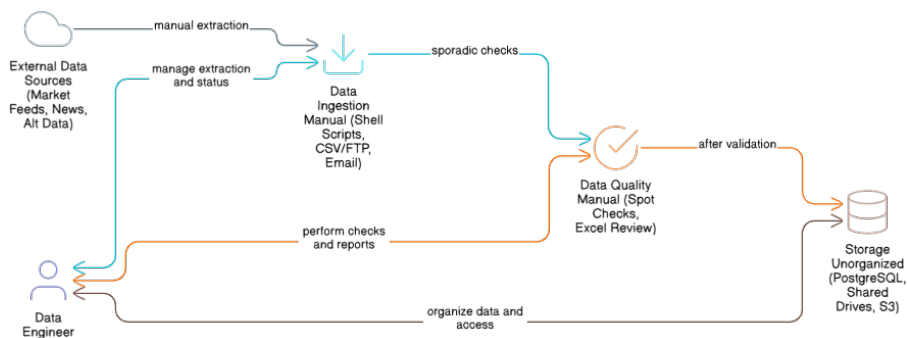
Peran *business user* biasanya berinteraksi dengan tumpukan visualisasi dan pelaporan yang terpisah dari sumber data analitik dan model. Gambar C.1 memperlihatkan posisi peran ini pada arsitektur fragmentasi dengan akses utama melalui *dashboard* dan laporan. Konsekuensinya, informasi tentang metrik *drift*, kualitas data, dan kebijakan akses sering tidak tersaji dalam satu antarmuka, sehingga keputusan bisnis mengandalkan ringkasan berkala yang tidak selalu terhubung dengan kondisi sistem terkini.



Gambar C.1 Fragmentasi pada Peran *Business User*

C.2 Fragmentasi pada Peran *Data Engineer*

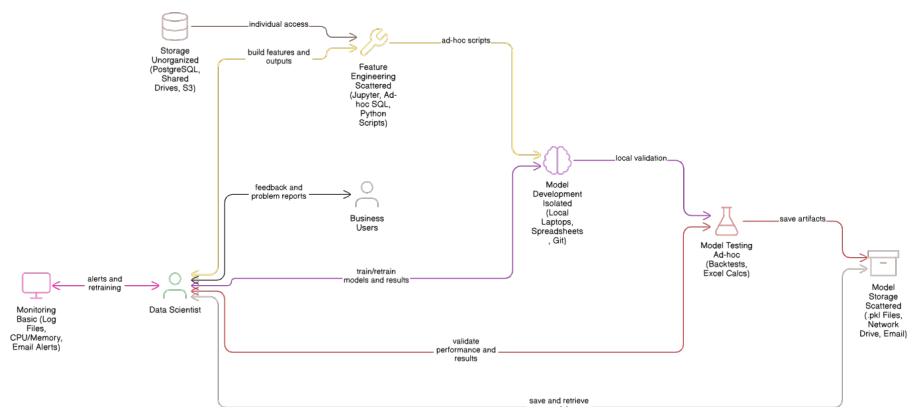
Data engineer bertanggung jawab atas ingestasi, pemrosesan, dan penyimpanan data. Gambar C.2 memperlihatkan luasnya cakupan tugas pada lapisan ingestasi, transformasi, dan tata kelola. Lamer dkk. (2024) menyoroti kompleksitas tata kelola data yang sering muncul ketika *tool* ingestasi, kualitas, dan katalog tidak terintegrasi pada satu sumber kebenaran. Akibatnya, kebijakan kualitas dan *lineage* antar lapisan data harus disinkronkan secara manual.



Gambar C.2 Fragmentasi pada Peran *Data Engineer*

C.3 Fragmentasi pada Peran *Data Scientist*

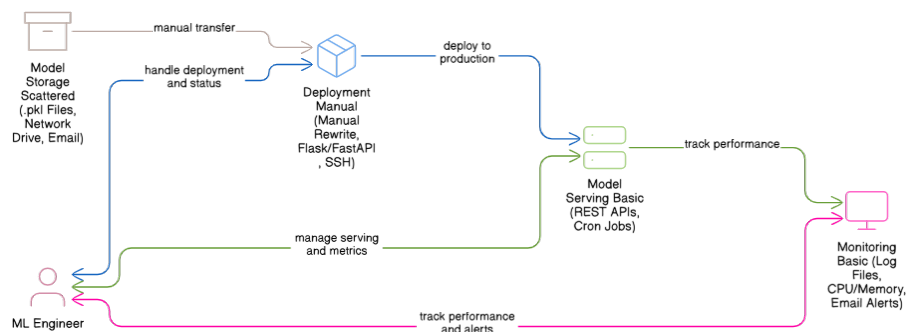
Peran *data scientist* menghabiskan sebagian besar waktu pada eksperimen dan rekayasa fitur. Gambar C.3 memperlihatkan ketergantungan pada *notebook*, *experiment tracking*, dan *feature engineering* yang sering berdiri sendiri. Anaconda, Inc. (July 2020) dan Pimentel dkk. (2019) memperlihatkan bahwa reproduksibilitas eksperimen terganggu apabila lingkungan eksekusi, versi data, dan versi kode tidak diatur dalam satu *pipeline* yang sama, dan kondisi ini diperburuk oleh *technical debt* yang khas pada sistem *machine learning* sebagaimana dipetakan oleh Sculley dkk. (2015).



Gambar C.3 Fragmentasi pada Peran *Data Scientist*

C.4 Fragmentasi pada Peran *Machine Learning Engineer*

Machine learning engineer bertugas membawa model dari fase eksperimen ke fase produksi. Gambar C.4 memperlihatkan kebutuhan pada *model registry*, *serving*, dan pemantauan model. Rella (2022) memperlihatkan dampak buruk yang muncul ketika siklus hidup model tidak terhubung dengan kontrol versi data dan fitur. Akibatnya, mekanisme *rollback*, *canary*, dan *retraining* tidak konsisten antar tim.



Gambar C.4 Fragmentasi pada Peran *Machine Learning Engineer*

